

EJEMPLAR DE LA OBRA

Registro de Programa de Cómputo — INDAUTOR

SMARTUR: Motor Híbrido de Recomendación Turística

Tipo de obra: Programa de cómputo

Autores

Martín Lara Olivares

Fernanda Pacheco Banda

Aarón Ochoa Ramírez

Universidad Tecnológica del Centro de Veracruz — Cuitláhuac, Veracruz, México

Contenido: código fuente del motor de recomendación (MODELO).
Este ejemplar contiene las primeras 10 y las últimas 10 páginas
del listado de código fuente, conforme al requisito de INDAUTOR.

```

===== MODELO/src/fusion.py =====
1 """
2 SMARTUR Fusion v4: Pipeline de dos etapas (Retrieval -> Ranking).
3
4 Fase A (Retrieval):
5 1. Pool amplio de ~200 candidatos vía KNN/Pearson
6 2. Filtro duro: elimina ítems que violen restricciones binarias
7 3. Filtro suave: prioriza categorías según tiposTurismo
8
9 Fase B (Ranking) – 4 señales:
10 - CF/SVD Pearson (warm users only)
11 - RF contextual [Item + User + Match features]
12 - LightFM WARP (cold-start + warm via feature embeddings)
13 - ContentModel TF-IDF (fallback cuando LightFM no disponible)
14
15 Blending (RF de-ponderado – el peor estimador en datos reales y sintéticos;
16 ver constantes LFM_W_* / RF_W_* más abajo):
17 Cold-start: 0.70 LightFM + 0.30 RF
18 Warm user: 0.40 LightFM + 0.35 CF + 0.25 RF
19 """
20
21 import numpy as np
22 import pandas as pd
23
24 from cf import REAL_SIGNAL_SOURCES, predict_cf_pearson_with_source
25 from context_encoder import MAPEO_CATEGORIAS
26 from poi_repository import fetch_all_items
27
28 # Center of Altas Montañas region – used as user location fallback
29 _ALTAS_MONTANAS_LAT = 18.95
30 _ALTAS_MONTANAS_LON = -97.05
31
32 # — Blend dinámico por densidad de datos reales (data_warmth ∈ [0,1]) —————
33 # El peso de la preferencia declarada (contenido) NO es fijo: depende de cuántos
34 # datos reales tiene el sistema. Con catálogo/uso frío (data_warmth=0), la
35 # preferencia DOMINA – es la única señal que discrimina cuando CF/RF aún no
36 # aprendieron nada (todos devuelven un valor plano). Conforme se acumulan
37 # interacciones reales (data_warmth=1), la preferencia baja a su peso "maduro"
38 # y los modelos aprendidos (CF/RF/LightFM) toman el control. Todo automático,
39 # sin ajustar pesos a mano.
40 # pref_weight = COLD - (COLD - WARM) * data_warmth
41 PREF_WEIGHT_COLD = 0.65 # frío: la preferencia manda
42 PREF_WEIGHT_WARM = 0.20 # maduro: peso "de fondo" (era el valor fijo anterior)
43
44 # Retrocompatibilidad: algunos módulos/tests importan esta constante. Equivale
45 # al peso maduro (warm). El valor efectivo lo calcula _preference_weight().
46 PREFERENCE_BOOST_WEIGHT = PREF_WEIGHT_WARM
47
48
49 # — Pesos del blend de servicio (bloque de modelos aprendidos) —————
50 # El Random Forest es, de forma CONSISTENTE, el peor estimador de rating: tanto
51 # en datos reales (RMSE 2.227 vs item_mean 0.883) como en el experimento
52 # sintético con estructura latente aprendible a propósito (item_mean 1.067 <
53 # CF 1.174 < RF 1.603, ver synthetic_persona_validation.py). Ni siquiera cuando
54 # hay estructura que aprender el RF/CF superan a un simple promedio. Por eso su
55 # peso en la recomendación SERVIDA se redujo a favor de LightFM (basado en
56 # features, el mejor en arranque en frío) y CF. Los tres modelos siguen
57 # construyéndose, entrenándose, evaluándose y comparándose en el dashboard
58 # (requisito académico intacto); esto solo ajusta cuánto influye cada uno en el
59 # score final que ve el usuario.

```

```

60 LFM_W_COLD, RF_W_COLD          = 0.70, 0.30      # antes 0.60 / 0.40
61 LFM_W_WARM, CF_W_WARM, RF_W_WARM = 0.40, 0.35, 0.25 # antes 0.30 / 0.30 / 0.40
62 RF_W_MAX_NOLFM                 = 0.50           # tope al RF sin LightFM (antes ~0.94)
63
64
65 def _preference_weight(data_warmth: float) -> float:
66     """Peso de la preferencia en el blend final según data_warmth ∈ [0,1].
67     Interpola linealmente entre PREF_WEIGHT_COLD (frío) y PREF_WEIGHT_WARM
68     (maduro). Fuera de rango se recorta a [0,1]."""
69     w = min(1.0, max(0.0, float(data_warmth)))
70     return PREF_WEIGHT_COLD - (PREF_WEIGHT_COLD - PREF_WEIGHT_WARM) * w
71
72 # presupuesto_bucket (fetch_traveler_profile, poi_repository.py) es un string;
73 # price_level en df_biz es numérico (misma escala 1-4 que usa rf_model.py para
74 # user_budget). Se mapea aquí para poder comparar ambos.
75 _BUDGET_BUCKET_LEVEL = {'bajo': 1, 'medio': 2, 'alto': 3, 'premium': 4}
76
77 # Keywords per tourism type for explanation tags
78 _TOURISM_KEYWORDS: dict[str, list[str]] = {
79     'naturaleza': ['park', 'hiking', 'waterfall', 'nature', 'botanical', 'outdoor', 'mountain',
80                   'volcano'],
81     'aventura': ['adventure', 'hiking', 'mountain', 'volcano', 'outdoor'],
82     'cultural': ['museum', 'history', 'monument', 'landmark', 'cathedral', 'market'],
83     'gastronomico': ['restaurant', 'food', 'gastronomy', 'cafe', 'market'],
84     'rural': ['hacienda', 'ranch', 'rural', 'countryside', 'sanctuary'],
85     'nocturno': ['bar', 'pub', 'nightlife', 'cantina', 'lounge', 'antro'],
86 }
87
88 def _compute_dist_km_series(df: pd.DataFrame, user_lat: float, user_lon: float) -> pd.Series:
89     """Haversine distance (km) from (user_lat, user_lon) to each row in df."""
90     R = 6371.0
91     lat1 = np.radians(user_lat)
92     lon1 = np.radians(user_lon)
93     lat2 = np.radians(df['latitude'].fillna(user_lat))
94     lon2 = np.radians(df['longitude'].fillna(user_lon))
95     dlat = lat2 - lat1
96     dlon = lon2 - lon1
97     a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
98     return (2 * R * np.arcsin(np.sqrt(a))).clip(0, 500)
99
100
101 def _build_reason_tags(
102     biz_id: str,
103     cats: str,
104     context: dict,
105     quality_scores: dict | None,
106     dist_km: float | None,
107 ) -> list[str]:
108     """
109     Generates up to 2 human-readable explanation tags for a recommendation:
110     1. Tourism-type match (e.g. "Coincide con naturaleza")
111     2. Quality signal (e.g. "Muy bien evaluado") - OR -
112     Distance signal (e.g. "A 3.2 km de ti")
113     """
114     tags: list[str] = []
115     cats_lower = cats.lower() if cats else ''
116     tipos: list[str] = context.get('tiposTurismo', []) or []
117
118     # Tourism match

```

```

119     for tipo in tipos:
120         keywords = _TOURISM_KEYWORDS.get(tipo, [])
121         if any(k in cats_lower for k in keywords):
122             tags.append(f"Coincide con {tipo}")
123             break
124
125     if len(tags) < 2:
126         # Quality signal (priority over distance when service is well evaluated)
127         q = (quality_scores or {}).get(biz_id, 0.0)
128         if q >= 0.7:
129             tags.append("Muy bien evaluado")
130         elif q >= 0.5:
131             tags.append("Bien evaluado")
132         # Distance signal (fallback when no quality data)
133         elif dist_km is not None:
134             if dist_km < 5:
135                 tags.append(f"A {dist_km:.1f} km de ti")
136             elif dist_km < 15:
137                 tags.append(f"Cerca ({dist_km:.0f} km)")
138
139     return tags[:2]
140
141 # -----
142 # Filtros
143 # -----
144
145 def filtro_duro(biz_df, context):
146     """
147     Elimina candidatos que violen restricciones binarias del usuario.
148     Son condiciones de 'no-negociable' (deal-breakers).
149     """
150     if not context:
151         return biz_df
152
153     filtered = biz_df.copy()
154
155     # Accesibilidad: si el usuario la requiere, eliminar negocios sin ella
156     if context.get('requiere_accesibilidad') is True:
157         if 'is_accessible' in filtered.columns:
158             filtered = filtered[filtered['is_accessible'] == 1]
159
160     # Outdoor: si el usuario lo prefiere como requisito duro
161     if context.get('pref_outdoor') is True:
162         if 'outdoor' in filtered.columns:
163             filtered = filtered[filtered['outdoor'] == 1]
164
165     # Hotel: si el usuario lo necesita como base
166     if context.get('needs_hotel') is True:
167         mask = filtered['categories'].str.contains('Hotels & Travel|Hotels', case=False, na=False)
168         filtered = filtered[mask]
169
170     # Comida: si el usuario NO quiere comida
171     if context.get('pref_food') is False:
172         mask = ~filtered['categories'].str.contains('Restaurants|Food', case=False, na=False)
173         filtered = filtered[mask]
174
175     return filtered
176
177
178 def _diversify(recs, biz_cat_lookup, top_n, max_per_main_cat=2):

```

```

179 """
180 Re-rankea la lista ordenada por score para que el top-N tenga
181 máximo max_per_main_cat ítems con la misma categoría principal.
182 Mantiene el orden de relevancia y solo desplaza los duplicados al final.
183 """
184 cat_counts = {}
185 selected = []
186 overflow = []
187
188 for rec in recs:
189     cats_str = str(biz_cat_lookup.get(rec['item_id'], ''))
190     main_cat = cats_str.split(',')[0].strip() if cats_str else 'Unknown'
191     if cat_counts.get(main_cat, 0) < max_per_main_cat:
192         selected.append(rec)
193         cat_counts[main_cat] = cat_counts.get(main_cat, 0) + 1
194     else:
195         overflow.append(rec)
196     if len(selected) >= top_n:
197         break
198
199 if len(selected) < top_n:
200     selected.extend(overflow[:top_n - len(selected)])
201 return selected[:top_n]
202
203
204 def _categorias_para_tipos(tipos: list[str]) -> list[str]:
205     """Expande una lista de tiposTurismo a sus palabras clave de categoría
206     (MAPEO_CATEGORIAS) – compartido entre el filtro suave (legado) y el
207     boost de preferencia."""
208     categorias: list[str] = []
209     for tipo in tipos or []:
210         categorias.extend(MAPEO_CATEGORIAS.get(tipo, []))
211     return categorias
212
213
214 def filtrar_candidatos_por_contexto(biz_df, context):
215     """Filtro suave: prioriza negocios cuyas categorías coincidan con tiposTurismo."""
216     if not context:
217         return biz_df
218
219     filtered_df = biz_df.copy()
220
221     if 'tiposTurismo' in context and context['tiposTurismo']:
222         yelp_categories = _categorias_para_tipos(context['tiposTurismo'])
223         if yelp_categories:
224             mask = filtered_df['categories'].str.contains(
225                 '|'.join(yelp_categories), case=False, na=False
226             )
227             # Si ningún candidato coincide, mantener todos (degradación graciosa)
228             if mask.any():
229                 filtered_df = filtered_df[mask]
230
231     return filtered_df
232
233
234 # Categorías que cuentan como "turismo sostenible" – mismo criterio que ya
235 # se usa para tiposTurismo naturaleza/rural (áreas naturales, comunidades
236 # rurales), no hay una bandera dedicada de sostenibilidad a nivel de ítem.
237 _SUSTAINABLE_TIPOS = ['naturaleza', 'rural']
238

```

```

239
240 def preference_match_score(
241     categories: str,
242     context: dict | None,
243     price_level: float | None = None,
244     is_romantic: int = 0,
245     is_good_for_kids: int = 0,
246     outdoor: int = 0,
247 ) -> float:
248     """
249     Qué tan bien encaja un candidato con las preferencias DECLARADAS por el
250     usuario (perfil real), independiente de lo que CF/RF hayan aprendido de
251     calificaciones. Devuelve un valor en [0, 1]:
252     - 0.45 si la categoría coincide con algún tiposTurismo declarado
253     - 0.20 según qué tan cerca esté price_level del presupuesto_bucket
254       (penalización gradual, no todo-o-nada)
255     - 0.10 si el tipo de grupo (group_type) coincide con las banderas
256       is_romantic/is_good_for_kids del ítem
257     - 0.15 si preferred_place ('aire'/'cerrado') coincide con el flag
258       outdoor del ítem
259     - 0.10 si sustainable_preferences está activo y la categoría es de
260       naturaleza/rural
261
262     Estas dos últimas señales ('preferred_place', 'sustainable_preferences')
263     ya se le preguntan al usuario en el perfil (traveler_profile) y se
264     guardan en la BD, pero antes nunca llegaban al motor de recomendación
265     – se pedían y se ignoraban.
266
267     A diferencia de filtrar_candidatos_por_contexto (que excluye candidatos
268     y puede degradar en silencio a "sin filtro" si el match da cero), esto
269     nunca elimina candidatos – solo reordena empujando hacia arriba lo que
270     sí coincide, así que la preferencia declarada siempre tiene efecto en el
271     ranking aunque el catálogo de categorías esté mal etiquetado.
272     """
273     if not context:
274         return 0.0
275
276     score = 0.0
277     cats_lower = (categories or '').lower()
278
279     # Tipo de turismo (peso mayor)
280     tipos = context.get('tiposTurismo') or []
281     keywords = _categorias_para_tipos(tipos)
282     if keywords and any(k.lower() in cats_lower for k in keywords):
283         score += 0.45
284
285     # Presupuesto (peso menor, gradual)
286     bucket = context.get('presupuesto_bucket')
287     bucket_level = _BUDGET_BUCKET_LEVEL.get(bucket) if bucket else None
288     if bucket_level is not None and price_level is not None:
289         try:
290             delta = abs(float(price_level) - bucket_level)
291             score += 0.20 * max(0.0, 1.0 - delta / 3.0)
292         except (TypeError, ValueError):
293             pass
294
295     # Tipo de grupo (peso menor)
296     group_type = (context.get('group_type') or '').lower()
297     if group_type == 'familia' and is_good_for_kids:
298         score += 0.10

```

```

299     elif group_type == 'pareja' and is_romantic:
300         score += 0.10
301
302     # Lugar preferido (aire libre / cerrado) – 'indiferente' no suma ni resta.
303     preferred_place = (context.get('preferred_place') or 'indiferente').lower()
304     if preferred_place == 'aire' and outdoor:
305         score += 0.15
306     elif preferred_place == 'cerrado' and not outdoor:
307         score += 0.15
308
309     # Preferencia de turismo sostenible
310     if context.get('sustainable_preferences'):
311         sustainable_keywords = _categorias_para_tipos(_SUSTAINABLE_TIPOS)
312         if sustainable_keywords and any(k.lower() in cats_lower for k in sustainable_keywords):
313             score += 0.10
314
315     return min(1.0, score)
316
317
318 def _resolve_cf_score(
319     user_id,
320     biz_id,
321     engine,
322     *,
323     matrix_col_set: set,
324     local_id_set: set,
325     quality_scores: dict | None,
326     global_mean_rating: float,
327 ) -> tuple[float, str]:
328     """
329     Decide la señal colaborativa de un candidato y de dónde salió.
330
331     ORDEN IMPORTANTE – aquí vivía un bug estructural: antes se preguntaba
332     PRIMERO 'if biz_id in local_id_set' (y se usaba el proxy de calidad).
333     Como en producción TODOS los candidatos son POIs locales, esa rama
334     siempre ganaba y 'predict_cf_pearson' NUNCA se ejecutaba para un lugar
335     recomendado – el CF-KNN estaba muerto y no podía despertar por más
336     interacciones reales que se acumularan.
337
338     Ahora se intenta el CF SIEMPRE primero, y el proxy de calidad solo entra
339     cuando el CF no tiene señal real (devuelve 'fallback_mean', o sea un
340     promedio sin información). Así el CF-KNN se activa solo en cuanto haya
341     interacciones reales sobre POIs locales: los IDs ya coinciden (poi_N /
342     svc_N tanto en fetch_all_items como en fetch_real_interactions) y esas
343     interacciones ya se mezclan al entrenamiento del engine.
344
345     Returns:
346         (score, signal) donde signal ∈ {'knn', 'svd', 'quality_proxy', 'global_mean'}
347         – 'knn'/'svd' significan que el CF realmente aportó.
348     """
349     if engine is not None and biz_id in matrix_col_set:
350         cf_pred, cf_source = predict_cf_pearson_with_source(user_id, biz_id, engine)
351         if cf_source in REAL_SIGNAL_SOURCES:
352             return cf_pred, cf_source
353
354     # Sin señal CF real todavía.
355     if biz_id in local_id_set:
356         # Proxy: evaluación de calidad del admin (service_evaluation) – así al
357         # menos los POIs locales se diferencian entre sí en vez de compartir
358         # un score plano.

```

```

359         q = (quality_scores or {}).get(biz_id)
360         if q is not None:
361             return 3.0 + 2.0 * q, 'quality_proxy'
362
363     return global_mean_rating, 'global_mean'
364
365
366 # -----
367 # Pipeline principal
368 # -----
369
370 def recommend_hybrid(
371     user_id, engine, context_model,
372     alpha=0.4, context=None, top_n=5,
373     lightfm_model=None, content_model=None,
374     quality_scores=None, data_warmth=0.0,
375 ):
376     """
377     Sistema Híbrido SMARTUR v4 (Retrieval + Ranking multi-señal).
378
379     Modo producción (local POIs disponibles):
380     - Candidatos: SOLO POIs de la BD local (Altas Montañas, Veracruz)
381     - Ranking: LightFM + RF (cold-start); LightFM + CF + RF (warm)
382
383     Modo desarrollo/fallback (sin BD local):
384     - Candidatos: 200 negocios Yelp vía KNN
385     - Ranking:  $\alpha \times CF + (1-\alpha) \times RF$  (+ LightFM si disponible)
386     """
387     # — Fase A: Retrieval -----
388     try:
389         local_pois = fetch_all_items()
390         local_biz = context_model.prepare_local_items(local_pois)
391     except Exception:
392         local_biz = pd.DataFrame()
393
394     if not local_biz.empty:
395         # Producción: POIs locales como pool exclusivo.
396         # Usamos un alpha reducido (no cero) para que la señal latente de CF/SVD
397         # siga contribuyendo al score final, especialmente con el fallback SVD.
398         local_biz = context_model.add_category_features(local_biz)
399         biz_candidates = local_biz
400         try:
401             from model_metrics import load_metrics as _lm
402             _saved_alpha = float(_lm().get('best_alpha', 0.2))
403             # Reducir alpha a 30% del óptimo global para dar más peso al RF contextual
404             # que sí fue entrenado con features de los POIs locales.
405             effective_alpha = max(0.05, _saved_alpha * 0.3)
406         except Exception:
407             effective_alpha = 0.1 # fallback conservador
408     else:
409         # Fallback desarrollo: candidatos Yelp KNN
410         if engine is None:
411             return [] # no local POIs and no engine -> cannot recommend
412         candidate_ids = engine.get_candidate_pool(user_id, top_n=200)
413         biz_candidates = engine.df_biz[engine.df_biz['business_id'].isin(candidate_ids)]
414         effective_alpha = alpha
415
416     # Filtro duro (restricciones binarias – deal-breakers reales, ej. accesibilidad)
417     refined_df = filtro_duro(biz_candidates, context)
418

```

```

419 # Fallback: si el filtro duro vació la lista, usar pool sin filtrar
420 if refined_df.empty:
421     refined_df = biz_candidates
422
423 # NOTA: el filtro suave por tiposTurismo (filtrar_candidatos_por_contexto)
424 # ya NO se aplica aquí – excluía candidatos por texto libre y podía
425 # degradar en silencio a "sin filtro" si el match daba cero resultados
426 # (categories_raw/categories_mapped son texto sin validación fuerte).
427 # En su lugar, la preferencia declarada se usa como boost de score más
428 # abajo (preference_match_score) – nunca excluye, siempre tiene efecto.
429
430 final_ids = refined_df['business_id'].tolist()
431
432 # — Fase B: Ranking —————
433 ref_df = local_biz if not local_biz.empty else (engine.df_biz if engine is not None else pd.DataFrame())
434 rf_scores = context_model.predict_with_context(final_ids, user_context=context, df_biz_override=ref_df)
435 rf_map = dict(zip(final_ids, rf_scores))
436
437 biz_cat_lookup = ref_df.set_index('business_id')['categories'].to_dict()
438 all_biz_names = ref_df.set_index('business_id')['name'].to_dict()
439 biz_kind_lookup = ref_df.set_index('business_id')['kind'].to_dict() if 'kind' in ref_df.columns else {}
440 biz_desc_lookup = ref_df.set_index('business_id')['description'].to_dict() if 'description' in ref_df.columns else {}
441 local_id_set = set(local_biz['business_id']) if not local_biz.empty else set()
442 matrix_col_set = set(engine.user_item_matrix_columns) if engine is not None else set()
443
444 # Lookups para el boost de preferencia (preference_match_score)
445 price_lookup = ref_df.set_index('business_id')['price_level'].to_dict() if 'price_level' in ref_df.columns else {}
446 is_romantic_lookup = ref_df.set_index('business_id')['is_romantic'].to_dict() if 'is_romantic' in ref_df.columns else {}
447 is_kids_lookup = ref_df.set_index('business_id')['is_good_for_kids'].to_dict() if 'is_good_for_kids' in ref_df.columns else {}
448 outdoor_lookup = ref_df.set_index('business_id')['outdoor'].to_dict() if 'outdoor' in ref_df.columns else {}
449
450 # — Distance map for explanation tags —————
451 # Use user-supplied lat/lon from context; fall back to region center.
452 dist_km_map: dict[str, float] = {}
453 if not ref_df.empty and 'latitude' in ref_df.columns and 'longitude' in ref_df.columns:
454     try:
455         user_lat = float(context.get('lat') or ALTAS_MONTANAS_LAT)
456         user_lon = float(context.get('lon') or ALTAS_MONTANAS_LON)
457         distances = _compute_dist_km_series(ref_df, user_lat, user_lon)
458         dist_km_map = dict(zip(ref_df['business_id'], distances))
459     except Exception:
460         pass # explanation tags gracefully degrade without distance
461
462 # — LightFM or ContentModel scores —————
463 # Determine cold-start status for blending weights
464 is_cold_start = str(user_id) not in (
465     getattr(lightfm_model, '_known_users', set()) if lightfm_model else set()
466 ) and str(user_id) not in (getattr(engine, 'user_index', {}) or {})
467
468 if lightfm_model is not None and getattr(lightfm_model, 'is_fitted', False):
469     lfm_scores = lightfm_model.predict(str(user_id), final_ids, user_context=context)
470     lfm_map = dict(zip(final_ids, lfm_scores))

```

```

471     use_content = False
472 elif content_model is not None and getattr(content_model, 'is_fitted', False):
473     cb_scores = content_model.score(final_ids, user_context=context)
474     lfm_map = dict(zip(final_ids, cb_scores))
475     use_content = True
476 else:
477     lfm_map = {}
478     use_content = False
479
480 recommendations = []
481
482 global_mean_rating = engine.train_data['stars'].mean() if engine is not None else 3.5
483
484 # Peso dinámico de la preferencia: alto en frío (domina), bajo cuando hay
485 # datos reales suficientes (los modelos aprendidos toman el control).
486 pref_weight = _preference_weight(data_warmth)
487
488 for biz_id in final_ids:
489     # — Señal colaborativa (CF-KNN) con fallback a calidad —————
490     # ORDEN IMPORTANTE. Antes la rama `if biz_id in local_id_set` iba
491     # PRIMERO y siempre ganaba: como en producción todos los candidatos
492     # son POIs locales, predict_cf_pearson NUNCA se ejecutaba para un
493     # lugar recomendado — el CF-KNN estaba estructuralmente muerto y no
494     # podía despertar por más datos reales que se acumularan.
495     #
496     # Ahora se intenta CF primero SIEMPRE, y solo se usa el proxy de
497     # calidad (evaluación del admin) cuando el CF no tiene señal real
498     # (devuelve 'fallback_mean', o sea un promedio sin información).
499     # Así el CF se activa solo en cuanto haya interacciones reales sobre
500     # POIs locales — los IDs ya coinciden (poi_N / svc_N tanto en
501     # fetch_all_items como en fetch_real_interactions), y esas
502     # interacciones ya se mezclan al entrenamiento del engine.
503     score_cf, cf_signal = _resolve_cf_score(
504         user_id, biz_id, engine,
505         matrix_col_set=matrix_col_set,
506         local_id_set=local_id_set,
507         quality_scores=quality_scores,
508         global_mean_rating=global_mean_rating,
509     )
510
511     score_rf = rf_map.get(biz_id, 3.0)
512     score_lfm = lfm_map.get(biz_id, 3.0)
513     cats = biz_cat_lookup.get(biz_id, '')
514
515     # Blending weights (RF de-ponderado — ver constantes arriba):
516     # Cold-start user -> LightFM (feature-based) domina, RF de apoyo
517     # Warm user -> LightFM + CF por delante, RF de apoyo
518     if lfm_map:
519         if is_cold_start:
520             final_score = LFM_W_COLD * score_lfm + RF_W_COLD * score_rf
521         else:
522             final_score = LFM_W_WARM * score_lfm + CF_W_WARM * score_cf + RF_W_WARM * score_rf
523     else:
524         # LightFM not available -> classic blend, pero con tope al RF para no
525         # dejar que el peor estimador domine (antes llegaba a ~0.94).
526         rf_w = min(RF_W_MAX_NOLFM, 1 - effective_alpha)
527         final_score = ((1 - rf_w) * score_cf) + (rf_w * score_rf)
528
529 # Boost de preferencia declarada (perfil real del usuario), no de lo
530 # que CF/RF aprendieron de calificaciones. Nunca excluye candidatos —

```

```

531     # solo empuja hacia arriba lo que coincide con tiposTurismo/presupuesto/
532     # grupo, así que la preferencia siempre influye en el ranking aunque
533     # el modelo no tenga suficientes datos para haberla aprendido solo.
534     score_pref = preference_match_score(
535         cats, context,
536         price_level=price_lookup.get(biz_id),
537         is_romantic=is_romantic_lookup.get(biz_id, 0),
538         is_good_for_kids=is_kids_lookup.get(biz_id, 0),
539         outdoor=outdoor_lookup.get(biz_id, 0),
540     )
541     final_score = (
542         (1 - pref_weight) * final_score
543         + pref_weight * (score_pref * 5.0)
544     )
545
546     # Quality boost from admin service_evaluation (service_evaluation.total_score).
547     # Non-linear: 0 boost for quality ≤ 0.5 (score ≤ 50/100), +0.3 max at quality = 1.0.
548     # This rewards well-evaluated services without demoting unevaluated POIs.
549     if quality_scores:
550         q = quality_scores.get(biz_id, 0.0)
551         if q > 0.5:
552             quality_boost = 0.3 * (q - 0.5) / 0.5 # linear: 0 @ q=0.5, 0.3 @ q=1.0
553             final_score = min(5.0, final_score + quality_boost)
554
555     nombre = all_biz_names.get(biz_id, 'Desconocido')
556     kind = biz_kind_lookup.get(biz_id, 'poi')
557     desc = biz_desc_lookup.get(biz_id, '') or ''
558
559     recommendations.append({
560         'item_id': str(biz_id),
561         'title': str(nombre),
562         'description': str(desc),
563         'category': str(cats),
564         'score': float(round(final_score, 3)),
565         'pred_cf': float(round(score_cf, 3)),
566         # De dónde salió pred_cf: 'knn'/'svd' = CF real aportó;
567         # 'quality_proxy'/'global_mean' = CF aún sin señal (esperando
568         # interacciones reales sobre este POI). Permite ver en /metrics
569         # cuándo el CF-KNN por fin despierta, sin adivinar.
570         'cf_signal': cf_signal,
571         'pred_rf': float(round(score_rf, 3)),
572         'pred_pref': float(round(score_pref, 3)),
573         'kind': kind,
574         'reason_tags': _build_reason_tags(
575             biz_id, cats, context or {},
576             quality_scores, dist_km_map.get(biz_id),
577         ),
578     })
579
580     sorted_recs = sorted(recommendations, key=lambda x: x['score'], reverse=True)
581     return _diversify(sorted_recs, biz_cat_lookup, top_n)

```

```

===== MODELO/src/cf.py =====
1 import numpy as np
2
3 # Contador de qué "camino" toma cada predicción de CF – instrumentación para
4 # medir qué tan seguido el KNN cae en el fallback constante (media global/del
5 # usuario) en vez de predecir con señal real. Con 198K usuarios y 233 items
6 # en el dataset de Yelp, la sospecha es que la mayoría de las predicciones
7 # de evaluación caen en 'fallback_mean', lo que explicaría por qué el

```

```

8 # baseline (predecir la media) le gana a CF en RMSE. No es thread-safe a
9 # propósito: los loops de evaluación (compare algorithms, _compute_simple_
10 # metrics) corren secuenciales en un solo proceso.
11 _prediction_source_counts = {'knn': 0, 'svd': 0, 'fallback_mean': 0}
12
13
14 def reset_prediction_stats():
15     """Reinicia los contadores – llamar antes de un batch de evaluación."""
16     global _prediction_source_counts
17     _prediction_source_counts = {'knn': 0, 'svd': 0, 'fallback_mean': 0}
18
19
20 def get_prediction_stats() -> dict:
21     """Devuelve los contadores acumulados desde el último reset, más el %
22     de predicciones que NO tuvieron señal real de vecinos (fallback_mean)."""
23     counts = dict(_prediction_source_counts)
24     total = sum(counts.values())
25     counts['total'] = total
26     counts['fallback_rate'] = round(counts['fallback_mean'] / total, 4) if total > 0 else None
27     return counts
28
29
30 # Fuentes que cuentan como SEÑAL REAL (el CF de verdad aportó algo):
31 # 'knn' = vecinos que sí calificaron este ítem; 'svd' = factorización latente.
32 # 'fallback_mean' NO es señal – es solo un promedio de relleno.
33 REAL_SIGNAL_SOURCES = ('knn', 'svd')
34
35
36 def _predict_cf_pearson_impl(user_id, item_id, engine, k=20):
37     """
38     Núcleo del CF. Devuelve (score, source) donde source ∈
39     {'knn', 'svd', 'fallback_mean'} – así los llamadores pueden distinguir
40     una predicción con señal real de un simple promedio de relleno.
41     """
42     user_idx = engine.get_user_idx(user_id)
43     item_idx = engine.get_biz_idx(item_id)
44
45     if user_idx is None or item_idx is None:
46         # Si el usuario o el ítem solicitado sufren de cold-start total
47         return float(engine.train_data['stars'].mean()), 'fallback_mean'
48
49     user_vector = engine.matrix_centered[user_idx]
50     n_neighbors = min(k + 1, engine.matrix_centered.shape[0])
51     distances, indices = engine.knn_model.kneighbors(
52         user_vector, n_neighbors=n_neighbors
53     )
54
55     neighbor_indices = indices[0]
56     similarities = 1 - distances[0]
57
58     weighted_sum = 0.0
59     sim_sum = 0.0
60
61     for i, neighbor_id in enumerate(neighbor_indices):
62         if neighbor_id == user_idx:
63             continue
64
65         rating = engine.user_item_matrix[neighbor_id, item_idx]
66         if rating > 0:
67             diff = engine.matrix_centered[neighbor_id, item_idx]

```

```

68         weighted_sum += similarities[i] * diff
69         sim_sum += abs(similarities[i])
70
71     user_mean = float(engine.user_means[user_id])
72     if np.isnan(user_mean):
73         user_mean = float(engine.train_data['stars'].mean())
74
75     if sim_sum == 0:
76         # No KNN neighbors rated this item -> try SVD dot-product for a better estimate
77         if hasattr(engine, 'user_latent') and hasattr(engine, 'item_latent'):
78             u = engine.user_index.get(user_id)
79             it = engine.item_index.get(item_id)
80             if u is not None and it is not None:
81                 try:
82                     svd_pred = float(np.dot(engine.user_latent[u], engine.item_latent[it]))
83                     # SVD output is on centered scale; shift back to [1, 5]
84                     svd_pred = float(np.clip(svd_pred + user_mean, 1, 5))
85                     if not np.isnan(svd_pred):
86                         return svd_pred, 'svd'
87                     return user_mean, 'fallback_mean'
88                 except Exception:
89                     pass
90             return user_mean, 'fallback_mean'
91
92     prediction = user_mean + (weighted_sum / sim_sum)
93     result = float(np.clip(prediction, 1, 5))
94     if np.isnan(result):
95         return user_mean, 'fallback_mean'
96     return result, 'knn'
97
98
99 def predict_cf_pearson_with_source(user_id, item_id, engine, k=20):
100     """
101     Igual que predict_cf_pearson pero devuelve (score, source).
102
103     Permite al ranking (fusion.recommend.hybrid) decidir si el CF realmente
104     tuvo algo que decir sobre este par usuario-ítem, o si solo devolvió un
105     promedio – en cuyo caso conviene usar otra señal (ej. la evaluación de
106     calidad del admin) en lugar de un número sin información.
107     """
108     score, source = _predict_cf_pearson_impl(user_id, item_id, engine, k)
109     _prediction_source_counts[source] += 1
110     return score, source
111
112
113 def predict_cf_pearson(user_id, item_id, engine, k=20):
114     """
115     Predicción utilizando Filtrado Colaborativo Basado en Memoria (K-NN user-based approach).
116     Predice la puntuación (1 a 5 estrellas) matemática extrapolada que un `user_id` le otorgaría
117     a un `item_id` determinado, en base a su similitud vectorial (con coseno geométrico) respecto
118     a una vecindad con los K-usuarios más similares a su perfil.
119
120     Fórmula Matemática general:
121     
$$\text{Pred} = \text{Media}_u + \frac{\sum(\text{Sim} \cdot (\text{Rating}_v - \text{Media}_v))}{\sum|\text{Sim}|}$$

122
123     Args:
124         user_id (str): Identificador textual en la db del usuario objetivo.
125         item_id (str): Identificador de Yelp o negocio a calificar.
126         engine (SmarturEngine): Referencia general en memoria al Motor construido con scikit Spars
127         e matrix.

```

```

127         k (int): Cantidad máxima de vecinos a buscar.
128
129     Returns:
130         float: Una predicción de puntaje inferido delimitado (clip) entre 1.0 y 5.0.
131             Devuelve por defecto la media del individuo si no hay datos de vecinos disponibles.
132             (Usa predict_cf_pearson_with_source si necesitas saber si hubo señal real.)
133     """
134     score, _ = predict_cf_pearson_with_source(user_id, item_id, engine, k)
135     return score

===== MODELO/src/engine.py =====
1 import logging
2 import os
3 import pandas as pd
4 import numpy as np
5 from sklearn.neighbors import NearestNeighbors
6 from sklearn.decomposition import TruncatedSVD
7 from sklearn.model_selection import train_test_split
8 from scipy.sparse import csr_matrix
9
10 logger = logging.getLogger(__name__)
11
12 _DIR = os.path.dirname(os.path.abspath(__file__))
13 _DATA = os.path.join(_DIR, '..', 'data')
14
15 # Mínimo de interacciones por usuario para entrar al set de entrenamiento
16 # ("k-core filtering"). Con ~198K usuarios y solo 233 negocios en el dataset
17 # base, la matriz usuario-item es extremadamente dispersa: la mayoría de los
18 # usuarios calificaron 1-2 negocios, así que el KNN casi nunca encuentra
19 # vecinos con solapamiento real y cae en el fallback constante (ver
20 # cf.py:get_prediction_stats). Filtrar a usuarios con más historial sube la
21 # densidad real de señal, a costa de menos usuarios en el set.
22 K_CORE_MIN_INTERACTIONS = 5
23 # Si el filtro deja menos filas que esto, se considera que el dataset no
24 # tiene la forma esperada (ej. en tests con datos sintéticos chicos) y se
25 # usa el dataset completo sin filtrar en vez de romper el entrenamiento.
26 _K_CORE_MIN_ROWS_SAFETY = 500
27
28
29 def _apply_k_core_filter(df: pd.DataFrame, min_interactions: int = K_CORE_MIN_INTERACTIONS) -> pd.
    DataFrame:
30     """Conserva solo las interacciones de usuarios con >= min_interactions
31     reviews. Devuelve el df sin filtrar si el resultado queda demasiado chico
32     (red de seguridad para datasets pequeños/sintéticos, ej. en tests)."""
33     if 'user_id' not in df.columns or df.empty:
34         return df
35     counts = df['user_id'].value_counts()
36     keep_users = counts[counts >= min_interactions].index
37     filtered = df[df['user_id'].isin(keep_users)]
38     if len(filtered) < _K_CORE_MIN_ROWS_SAFETY:
39         return df
40     return filtered
41
42
43 class SmarturEngine:
44     """
45     Motor principal de Inteligencia para Recomendaciones (Basado en Filtrado Colaborativo K-Neares
46     t Neighbors).
47     Se encarga de cargar la información histórica de interacciones ('reviews') y datos de negocios
48     ('business'),

```

```

47 y genera una matriz matemática optimizada de Utilidad Esparcida (User-Item Sparse Matrix).
48 """
49
50 def __init__(self, reviews_path=None, business_path=None, data_source='mexico'):
51     """
52     Inicializa el motor cargando los datos de disco y particionando para entrenamiento/test.
53
54     Args:
55         reviews_path (str, opcional): Ruta al CSV que contiene las reseñas (interacciones).
56         business_path (str, opcional): Ruta al CSV que contiene los detalles de los negocios.
57         data_source (str): 'mexico' (default) para datos mexicanos, 'yelp' para Yelp dataset l
58     egacy.
59     """
60     if reviews_path is None:
61         file_name = 'data_reviews_mexico.csv' if data_source == 'mexico' else 'data_reviews_li
62     mpio.csv'
63     reviews_path = os.path.join(_DATA, file_name)
64     if business_path is None:
65         file_name = 'data_negocios_mexico.csv' if data_source == 'mexico' else 'data_negocios_
66     limpio.csv'
67     business_path = os.path.join(_DATA, file_name)
68
69     raw_df = pd.read_csv(reviews_path)
70     self.df = _apply_k_core_filter(raw_df)
71     if len(self.df) < len(raw_df):
72         n_users_before = raw_df['user_id'].nunique() if 'user_id' in raw_df.columns else 0
73         n_users_after = self.df['user_id'].nunique() if 'user_id' in self.df.columns else 0
74         logger.info(
75             f"[k-core] Filtrado a usuarios con >= {K_CORE_MIN_INTERACTIONS} interacciones: "
76             f"{len(raw_df)} -> {len(self.df)} filas, {n_users_before} -> {n_users_after} usuar
77     ios "
78         f"(sube la densidad real para que KNN tenga vecinos con solapamiento).")
79     )
80     self.train_data, self.test_data = train_test_split(
81         self.df, test_size=0.2, random_state=42
82     )
83     self.df_biz = pd.read_csv(business_path)
84
85     # Matriz Sparse (Comprimida) Original de los ratings exactos
86     self.user_item_matrix = None
87     # Matriz Sparse Centrada respecto a la media de cada usuario (esencial para el cálculo tip
88     o Pearson)
89     self.matrix_centered = None
90     # Índices y nombres de columnas mapeados (pandas Categories)
91     self.user_item_matrix_index = None
92     self.user_item_matrix_columns = None
93     # Media de calificación (rating) por individuo
94     self.user_means = None
95     # Objeto NearestNeighbors entrenado de scikit-learn
96     self.knn_model = None
97     # Diccionarios de mapeo rápido entre ID de texto a índice entero en la Matriz
98     self._user_idx_map = None
99     self._biz_idx_map = None
100
101 def get_user_idx(self, user_id):
102     """Devuelve el índice matricial interno (entero posicional) de un ID de usuario en base al
103     mapa cacheado."""
104     if self._user_idx_map is None:
105         self._user_idx_map = {u: i for i, u in enumerate(self.user_item_matrix_index)}
106     return self._user_idx_map.get(user_id)

```

```

101
102 def get_biz_idx(self, biz_id):
103     """Devuelve el índice matricial interno (entero posicional) de un ID de negocio en base al
        mapa cacheado."""
104     if self.biz_idx_map is None:
105         self.biz_idx_map = {b: i for i, b in enumerate(self.user_item_matrix_columns)}
106     return self.biz_idx_map.get(biz_id)
107
108 def prepare_pearson_matrix(self):
109     """
110     Construye la matriz matemática de Utilidad optimizada utilizando scipy.sparse.csr_matrix.
111     Este formato previene ineficiencias y el error 'MemoryError' de memoria RAM cuando existen
112     decenas de miles de combinaciones usuario-ítem (evitando guardar los ceros y celdas vacías
        explícitamente).
113
114     Además, prepara el centrado matemático de los datos (la calificación restada por la media
        personal del usuario).
115     Para los datos centrados explícitamente, la correlación de Pearson y la Distancia del Cose
        no ('cosine')
116     miden la misma relación geométrica de similitud; por lo tanto, usamos cosine que sí soport
        a estas matrices sparse.
117     """
118     cats_u = self.train_data['user_id'].astype('category')
119     cats_i = self.train_data['business_id'].astype('category')
120
121     row = cats_u.cat.codes
122     col = cats_i.cat.codes
123     data = self.train_data['stars'].values.astype(np.float32)
124
125     n_u = len(cats_u.cat.categories)
126     n_i = len(cats_i.cat.categories)
127
128     self.user_item_matrix_index = cats_u.cat.categories
129     self.user_item_matrix_columns = cats_i.cat.categories
130
131     # 1. Construir matriz estructural csr_matrix de valor original no modificado (ratings exac
132     tos 1 a 5)
133     self.user_item_matrix = csr_matrix((data, (row, col)), shape=(n_u, n_i))
134
135     # Calcular medias por usuario
136     user_sums = np.bincount(row, weights=data)
137     user_counts = np.bincount(row)
138     user_sums = np.pad(user_sums, (0, n_u - len(user_sums)), 'constant')
139     user_counts = np.pad(user_counts, (0, n_u - len(user_counts)), 'constant')
140
141     user_means = np.divide(user_sums, user_counts, out=np.zeros_like(user_sums), where=user_co
142     unts!=0)
143     self.user_means = pd.Series(user_means, index=self.user_item_matrix_index)
144
145     # Centrado
146     centered_data = data - user_means[row]
147     self.matrix_centered = csr_matrix((centered_data, (row, col)), shape=(n_u, n_i))
148
149     # Cosine distance equivale a Pearson correlation para datos centrados.
150     self.knn_model = NearestNeighbors(
151         metric='cosine', algorithm='brute', n_jobs=-1
152     )
153     self.knn_model.fit(self.matrix_centered)
154
155     # — TruncatedSVD: Factorización Latente —————

```

```

154 # Complementa el KNN de Pearson con representaciones latentes densas.
155 # Permite predicciones más robustas cuando el KNN no tiene vecinos con
156 # el ítem específico (sim_sum = 0), reduciendo los fallbacks a la media.
157 try:
158     n_components = min(50, min(n_u, n_i) - 1)
159     if n_components >= 2:
160         svd = TruncatedSVD(n_components=n_components, random_state=42)
161         # Factorizar sobre la matriz centrada para mantener coherencia con KNN
162         self.user_latent = svd.fit_transform(self.matrix_centered) # (n_u, k)
163         self.item_latent = svd.components.T # (n_i, k)
164         # Mapeos de ID -> índice en la matriz de factores latentes
165         self.user_index = {uid: i for i, uid in enumerate(self.user_item_matrix_index)}
166         self.item_index = {iid: i for i, iid in enumerate(self.user_item_matrix_columns)}
167         explained = svd.explained_variance_ratio_.sum()
168         print(
169             f"SVD listo: {n_components} componentes, "
170             f"varianza explicada={explained:.1%}"
171         )
172     else:
173         print("SVD omitido: matrix demasiado pequeña para factorizar.")
174 except Exception as svd_err:
175     print(f"SVD falló (CF KNN sigue activo): {svd_err}")
176
177 print(f"Engine listo: {n_u} usuarios y {n_i} negocios.")
178
179 def get_candidate_pool(self, user_id, top_n=50):
180     """
181     Calcula un abanico (pool) inicial rápido de negocios como recomendaciones candidatas
182     para un usuario, apoyándose de los usuarios más similares encontrados mediante K-NN (K-vec
183     inos más cercanos).
184
185     Args:
186         user_id (str): La ID única tipo UUID del usuario.
187         top_n (int): La cantidad deseada de recomendaciones candidatas base.
188
189     Returns:
190         list: Una lista que contiene los IDs en texto de negocios que podrían gustarle al usua
191         rio.
192         Si el usuario no tiene historial, se le retornan los tópicos más populares de forma gl
193         obal.
194     """
195     user Rated = set(
196         self.train_data[self.train_data['user_id'] == user_id]['business_id'].tolist()
197     )
198
199     if self.user_item_matrix_index is None or user_id not in self.user_item_matrix_index:
200         # Cold-start: Usuario nuevo sin ningún historial de interacciones en el dataset
201         return (
202             self.train_data.groupby('business_id')['stars']
203             .count()
204             .sort_values(ascending=False)
205             .loc[lambdas: ~s.index.isin(user Rated)]
206             .head(top_n)
207             .index.tolist()
208         )
209
210     k = min(top_n, self.user_item_matrix.shape[0] - 1)
211     user_idx = self.get_user_idx(user_id)
212     query = self.matrix_centered[user_idx]

```

```

211         distances, indices = self.knn_model.kneighbors(
212             query, n_neighbors=k
213         )
214
215         similar_users = self.user_item_matrix_index[indices[0]]
216         candidates = self.train_data[self.train_data['user_id'].isin(similar_users)]
217         return (
218             candidates.groupby('business_id')['stars']
219             .mean()
220             .sort_values(ascending=False)
221             .loc[lambda s: ~s.index.isin(user Rated)]
222             .head(top_n)
223             .index.tolist()
224         )

```

===== MODELO/src/rf_model.py =====

```

1 import os
2 import pandas as pd
3 import numpy as np
4 import joblib
5 from sklearn.ensemble import RandomForestRegressor
6 from context_encoder import ContextEncoder, MAPEO_CATEGORIAS
7
8 _DIR = os.path.dirname(os.path.abspath(__file__))
9 _DATA = os.path.join(_DIR, '..', 'data')
10 _MODELS = os.path.join(_DIR, '..', 'models')
11 TOP_N_CATEGORIES = 15
12
13 # Categorías turísticas garantizadas en el feature set aunque no estén en el top-15 por frecuencia
14
15 # Necesarias para que el RF aprenda señales de turismo de naturaleza/aventura/cultural
16 # cuando el dataset base es mayoritariamente gastronómico (ej. Yelp).
17 TOURISM_ANCHOR_CATEGORIES = [
18     # Yelp tourism categories (always needed for fallback dev mode)
19     'Parks', 'Hiking', 'Museums', 'Hotels', 'Hotels & Travel',
20     'Arts & Entertainment', 'Active Life', 'Tours',
21     'Landmarks & Historical Buildings', 'Bed & Breakfast',
22     'Campgrounds', 'Zoos', 'Botanical Gardens',
23     # Local POI categories – Altas Montañas, Veracruz (categories_mapped + categories_raw)
24     'nature', 'culture', 'gastronomy',
25     'park', 'viewpoint', 'waterfall', 'volcano', 'mountain',
26     'hacienda', 'cathedral', 'sanctuary', 'market', 'museum',
27     'history', 'monument', 'zocalo', 'botanical garden',
28 ]
29
30 # Geographic center of Altas Montañas, Veracruz – used as fallback when user location is unknown.
31 ALTAS_MONTANAS_CENTER = (18.95, -97.05)
32
33 def _compute_dist_km(df, user_lat=None, user_lon=None):
34     """
35     Haversine distance (km) from each row's lat/lon to the user's position.
36     Falls back to ALTAS_MONTANAS_CENTER when user location is unknown.
37     Result is clipped to [0, 500] km so distant Yelp training items saturate
38     at 500 (no variance for them) while local POIs spread from 0–50 km.
39     """
40     lat0 = float(user_lat) if user_lat is not None else ALTAS_MONTANAS_CENTER[0]
41     lon0 = float(user_lon) if user_lon is not None else ALTAS_MONTANAS_CENTER[1]
42     if 'latitude' not in df.columns or 'longitude' not in df.columns:
43         return pd.Series(0.0, index=df.index)

```

```

44 R = 6371.0
45 lat1 = np.radians(lat0)
46 lon1 = np.radians(lon0)
47 lat2 = np.radians(df['latitude'].fillna(lat0).astype(float))
48 lon2 = np.radians(df['longitude'].fillna(lon0).astype(float))
49 dlat = lat2 - lat1
50 dlon = lon2 - lon1
51 a = np.sin(dlat / 2) ** 2 + np.cos(lat1) * np.cos(lat2) * np.sin(dlon / 2) ** 2
52 return (2 * R * np.arcsin(np.sqrt(a.clip(0, 1)))).clip(0, 500)
53
54
55 class SmarturContextModel:
56     """
57     Modelo contextual SMARTUR v4 basado en Random Forest Regressor.
58     True Machine Learning Contextual:
59     El modelo aprende las interacciones dinámicamente mediante la inyección de
60     datos de contexto de usuario sintéticos durante el entrenamiento.
61     """
62     def __init__(self, business_path=None, data_source='mexico'):
63         if business_path is None:
64             file_name = 'data_negocios_mexico.csv' if data_source == 'mexico' else 'data_negocios_
limpio.csv'
65             business_path = os.path.join(_DATA, file_name)
66             self.df_biz = pd.read_csv(business_path)
67             self.model = RandomForestRegressor(
68                 n_estimators=200, max_depth=12, min_samples_leaf=5, n_jobs=-1
69             )
70             self.top_categories = []
71             self.numeric_features = [
72                 'review_count', 'is_open',
73                 'price_level', 'is_accessible', 'outdoor',
74                 'is_good_for_kids', 'is_romantic',
75                 'dist_km', # haversine distance from user / Altas Montañas center
76             ]
77             self.cat_features = []
78             self.features = [] # Item + Context (todas agregadas al train)
79             self.is_fitted = False
80             self.encoder = ContextEncoder()
81             self._local_defaults = {
82                 'review_count': 10,
83                 'is_open': 1,
84                 'is_good_for_kids': 0,
85                 'is_romantic': 0,
86             }
87
88     def _extract_top_categories(self, df):
89         """Encuentra las N categorías más frecuentes + anclas turísticas garantizadas."""
90         all_cats = (
91             df['categories']
92             .dropna()
93             .str.split(',')
94             .explode()
95             .str.strip()
96         )
97         top_by_freq = all_cats.value_counts().head(TOP_N_CATEGORIES).index.tolist()
98         present = set(top_by_freq)
99         anchors = [c for c in TOURISM_ANCHOR_CATEGORIES if c not in present]
100         self.top_categories = top_by_freq + anchors
101         self.cat_features = [f'cat_{c}' for c in self.top_categories]
102

```

```

103 def _add_category_features(self, df):
104     """Agrega columnas binarias para las top categorías."""
105     result = df.copy()
106     if 'review_count' in result.columns:
107         result['review_count'] = np.log1p(result['review_count'].fillna(0))
108     for cat in self.top_categories:
109         result[f'cat_{cat}'] = (
110             result['categories']
111             .fillna('')
112             .str.contains(cat, case=False, regex=False)
113             .astype(int)
114         )
115     for col in self.numeric_features:
116         if col not in result.columns:
117             result[col] = 0
118     return result
119
120 def prepare_local_items(self, poi_df):
121     """Normaliza POIs locales al esquema del modelo."""
122     if poi_df is None or poi_df.empty:
123         return pd.DataFrame(columns=self.df_biz.columns)
124
125     df = poi_df.copy()
126     # POIs use numeric id → prefix with "poi_" to match mobile's Place.id format ("poi_N").
127     # Services already have "svc_N" string ids from fetch_tourist_services.
128     # This also aligns with fetch_real_interactions which builds item_id as "poi_N"/"svc_N",
129     # so CF/SVD interaction lookup works correctly for warm users.
130     df['business_id'] = df.apply(
131         lambda r: str(r['id']) if str(r['id']).startswith('svc_') else f"poi_{r['id']}",
132         axis=1,
133     )
134     df['name'] = df['name'].fillna('Local POI')
135
136     def _join_categories(row):
137         parts = []
138         raw = row.get('categories_raw')
139         if isinstance(raw, str) and raw.strip():
140             parts.append(raw.strip())
141         mapped = row.get('categories_mapped', [])
142         if isinstance(mapped, list) and mapped:
143             parts.extend(mapped)
144         return ','.join(parts) if parts else ''
145
146     df['categories'] = df.apply(_join_categories, axis=1)
147
148     # Preserve description from DB (may be None if not fetched)
149     if 'description' not in df.columns:
150         df['description'] = ''
151     df['description'] = df['description'].fillna('')
152
153     if 'price_level' not in df.columns:
154         df['price_level'] = 2
155     if 'is_accessible' not in df.columns:
156         df['is_accessible'] = 0
157     if 'outdoor' not in df.columns:
158         df['outdoor'] = 0
159     if 'latitude' not in df.columns:
160         df['latitude'] = 0
161     if 'longitude' not in df.columns:
162         df['longitude'] = 0

```

```

163
164     df['price_level'] = df['price_level'].fillna(2).astype(int)
165     df['is_accessible'] = df['is_accessible'].fillna(0).astype(int)
166     df['outdoor'] = df['outdoor'].fillna(0).astype(int)
167     # Altas Montañas region center - avoids placing missing coords in the Atlantic Ocean
168     df['latitude'] = df['latitude'].fillna(18.85).astype(float)
169     df['longitude'] = df['longitude'].fillna(-96.95).astype(float)
170
171     for col, default in self._local_defaults.items():
172         if col not in df.columns:
173             df[col] = default
174             df[col] = df[col].fillna(default)
175
176     return df
177
178 def _simulate_user_contexts(self, df):
179     """
180     Infiere el perfil de usuario desde su historial de ratings en lugar de asignarlo
181     aleatoriamente. Tipos de turismo y tipo de grupo se derivan de las categorías y
182     atributos de los negocios que el usuario calificó con ≥4 estrellas, haciendo que
183     los match features sean correlacionados con los ratings reales.
184     """
185     print("Inferiendo contextos de usuario desde historial de ratings...")
186     result = df.copy()
187     np.random.seed(42)
188     n = len(df)
189     liked_mask = df['stars_user'] >= 4
190     cats_lower = df['categories'].fillna('').str.lower()
191
192     # 1. Inferir user_budget desde precio promedio de items valorados ≥4
193     if 'price_level' in df.columns:
194         user_mean_price = (
195             df[liked_mask]
196             .groupby('user_id')['price_level']
197             .mean().round().fillna(2).astype(int)
198         )
199     else:
200         user_mean_price = pd.Series(dtype=int)
201     result['user_budget'] = result['user_id'].map(user_mean_price).fillna(2).astype(int)
202
203     # 2. Inferir tipos de turismo desde categorías de items valorados ≥4
204     for t in self.encoder.tourism_types:
205         cats_to_match = [c.lower() for c in MAPEO_CATEGORIAS.get(t, [])]
206         if not cats_to_match:
207             result[f'user_tur_{t}'] = 0
208             continue
209         biz_has_tur = cats_lower.str.contains('|'.join(cats_to_match), regex=True, na=False)
210         user_tur_map = (liked_mask & biz_has_tur).groupby(df['user_id']).any().astype(int)
211         result[f'user_tur_{t}'] = result['user_id'].map(user_tur_map).fillna(0).astype(int)
212
213     # 3. Inferir group_type desde items románticos / kid-friendly valorados ≥4
214     romantic_col = df['is_romantic'] if 'is_romantic' in df.columns else pd.Series(0, index=df
.index)
215     kids_col = df['is_good_for_kids'] if 'is_good_for_kids' in df.columns else pd.Series(0, in
dex=df.index)
216     likes_romantic_mask = liked_mask & (romantic_col == 1)
217     likes_kids_mask = liked_mask & (kids_col == 1)
218     romantic_map = likes_romantic_mask.groupby(df['user_id']).any()
219     kids_map = likes_kids_mask.groupby(df['user_id']).any()
220

```

```

221     user_is_romantic = result['user_id'].map(romantic_map).fillna(False)
222     user_is_kids = result['user_id'].map(kids_map).fillna(False)
223
224     rand_fallback = np.random.choice(['solo', 'amigos'], size=n, p=[0.4, 0.6])
225     groups = np.where(
226         user_is_kids, 'familia',
227         np.where(user_is_romantic & ~user_is_kids, 'pareja', rand_fallback)
228     )
229     for g in self.encoder.group_types:
230         result[f'user_group_{g}'] = (groups == g).astype(int)
231
232     # 4. Age range – neutral value (not reliably inferable from Yelp ratings)
233     result['user_age_range'] = 3
234
235     # 5. Infer boolean preferences from categories of items rated ≥4.
236     # Replacing random noise with inferred signal eliminates the main source of
237     # spurious correlations; the RF now learns real user-category co-occurrence.
238     outdoor_rx = r'park|hiking|outdoor|nature|adventure|volcano|mountain|viewpoint|waterfall'
239     tours_rx = r'\btours?\b|excursion|senderismo'
240     hotel_rx = r'hotel|accommodation|bed & breakfast|hostel|campground'
241     food_rx = r'restaurant|food|caf[eé]|dining|gastronomy|market|local food'
242
243     def infer_pref(pattern, default=0):
244         has_cat = cats_lower.str.contains(pattern, na=False, regex=True)
245         user_map = (liked_mask & has_cat).groupby(df['user_id']).any().astype(int)
246         return result['user_id'].map(user_map).fillna(default).astype(int)
247
248     result['user_pref_outdoor'] = _infer_pref(outdoor_rx)
249     result['user_wants_tours'] = _infer_pref(tours_rx)
250     result['user_needs_hotel'] = _infer_pref(hotel_rx)
251     result['user_pref_food'] = _infer_pref(food_rx, default=1) # food pref is co
252 mmon
253     result['user_requires_accessibility'] = 0 # cannot infer from ratings; keep neutral
254     result['user_has_visited_region'] = 0 # cannot infer from ratings; keep neutral
255
256     # 6. Match features (ahora correlacionados con ratings reales)
257     result['budget_delta'] = (result['user_budget'] - result.get('price_level', 2)).abs()
258
259     overlap = np.zeros(n)
260     for t in self.encoder.tourism_types:
261         mask_user = result[f'user_tur_{t}'] == 1
262         cats_to_match = [c.lower() for c in MAPEO_CATEGORIAS.get(t, [])]
263         if cats_to_match:
264             mask_biz = cats_lower.str.contains('|'.join(cats_to_match), regex=True, na=False)
265             overlap += (mask_user & mask_biz).astype(int)
266     result['interest_overlap'] = overlap
267
268     is_good_kids = result.get('is_good_for_kids', np.zeros(n))
269     result['kids_match'] = ((result['user_group_familia'] == 1) & (is_good_kids == 1)).astype(
270 int)
271
272     is_rom = result.get('is_romantic', np.zeros(n))
273     result['romantic_match'] = ((result['user_group_pareja'] == 1) & (is_rom == 1)).astype(int)
274
275     result['tours_match'] = (
276         (result['user_wants_tours'] == 1) & cats_lower.str.contains('tours', na=False)
277     ).astype(int)
278
279     return result

```

```

278
279 def train(self, reviews_df, df_biz_extra=None, dynamic_override=True):
280     if df_biz_extra is not None:
281         biz_ids_extra = df_biz_extra['business_id'].unique()
282         self.df_biz = pd.concat([
283             self.df_biz[~self.df_biz['business_id'].isin(biz_ids_extra)],
284             df_biz_extra
285         ], ignore_index=True)
286         print(f"[RF] Añadidos {len(df_biz_extra)} negocios extra (Rest-Mex)")
287     train_df = reviews_df.merge(self.df_biz, on='business_id', suffixes=('_user', '_biz'))
288
289     # 1. Pipeline de Item
290     self._extract_top_categories(train_df)
291
292     # Compute geographic distance from Altas Montañas center (proxy for training data).
293     # Yelp items (Phoenix/Las Vegas) will saturate at 500 km; local POIs spread 0-50 km.
294     train_df['dist_km'] = _compute_dist_km(train_df)
295
296     train_df = self._add_category_features(train_df)
297
298     # 2. Pipeline de User (Generación Sintética)
299     train_df = self._simulate_user_contexts(train_df)
300
301     # 3. Features combinadas: Item + User + Match
302     self.features = self.numeric_features + self.cat_features + self.encoder.all_context_featu
303 re_names
304
305     X = train_df[self.features].fillna(0)
306     y = train_df['stars_user']
307
308     # Profundidad dinámica: evita sobreajuste con datasets pequeños (26 POIs locales)
309     # Se omite si dynamic_override=False (usado por grid search)
310     n_items = len(train_df['business_id'].unique())
311     if dynamic_override:
312         max_depth = 12 if n_items > 5000 else (7 if n_items > 500 else 4)
313         n_est = min(200, max(50, n_items * 5))
314         self.model.set_params(max_depth=max_depth, n_estimators=n_est)
315     else:
316         max_depth = self.model.max_depth
317         n_est = self.model.n_estimators
318
319     print(f"[RF (True ML Contextual): entrenando sobre {X.shape[0]} interacciones con {X.shape[
320 1]} variables cruzadas.")
321     print(f"    max_depth={max_depth}, n_estimators={n_est}, n_items={n_items}")
322     print(f"Features en red: {self.features}")
323
324     self.model.fit(X, y)
325
326     # — Feature importances —————
327     fi = pd.DataFrame({
328         'feature': self.features,
329         'importance': self.model.feature_importances_,
330     }).sort_values('importance', ascending=False)
331     print(f"\nFeature Importances (top 20):\n{fi.head(20).to_string(index=False)}\n")
332     fi.to_json(os.path.join(_MODELS, 'rf_feature_importances.json'), orient='records', indent=
333 2)
334
335     # — Hyperparameters —————
336     import json as _json
337     hp = {k: str(v) for k, v in self.model.get_params().items()}

```

```

335         with open(os.path.join(_MODELS, 'rf_hyperparams.json'), 'w') as _f:
336             _json.dump(hp, _f, indent=2)
337         print(f"Hyperparams: {hp}")
338
339         os.makedirs(_MODELS, exist_ok=True)
340         joblib.dump({
341             'model': self.model,
342             'top_categories': self.top_categories,
343             'features': self.features,
344         }, os.path.join(_MODELS, 'rf_context_yelp.joblib'))
345
346         self.is_fitted = True
347         print("Random Forest Contextual (ML) entrenado y guardado.")
348
349     def load(self, model_path=None):
350         """Carga modelo Random Forest con features combinadas desde disco."""
351         if model_path is None:
352             model_path = os.path.join(_MODELS, 'rf_context_yelp.joblib')
353         if os.path.exists(model_path):
354             data = joblib.load(model_path)
355             self.model = data['model']
356             self.top_categories = data['top_categories']
357             self.features = data['features']
358             self.is_fitted = True
359             self.cat_features = [f'cat_{c}' for c in self.top_categories]
360             print("Random Forest Contextual cargado desde disco.")
361             return True
362         return False
363
364     def predict_context(self, business_ids, df_biz_override=None):
365         """Fallback si no mandan ningún contexto."""
366         return self.predict_with_context(business_ids, user_context=None, df_biz_override=df_biz_o
367                                         verride)
368
369     def predict_with_context(self, business_ids, user_context=None, df_biz_override=None):
370         """
371         Inferencia Pura ML de 2da Etapa:
372         Procesa el dataset crudo en vectores ampliados [Item] + [User] + [Match]
373         y los inyecta al modelo Random Forest pre-entrenado.
374         El modelo toma la decisión final evaluando todas las colisiones sin sumar/restar manualmen
375         te.
376         """
377         ref_df = df_biz_override if df_biz_override is not None else self.df_biz
378         biz_indexed = ref_df.set_index('business_id')
379         biz_features = biz_indexed.reindex(business_ids).reset_index()
380
381         # Compute geographic distance using real user location if available
382         user_lat = user_context.get('lat') if user_context else None
383         user_lon = user_context.get('lon') if user_context else None
384         biz_features['dist_km'] = _compute_dist_km(biz_features, user_lat, user_lon)
385
386         biz_features = self._add_category_features(biz_features)
387
388         # Codificamos al usuario real (React Form)
389         # Si no hay contexto, el encoder emite defaults limpios ('cold start usuario medio')
390         user_feats = self.encoder.encode_user(user_context)
391
392         # Generamos la colisión de Match cruzando contra la lista de businesses obtenidos
393         context_rows = []
394         for _, biz_row in biz_features.iterrows():

```

```

393         match_feats = self.encoder.compute_match_features(user_feats, biz_row)
394         context_rows.append(**user_feats, **match_feats})
395
396         context_df = pd.DataFrame(context_rows, index=biz_features.index)
397
398         # Unimos las tablas ItemFeature | UserFeature | MatchFeature
399         combined_df = pd.concat([biz_features, context_df], axis=1)
400
401         # Aseguramos un DataFrame rígido con las mismas features de `self.features` original
402         for col in self.features:
403             if col not in combined_df.columns:
404                 combined_df[col] = 0
405
406         X = combined_df[self.features].fillna(0)
407
408         # Predicción pura dictaminada 100% por el Random Forest
409         preds = self.model.predict(X)
410
411         return np.clip(preds, 1, 5)

```

===== MODELO/src/context_encoder.py =====

```

1 """
2 SMARTUR Context Encoder v4 (True ML)
3 Transforma el JSON del formulario React en vectores numéricos para el modelo RF.
4 Responsable de:
5 - Mapeo ordinal de presupuesto y rango de edad
6 - Multi-hot encoding de tipos de turismo, y group_type
7 - Features de match (budget_delta, interest_overlap, kids_match, romantic_match, tours_match)
8 """
9
10 # Mapeo de categorías por tipo de turismo.
11 # Incluye términos Yelp (fallback desarrollo) y términos locales de Altas Montañas, Veracruz
12 # (producción: categories_raw + categories_mapped de PostgreSQL).
13 MAPEO_CATEGORIAS = {
14     'naturaleza': [
15         # Yelp
16         'Parks', 'Botanical Gardens', 'Hiking', 'Landmarks & Historical Buildings', 'Lakes',
17         # Local POI - categories_mapped
18         'nature',
19         # Local POI - categories_raw
20         'park', 'viewpoint', 'waterfall', 'volcano', 'mountain', 'botanical garden', 'hiking',
21     ],
22     'aventura': [
23         # Yelp
24         'Active Life', 'Hiking', 'Rafting', 'Mountain Biking', 'Tours',
25         # Local POI
26         'nature', 'park', 'hiking', 'volcano', 'mountain', 'viewpoint',
27     ],
28     'gastronomico': [
29         # Yelp
30         'Restaurants', 'Food', 'Cafes', 'Traditional Mexican', 'Bakeries',
31         # Local POI - categories_mapped
32         'gastronomy',
33         # Local POI - categories_raw
34         'market', 'local food', 'restaurant', 'food', 'coffee',
35     ],
36     'cultural': [
37         # Yelp
38         'Museums', 'Art Galleries', 'Arts & Entertainment', 'Historical Tours', 'Festivals',
39         # Local POI - categories_mapped

```

```

40     'culture',
41     # Local POI – categories_raw
42     'museum', 'history', 'hacienda', 'monument', 'cathedral', 'religion',
43     'zocalo', 'town square', 'sanctuary', 'architecture', 'neighborhood', 'science',
44 ],
45 'rural': [
46     # Yelp
47     'Hotels', 'Bed & Breakfast', 'Campgrounds', 'Farm Stays', 'Guest Houses',
48     # Local POI
49     'culture', 'hacienda',
50 ],
51 'nocturno': [
52     # Vida nocturna / bares. Hoy el catálogo de las Altas Montañas no tiene
53     # POIs de este tipo (0 coincidencias), pero el tipo queda listo para
54     # activarse solo en cuanto se registre un bar/antro – alineado con el
55     # crecimiento del catálogo lugar por lugar.
56     # Yelp
57     'Bars', 'Nightlife', 'Pubs', 'Lounges', 'Dance Clubs', 'Cocktail Bars',
58     # Local POI – categories_raw
59     'bar', 'pub', 'nightlife', 'cantina', 'antro', 'lounge',
60 ],
61 }
62
63 TOURISM_TYPES = list(MAPEO_CATEGORIAS.keys())
64 GROUP_TYPES = ['solo', 'pareja', 'familia', 'amigos']
65
66 BUDGET_MAP = {
67     'bajo': 1,
68     'medio': 2,
69     'alto': 3,
70     'premium': 4,
71 }
72
73 AGE_MAP = {
74     '18-24': 1,
75     '25-34': 2,
76     '35-44': 3,
77     '45-54': 4,
78     '55+': 5,
79 }
80
81 class ContextEncoder:
82     """Codifica el contexto del turista en features numéricas."""
83
84     def __init__(self):
85         self.tourism_types = TOURISM_TYPES
86         self.group_types = GROUP_TYPES
87         self.budget_map = BUDGET_MAP
88         self.age_map = AGE_MAP
89
90     def encode_user(self, context):
91         """
92         Convierte el dict de contexto del formulario React en features numéricas del usuario.
93         """
94         if not context:
95             return self._default_user_features()
96
97         features = {}
98
99         # Ordinal: presupuesto

```

```

100     budget_raw = str(context.get('presupuesto_bucket', '')).lower().strip()
101     features['user_budget'] = self.budget_map.get(budget_raw, 2)
102
103     # Ordinal: rango de edad
104     age_raw = str(context.get('edad_range', '')).strip()
105     features['user_age_range'] = self.age_map.get(age_raw, 2)
106
107     # Multi-hot: tipos de turismo
108     tipos = context.get('tiposTurismo', [])
109     if isinstance(tipos, str):
110         tipos = [tipos]
111     # Cold-start fallback: when no tourism preference is set, use the most
112     # common types for the Altas Montañas region so the user gets a
113     # meaningful ranking instead of a flat all-zeros feature vector.
114     if not tipos:
115         tipos = ['naturaleza', 'cultural']
116     for t in self.tourism_types:
117         features[f'user_tur_{t}'] = 1 if t in tipos else 0
118
119     # One-hot: group_type
120     gtype = str(context.get('group_type', 'solo')).lower().strip()
121     for g in self.group_types:
122         features[f'user_group_{g}'] = 1 if g == gtype else 0
123
124     # Boolean preferences
125     features['user_requires_accessibility'] = 1 if context.get('requiere_accesibilidad', False)
126 ) else 0
127     features['user_pref_outdoor'] = 1 if context.get('pref_outdoor', False) else 0
128     features['user_wants_tours'] = 1 if context.get('wants_tours', False) else 0
129     features['user_needs_hotel'] = 1 if context.get('needs_hotel', False) else 0
130     # has visited region: repeat visitor -> prefer lesser-known spots (exploratory signal)
131     features['user_has_visited_region'] = 1 if context.get('has_visited_region', False) else 0
132
133     # pref_food by default is true (assume they want to see restaurants unless explicitly fals
134 e)
135     features['user_pref_food'] = 1 if context.get('pref_food', True) else 0
136
137     return features
138
139 def _default_user_features(self):
140     """Features por defecto cuando no hay contexto (cold start de contexto)."""
141     features = {
142         'user_budget': 2,
143         'user_age_range': 2,
144         'user_requires_accessibility': 0,
145         'user_pref_outdoor': 0,
146         'user_wants_tours': 0,
147         'user_needs_hotel': 0,
148         'user_pref_food': 1,
149         'user_has_visited_region': 0,
150     }
151     for t in self.tourism_types:
152         features[f'user_tur_{t}'] = 0
153     for g in self.group_types:
154         features[f'user_group_{g}'] = 0
155     return features
156
157 def compute_match_features(self, user_features, biz_row):
158     """
159     Calcula features de interacción user x item de alto nivel para el RandomForest.

```

```

158     """
159     # Budget delta
160     user_budget = user_features.get('user_budget', 2)
161     price_level = int(biz_row.get('price_level', 2)) if 'price_level' in biz_row else 2
162     budget_delta = abs(user_budget - price_level)
163
164     # Interest overlap
165     categories_str = str(biz_row.get('categories', '')).lower()
166     overlap = 0
167     for t in self.tourism_types:
168         if user_features.get(f'user_tur_{t}', 0) == 1:
169             yelp_cats = MAPEO_CATEGORIAS.get(t, [])
170             for cat in yelp_cats:
171                 if cat.lower() in categories_str:
172                     overlap += 1
173                     break
174
175     # Specific contextual matches
176     # Kids match: family group + good for kids
177     is_good_kids = int(biz_row.get('is_good_for_kids', 0)) if 'is_good_for_kids' in biz_row else 0
178     kids_match = 1 if user_features.get('user_group_familia') == 1 and is_good_kids == 1 else 0
179
180     # Romantic match: couple group + romantic/intimate
181     is_rom = int(biz_row.get('is_romantic', 0)) if 'is_romantic' in biz_row else 0
182     romantic_match = 1 if user_features.get('user_group_pareja') == 1 and is_rom == 1 else 0
183
184     # Tours match: user wants tours + category includes tours
185     tours_match = 1 if user_features.get('user_wants_tours') == 1 and 'tours' in categories_str else 0
186
187     return {
188         'budget_delta': budget_delta,
189         'interest_overlap': overlap,
190         'kids_match': kids_match,
191         'romantic_match': romantic_match,
192         'tours_match': tours_match,
193     }
194
195 def encode_pair(self, context, biz_row):
196     """
197     Construye el vector completo [User + Item + Interaction] para una pareja usuario-negocio.
198     """
199     user_feats = self.encode_user(context)
200     match_feats = self.compute_match_features(user_feats, biz_row)
201
202     return {**user_feats, **match_feats}
203
204 @property
205 def user_feature_names(self):
206     """Nombres de las features de usuario (para columnas del DataFrame)."""
207     names = ['user_budget', 'user_age_range']
208     names += [f'user_tur_{t}' for t in self.tourism_types]
209     names += [f'user_group_{g}' for g in self.group_types]
210     names += [
211         'user_requires_accessibility', 'user_pref_outdoor',
212         'user_wants_tours', 'user_needs_hotel', 'user_pref_food',
213         'user_has_visited_region',
214     ]

```

```

215         return names
216
217     @property
218     def match_feature_names(self):
219         """Nombres de las features de interacción."""
220         return ['budget_delta', 'interest_overlap', 'kids_match', 'romantic_match', 'tours_match']
221
222     @property
223     def all_context_feature_names(self):
224         """Todas las features de contexto (user + match)."""
225         return self.user_feature_names + self.match_feature_names

```

===== MODEL0/src/content_model.py =====

```

1  """
2  SMARTUR Content Model – TF-IDF Cold-Start Fallback.
3
4  Usado como fallback cuando LightFM no está disponible.
5  Calcula similitud coseno entre el perfil de preferencias del usuario (vector TF-IDF
6  de sus tipos de turismo) y los POIs (vector TF-IDF de sus categorías).
7
8  No requiere historial de interacciones ni dependencias adicionales (sklearn).
9  """
10
11  import logging
12  import numpy as np
13  import pandas as pd
14
15  _logger = logging.getLogger('smartur-api')
16
17  # Mapeo tipo de turismo -> términos de búsqueda para el query del usuario
18  _TURISMO_QUERY = {
19      'naturaleza': 'park hiking nature waterfall volcano mountain viewpoint botanical outdoor',
20      'aventura': 'adventure hiking active volcano mountain rafting biking nature park',
21      'gastronomico': 'restaurant food gastronomy cafe market traditional local',
22      'cultural': 'museum culture history hacienda monument cathedral sanctuary architecture art',
23      'rural': 'rural hacienda bed breakfast campground farm countryside',
24  }
25
26  _BUDGET_PRICE_HINT = {'bajo': '1', 'medio': '2', 'alto': '3', 'premium': '4'}
27
28
29  class SmarturContentModel:
30      """
31      TF-IDF content-based recommender.
32
33      - fit(biz_df): builds TF-IDF matrix from item categories + attributes
34      - score(item_ids, user_context): cosine similarity between user query and items
35      - Returns scores in [1, 5]
36      """
37
38      def __init__(self):
39          self._tfidf = None # fitted TfidfVectorizer
40          self._item_matrix = None # sparse (n_items, n_features)
41          self._item_ids = [] # ordered list of business_id
42          self.is_fitted = False
43
44      def fit(self, biz_df: pd.DataFrame) -> None:
45          """Build TF-IDF matrix from the business corpus."""
46          try:

```

```

47         from sklearn.feature_extraction.text import TfidfVectorizer
48
49         if biz_df is None or biz_df.empty:
50             return
51
52         df = biz_df.copy()
53         id_col = 'business_id' if 'business_id' in df.columns else 'id'
54         df['_bid'] = df[id_col].astype(str)
55
56         # Build text document for each item: categories + binary attributes as pseudo-words
57         docs = []
58         for _, row in df.iterrows():
59             cats = str(row.get('categories', '') or '')
60             parts = [cats]
61
62             # Numeric attributes -> pseudo-words
63             price = int(row.get('price_level', 2) or 2)
64             parts.append(f'price_{price}')
65             if int(row.get('is_accessible', 0) or 0):
66                 parts.append('accessible accessibility')
67             if int(row.get('outdoor', 0) or 0):
68                 parts.append('outdoor nature')
69             if int(row.get('is good for kids', 0) or 0):
70                 parts.append('kids family children')
71             if int(row.get('is romantic', 0) or 0):
72                 parts.append('romantic couple pareja')
73
74             docs.append(' '.join(parts).lower())
75
76         self._tfidf = TfidfVectorizer(
77             analyzer='word',
78             token_pattern=r'[a-záéíóúüñ&]+' ,
79             min_df=1,
80             max_features=3000,
81             sublinear_tf=True,
82         )
83         self._item_matrix = self._tfidf.fit_transform(docs)
84         self._item_ids = df['_bid'].tolist()
85         self.is_fitted = True
86         _logger.info(f"[ContentModel] TF-IDF listo: {len(self._item_ids)} ítems, "
87                     f"{self._item_matrix.shape[1]} features")
88     except Exception as exc:
89         _logger.warning(f"[ContentModel] Error en fit(): {exc}")
90
91 def score(self, item_ids: list, user_context: dict = None) -> list:
92     """
93     Compute content similarity scores for the given item_ids.
94
95     Args:
96         item_ids: list of business id strings to score
97         user_context: dict with tiposTurismo, presupuesto_bucket, etc.
98
99     Returns:
100         list of float scores in [1, 5], same length as item_ids.
101     """
102     if not self.is_fitted:
103         return [3.0] * len(item_ids)
104
105     try:
106         from sklearn.metrics.pairwise import cosine_similarity

```

```

107
108     query_text = self._build_query(user_context)
109     query_vec = self._tfidf.transform([query_text])
110
111     # Build index from item_id -> matrix row
112     id_to_row = {iid: i for i, iid in enumerate(self._item_ids)}
113
114     scores = []
115     for item_id in item_ids:
116         row_idx = id_to_row.get(str(item_id))
117         if row_idx is None:
118             scores.append(3.0)
119             continue
120         sim = float(cosine_similarity(query_vec, self._item_matrix[row_idx])[0, 0])
121         # Map cosine sim [0, 1] -> [1, 5]: 0 sim -> 1.0, perfect sim -> 5.0
122         scores.append(float(np.clip(1.0 + sim * 4.0, 1.0, 5.0)))
123
124     return scores
125 except Exception as exc:
126     _logger.warning(f"[ContentModel] Error en score(): {exc}")
127     return [3.0] * len(item_ids)
128
129 def _build_query(self, user_context: dict) -> str:
130     """Builds a pseudo-document representing user preferences for TF-IDF matching."""
131     if not user_context:
132         return 'culture gastronomy park restaurant'
133
134     parts = []
135
136     # Tourism types -> pseudo-words
137     tipos = user_context.get('tiposTurismo', [])
138     if isinstance(tipos, str):
139         tipos = [tipos]
140     for t in tipos:
141         q = _TURISMO_QUERY.get(t, '')
142         if q:
143             parts.append(q)
144
145     # Boolean prefs -> pseudo-words
146     if user_context.get('pref_outdoor', False):
147         parts.append('outdoor nature park hiking')
148     if user_context.get('wants_tours', False):
149         parts.append('tours excursion adventure')
150     if user_context.get('needs_hotel', False):
151         parts.append('hotel accommodation rural')
152     if user_context.get('requiere_accesibilidad', False):
153         parts.append('accessible accessibility')
154
155     # Group type hints
156     gtype = str(user_context.get('group_type', '')).lower()
157     if gtype == 'familia':
158         parts.append('kids family children')
159     elif gtype == 'pareja':
160         parts.append('romantic couple')
161
162     # Budget hint
163     budget = str(user_context.get('presupuesto_bucket', 'medio')).lower()
164     price_hint = _BUDGET_PRICE_HINT.get(budget, '2')
165     parts.append(f'price_{price_hint}')
166

```

```

167         return ' '.join(parts) if parts else 'culture gastronomy park restaurant'

===== MODEL0/src/lightfm_model.py =====
1 """
2 SMARTUR LightFM Model – Factorización Matricial con side features (WARP loss).
3
4 Por qué LightFM resuelve el cold-start de SMARTUR:
5 - Todos los usuarios reales son cold-start en CF (IDs enteros vs. UUIDs Yelp)
6 - LightFM aprende embeddings TANTO de ítems como de sus features (categorías, precio...)
7 - Para un usuario nuevo, predice con sus features de perfil (tiposTurismo, grupo, etc.)
8   sin necesitar historial de interacciones
9
10 Pérdida WARP (Weighted Approximate-Rank Pairwise):
11 Optimiza directamente la calidad del ranking (NDCG), mejor que MSE para listas Top-N.
12 """
13
14 import os
15 import re
16 import logging
17 import numpy as np
18 import pandas as pd
19 import joblib
20
21 _DIR = os.path.dirname(os.path.abspath(__file__))
22 _MODELS = os.path.join(_DIR, '..', 'models')
23 _logger = logging.getLogger('smartur-api')
24
25 # — Feature definitions —————
26
27 USER_FEATURE_NAMES = [
28     'uf:tur_naturaleza', 'uf:tur_aventura', 'uf:tur_gastronomico',
29     'uf:tur_cultural', 'uf:tur_rural',
30     'uf:group_solo', 'uf:group_pareja', 'uf:group_familia', 'uf:group_amigos',
31     'uf:budget_1', 'uf:budget_2', 'uf:budget_3', 'uf:budget_4',
32     'uf:wants_accessibility', 'uf:pref_outdoor',
33     'uf:wants_tours', 'uf:needs_hotel', 'uf:pref_food',
34 ]
35
36 ITEM_FEATURE_NAMES = [
37     'if:price_1', 'if:price_2', 'if:price_3', 'if:price_4',
38     'if:is_accessible', 'if:outdoor', 'if:is_good_for_kids', 'if:is_romantic',
39     'if:cat_nature', 'if:cat_culture', 'if:cat_gastronomy',
40     'if:cat_adventure', 'if:cat_rural',
41     'if:cat_waterfall', 'if:cat_volcano', 'if:cat_museum',
42     'if:cat_park', 'if:cat_hiking',
43     'if:cat_hotel', 'if:cat_restaurant', 'if:cat_tours',
44 ]
45
46 _TOURISM_PATTERNS = {
47     'naturaleza': r'park|hiking|nature|waterfall|volcano|mountain|viewpoint|botanical',
48     'aventura': r'adventure|hiking|active life|rafting|mountain biking|volcano',
49     'gastronomico': r'restaurant|food|caf[eé]|gastronomy|market|local food',
50     'cultural': r'museum|culture|history|hacienda|monument|cathedral|sanctuary|architecture',
51     'rural': r'rural|hacienda|bed & breakfast|campground|farm',
52 }
53
54 _ITEM_CAT_PATTERNS = {
55     'cat_nature': r'nature|park|hiking|waterfall|volcano|mountain|viewpoint|outdoor',
56     'cat_culture': r'culture|museum|history|hacienda|monument|cathedral|sanctuary',
57     'cat_gastronomy': r'gastronomy|restaurant|food|market|local food',

```

```

58 'cat_adventure': r'adventure|hiking|active|rafting|mountain biking',
59 'cat_rural': r'rural|hacienda|bed & breakfast|campground',
60 'cat_waterfall': r'waterfall|cascad',
61 'cat_volcano': r'volcano',
62 'cat_museum': r'museum',
63 'cat_park': r'\bpark\b|parque',
64 'cat_hiking': r'hiking|senderismo',
65 'cat_hotel': r'hotel|accommodation|hostel',
66 'cat_restaurant': r'restaurant|food|caf[eé]|gastronomy',
67 'cat_tours': r'tours?\b|excursion',
68 }
69
70 _BUDGET_MAP = {'bajo': 1, 'medio': 2, 'alto': 3, 'premium': 4}
71
72
73 # — Feature builders (module-level, reused in train + inference) —————
74
75 def context_to_user_features(context: dict) -> list:
76     """
77     Converts a context dict (from mobile form or traveler profile) to a list of
78     active LightFM user feature tags (e.g. ['uf:tur_naturaleza', 'uf:budget_2']).
79     """
80     if not context:
81         return ['uf:budget_2', 'uf:group_amigos', 'uf:pref_food']
82
83     active = []
84
85     # Tourism types (multi-hot)
86     tipos = context.get('tiposTurismo', [])
87     if isinstance(tipos, str):
88         tipos = [tipos]
89     for t in ['naturaleza', 'aventura', 'gastronomico', 'cultural', 'rural']:
90         if t in tipos:
91             active.append(f'uf:tur_{t}')
92
93     # Group type (one-hot)
94     gtype = str(context.get('group_type', 'amigos')).lower().strip()
95     matched_group = False
96     for g in ['solo', 'pareja', 'familia', 'amigos']:
97         if g == gtype:
98             active.append(f'uf:group_{g}')
99             matched_group = True
100             break
101     if not matched_group:
102         active.append('uf:group_amigos')
103
104     # Budget (one-hot)
105     braw = str(context.get('presupuesto_bucket', 'medio')).lower().strip()
106     b = _BUDGET_MAP.get(braw, 2)
107     active.append(f'uf:budget_{b}')
108
109     # Boolean preferences
110     if context.get('requiere_accesibilidad', False):
111         active.append('uf:wants_accessibility')
112     if context.get('pref_outdoor', False):
113         active.append('uf:pref_outdoor')
114     if context.get('wants_tours', False):
115         active.append('uf:wants_tours')
116     if context.get('needs_hotel', False):
117         active.append('uf:needs_hotel')

```

```

118     if context.get('pref_food', True):
119         active.append('uf:pref_food')
120
121     return active if active else ['uf:budget_2', 'uf:group_amigos']
122
123
124 def biz_row_to_item_features(row) -> list:
125     """
126     Converts a business row (dict or pandas Series) to a list of active
127     LightFM item feature tags.
128     """
129     if hasattr(row, 'to_dict'):
130         row = row.to_dict()
131
132     active = []
133
134     # Price level (one-hot)
135     price = max(1, min(4, int(row.get('price_level', 2) or 2)))
136     active.append(f'if:price_{price}')
137
138     # Binary attributes
139     for attr in ['is_accessible', 'outdoor', 'is_good_for_kids', 'is_romantic']:
140         if int(row.get(attr, 0) or 0):
141             active.append(f'if:{attr}')
142
143     # Category flags
144     cats = str(row.get('categories', '') or '').lower()
145     for feat, pattern in _ITEM_CAT_PATTERNS.items():
146         if re.search(pattern, cats):
147             active.append(f'if:{feat}')
148
149     return active if active else ['if:price_2']
150
151
152 # — Main class —————
153
154 class SmarturLightFMModel:
155     """
156     LightFM with WARP loss. Supports cold-start users via feature embeddings.
157
158     Training data: Yelp CSV reviews + optional real SMARTUR interactions
159     User features: tiposTurismo, grupo, presupuesto, prefs booleanas
160     Item features: categorías, precio, accesibilidad, outdoor, kids, romantic
161     Cold-start:   predice con embeddings de features del perfil del usuario
162                   (no necesita historial de ratings)
163     """
164
165     def __init__(self):
166         self._model = None
167         self._dataset = None
168         self._item_features_matrix = None # scipy sparse (n_items, n_item_features)
169         self._user_features_matrix = None # scipy sparse (n_users, n_user_features)
170         self._known_users: set = set()
171         self._known_items: set = set()
172         self._is_fitted: bool = False
173
174     # — Training —————
175
176     def train(self, reviews_df: pd.DataFrame, biz_df: pd.DataFrame) -> bool:
177         """

```

```

178     Trains LightFM.
179
180     Args:
181         reviews_df: DataFrame with [user_id, business_id, stars]
182         biz_df:      DataFrame with [business_id, categories, price_level, ...]
183                     Can also have 'id' column (local POIs).
184     Returns:
185         True on success, False if lightfm is not installed.
186     """
187     try:
188         from lightfm import LightFM
189         from lightfm.data import Dataset
190     except ImportError:
191         _logger.error("[LightFM] Librería no instalada. Ejecutar: pip install 'lightfm>=1.17'"
192     )
193     return False
194
195     _logger.info("[LightFM] Iniciando entrenamiento...")
196
197     # Normalize IDs to str
198     rev = reviews_df.copy()
199     rev['user_id'] = rev['user_id'].astype(str)
200     rev['business_id'] = rev['business_id'].astype(str)
201
202     # Use only ratings >= 3.5 as positive interactions (implicit feedback)
203     pos_rev = rev[rev['stars'].astype(float) >= 3.5]
204     if len(pos_rev) == 0:
205         _logger.warning("[LightFM] No hay interacciones positivas (stars >= 3.5). Abortando.")
206         return False
207
208     user_ids = rev['user_id'].unique().tolist()
209     item_ids = rev['business_id'].unique().tolist()
210
211     _logger.info(f"[LightFM] Dataset: {len(user_ids)} usuarios, {len(item_ids)} ítems, "
212                f"{len(pos_rev)} interacciones positivas")
213
214     # — Build LightFM Dataset —————
215     dataset = Dataset()
216     dataset.fit(
217         users=user_ids,
218         items=item_ids,
219         user_features=USER_FEATURE_NAMES,
220         item_features=ITEM_FEATURE_NAMES,
221     )
222
223     # Interactions matrix (weighted by stars)
224     interactions, weights = dataset.build_interactions([
225         (r['user_id'], r['business_id'], float(r['stars']))
226         for _, r in pos_rev.iterrows()
227     ])
228
229     # — User features (inferred from liked items) —————
230     _logger.info("[LightFM] Construyendo features de usuario...")
231     user_features_list = self._build_training_user_features(rev)
232     user_features_matrix = dataset.build_user_features(user_features_list)
233
234     # — Item features —————
235     _logger.info("[LightFM] Construyendo features de ítem...")
236     biz_lookup = self._build_biz_lookup(biz_df)
237     item_features_list = [

```

```

237         (iid, biz_row_to_item_features(biz_lookup.get(iid, {})))
238         for iid in item_ids
239     ]
240     item_features_matrix = dataset.build_item_features(item_features_list)
241
242     # --- Train ---
243     model = LightFM(
244         no_components=96,
245         loss='warp',
246         learning_rate=0.05,
247         item_alpha=1e-6,
248         user_alpha=1e-6,
249         random_state=42,
250     )
251
252     n_epochs = 50
253     _logger.info(f"[LightFM] Entrenando {n_epochs} epochs con pérdida WARP (96 componentes)...")
254
255     model.fit(
256         interactions,
257         user_features=user_features_matrix,
258         item_features=item_features_matrix,
259         sample_weight=weights,
260         epochs=n_epochs,
261         num_threads=4,
262         verbose=False,
263     )
264
265     self._model = model
266     self._dataset = dataset
267     self._item_features_matrix = item_features_matrix
268     self._user_features_matrix = user_features_matrix
269     self._known_users = set(user_ids)
270     self._known_items = set(item_ids)
271     self.is_fitted = True
272
273     _logger.info(
274         f"[LightFM] Listo: {n_epochs} epochs, loss=warp, "
275         f"{interactions.shape[0]} usuarios, {interactions.shape[1]} ítems"
276     )
277     self.save()
278     return True
279
280 def _build_training_user_features(self, rev: pd.DataFrame) -> list:
281     """
282     Infers user feature vectors from their liked items.
283     Fully vectorized via pandas groupby().any() – handles 60k+ users in seconds.
284     """
285     if 'categories' not in rev.columns:
286         return [(uid, ['uf:budget_2', 'uf:group_amigos'])
287                 for uid in rev['user_id'].unique()]
288
289     cats = rev['categories'].fillna('').str.lower()
290     liked = rev['stars'].astype(float) >= 4.0
291     uids = rev['user_id']
292
293     # Vectorized: one groupby per feature (single pass through data per pattern)
294     def _user_has(pattern):
295         return (liked & cats.str.contains(pattern, regex=True, na=False)).groupby(uids).any()

```

```

296     feat_series = {
297         f'uf:tur_{t}': _user_has(p) for t, p in _TOURISM_PATTERNS.items()
298     }
299     feat_series['uf:pref_outdoor'] = _user_has(
300         r'park|hiking|nature|outdoor|volcano|mountain')
301     feat_series['uf:wants_tours'] = _user_has(r'tours?\b|excursion')
302     feat_series['uf:pref_food'] = _user_has(r'restaurant|food|caf[eé]|gastronomy')
303
304     all_uids = uids.unique()
305     result = []
306     for uid in all_uids:
307         active = ['uf:group_amigos', 'uf:budget_2'] # stable defaults
308         for feat, series in feat_series.items():
309             if uid in series.index and series[uid]:
310                 active.append(feat)
311         result.append((uid, active))
312
313     return result
314
315 def _build_biz_lookup(self, biz_df: pd.DataFrame) -> dict:
316     """Returns a {business_id_str -> row_dict} lookup."""
317     if biz_df is None or biz_df.empty:
318         return {}
319     df = biz_df.copy()
320     if 'business_id' not in df.columns and 'id' in df.columns:
321         df['business_id'] = df['id'].astype(str)
322     if 'business_id' in df.columns:
323         df['business_id'] = df['business_id'].astype(str)
324     return df.set_index('business_id').to_dict('index')
325     return {}
326
327 # — Inference —————
328
329 def predict(self, user_id: str, item_ids: list, user_context: dict = None) -> list:
330     """
331     Scores items for a user. Returns a list of scores ∈ [1, 5].
332
333     - If user_context is supplied (real form data), always uses feature-based
334       scoring – more accurate than the inferred training features.
335     - Otherwise falls back to warm (trained embedding) for known users, or
336       cold-start feature scoring for unknown users.
337     """
338     if not self.is_fitted or not item_ids:
339         return [3.0] * len(item_ids)
340
341     # When we have real context (from the mobile form), prefer it
342     if user_context:
343         return self._predict_with_context_features(item_ids, user_context)
344
345     str_uid = str(user_id)
346     if str_uid in self._known_users:
347         return self._predict_warm(str_uid, item_ids)
348
349     # Cold-start without context -> neutral scores based on item popularity
350     return self._predict_cold_item_bias(item_ids)
351
352 def _predict_warm(self, user_id: str, item_ids: list) -> list:
353     """Prediction using trained user embedding (warm user)."""
354     try:
355         user_map, _, item_map, _ = self._dataset.mapping()

```

```

356         uid_idx = user_map.get(user_id)
357         if uid_idx is None:
358             return self._predict_cold_item_bias(item_ids)
359
360         scores = []
361         for item_id in item_ids:
362             iid_idx = item_map.get(str(item_id))
363             if iid_idx is None:
364                 scores.append(3.0)
365                 continue
366             raw = float(self._model.predict(
367                 user_ids=np.array([uid_idx], dtype=np.int32),
368                 item_ids=np.array([iid_idx], dtype=np.int32),
369                 item_features=self._item_features_matrix,
370                 user_features=self._user_features_matrix,
371                 num_threads=1,
372             )[0])
373             scores.append(float(np.clip(raw * 2.0 + 3.0, 1.0, 5.0)))
374         return scores
375     except Exception as exc:
376         _logger.warning(f"[LightFM] Warm predict error: {exc}")
377         return self._predict_cold_item_bias(item_ids)
378
379 def _predict_with_context_features(self, item_ids: list, user_context: dict) -> list:
380     """
381     Cold-start-aware scoring using LightFM's learned feature embeddings.
382
383     Manually computes:
384     score = user_feature_bias + item_bias + item_feature_bias
385           + dot(user_feature_emb, item_emb + item_feature_emb)
386
387     This is the correct LightFM prediction formula when the user's embedding
388     comes purely from their feature vector (no identity embedding).
389     """
390     try:
391         _, uf_map, item_map, _ = self._dataset.mapping()
392
393         # uf_map/item_feature_mapping YA vienen offseados por LightFM:
394         # Dataset.fit_partial() llena primero las identidades de usuario/ítem
395         # (índices 0..n-1) y sigue con el MISMO contador para las features
396         # declaradas – así que uf_map[tag] y las columnas de
397         # _item_features_matrix ya son índices globales válidos dentro de
398         # user_embeddings/item_embeddings. Sumarles n_users/n_items de nuevo
399         # duplicaba el offset (bug real: "index out of bounds", visto en
400         # producción con índices ~2x el tamaño real del embedding).
401         user_feat_tags = context_to_user_features(user_context)
402         uf_embed_idx = [uf_map[tag] for tag in user_feat_tags if tag in uf_map]
403
404         if uf_embed_idx:
405             user_repr = np.sum(self._model.user_embeddings[uf_embed_idx], axis=0)
406             user_bias = float(np.sum(self._model.user_biases[uf_embed_idx]))
407         else:
408             user_repr = np.zeros(self._model.no_components)
409             user_bias = 0.0
410
411         scores = []
412         for item_id in item_ids:
413             iid_idx = item_map.get(str(item_id))
414             if iid_idx is None:
415                 scores.append(3.0)

```

```

416         continue
417
418         # Item representation: identity + feature embeddings.
419         # LightFM ya incluye la columna de identidad del ítem dentro
420         # de item_feat_row.indices (Dataset.FeatureBuilder.build()
421         # agrega esa entrada automáticamente) – sumar iid_idx aparte
422         # lo contaba dos veces.
423         item_feat_row = self._item_features_matrix[iid_idx]
424         if_embed_idx = list(item_feat_row.indices)
425         item_repr = np.sum(self._model.item_embeddings[if_embed_idx], axis=0)
426         item_bias = float(np.sum(self._model.item_biases[if_embed_idx]))
427
428         raw = user_bias + item_bias + float(np.dot(user_repr, item_repr))
429         scores.append(float(np.clip(raw * 2.0 + 3.0, 1.0, 5.0)))
430
431     return scores
432 except Exception as exc:
433     _logger.warning(f"[LightFM] Context-feature predict error: {exc}")
434     return self._predict_cold_item_bias(item_ids)
435
436 def _predict_cold_item_bias(self, item_ids: list) -> list:
437     """
438     Last-resort fallback: rank items by their LightFM item bias (popularity signal).
439     Returns scores ∈ [1, 5].
440     """
441     try:
442         _, _, item_map, _ = self._dataset.mapping()
443         scores = []
444         for item_id in item_ids:
445             iid_idx = item_map.get(str(item_id))
446             if iid_idx is None:
447                 scores.append(3.0)
448                 continue
449             raw = float(self._model.item_biases[iid_idx])
450             scores.append(float(np.clip(raw * 2.0 + 3.0, 1.0, 5.0)))
451         return scores
452     except Exception:
453         return [3.0] * len(item_ids)
454
455 # — Persistence —————
456
457 def save(self, path: str = None) -> None:
458     if path is None:
459         path = os.path.join(_MODELS, 'lightfm_model.joblib')
460     os.makedirs(os.path.dirname(path), exist_ok=True)
461     joblib.dump({
462         'model': self._model,
463         'dataset': self._dataset,
464         'item_features_matrix': self._item_features_matrix,
465         'user_features_matrix': self._user_features_matrix,
466         'known_users': self._known_users,
467         'known_items': self._known_items,
468     }, path)
469     _logger.info(f"[LightFM] Modelo guardado en {path}")
470
471 def load(self, path: str = None) -> bool:
472     if path is None:
473         path = os.path.join(_MODELS, 'lightfm_model.joblib')
474     if not os.path.exists(path):
475         return False

```

```

476         try:
477             data = joblib.load(path)
478             self._model = data['model']
479             self._dataset = data['dataset']
480             self._item_features_matrix = data['item_features_matrix']
481             self._user_features_matrix = data.get('user_features_matrix')
482             self._known_users = data.get('known_users', set())
483             self._known_items = data.get('known_items', set())
484             self.is_fitted = True
485             _logger.info(f"[LightFM] Modelo cargado desde {path}")
486             return True
487         except Exception as exc:
488             _logger.warning(f"[LightFM] Error cargando modelo: {exc}")
489             return False

===== MODELO/src/gbm_model.py =====
1  """
2  Tercer algoritmo ML: Gradient Boosting contextual (mismas features que RF).
3  Complementa CF+KNN y Random Forest en la comparativa de modelos.
4  """
5  import json
6  import os
7  import joblib
8  import numpy as np
9  import pandas as pd
10 from sklearn.ensemble import GradientBoostingRegressor
11
12 from rf_model import SmarturContextModel, _DATA, _MODELS, _compute_dist_km
13
14
15 class SmarturGbmModel(SmarturContextModel):
16     """Gradient Boosting Regressor sobre vector [Item + User + Match]."""
17
18     def __init__(self, business_path=None):
19         super().__init__(business_path=business_path)
20         self.model = GradientBoostingRegressor(
21             n_estimators=150,
22             max_depth=6,
23             learning_rate=0.08,
24             min_samples_leaf=8,
25             subsample=0.85,
26             random_state=42,
27         )
28
29     def train(self, reviews_df, df_biz_extra=None):
30         if df_biz_extra is not None:
31             biz_ids_extra = df_biz_extra['business_id'].unique()
32             self.df_biz = pd.concat([
33                 self.df_biz[~self.df_biz['business_id'].isin(biz_ids_extra)],
34                 df_biz_extra
35             ], ignore_index=True)
36             print(f"[GBM] Añadidos {len(df_biz_extra)} negocios extra (Rest-Mex)")
37             train_df = reviews_df.merge(self.df_biz, on='business_id', suffixes=('_user', '_biz'))
38             self._extract_top_categories(train_df)
39
40             # Compute geographic distance from Altas Montañas center (proxy for training data)
41             train_df['dist_km'] = _compute_dist_km(train_df)
42
43             train_df = self._add_category_features(train_df)
44             train_df = self._simulate_user_contexts(train_df)

```

```

45         self.features = self.numeric_features + self.cat_features + self.encoder.all_context_featu
46 re_names
47     X = train_df[self.features].fillna(0)
48     y = train_df['stars_user']
49
50     print(
51         f"GBM contextual: entrenando sobre {X.shape[0]} interacciones "
52         f"con {X.shape[1]} variables."
53     )
54     self.model.fit(X, y)
55
56     # — Feature importances —————
57     fi = pd.DataFrame({
58         'feature': self.features,
59         'importance': self.model.feature_importances_,
60     }).sort_values('importance', ascending=False)
61     fi.to_json(os.path.join(_MODELS, 'gbm_feature_importances.json'), orient='records', indent
=2)
62
63     # — Hyperparameters —————
64     hp = {k: str(v) for k, v in self.model.get_params().items()}
65     with open(os.path.join(_MODELS, 'gbm_hyperparams.json'), 'w') as _f:
66         json.dump(hp, _f, indent=2)
67     print(f"[GBM] Feature importances + hyperparams guardados. Top: {fi.head(1)['feature'].val
ues[0]}")
68
69     os.makedirs(_MODELS, exist_ok=True)
70     joblib.dump(
71         {
72             'model': self.model,
73             'top_categories': self.top_categories,
74             'features': self.features,
75         },
76         os.path.join(_MODELS, 'gbm_context_yelp.joblib'),
77     )
78     self.is_fitted = True
79     print("Gradient Boosting contextual entrenado y guardado.")
80
81 def load(self, model_path=None):
82     if model_path is None:
83         model_path = os.path.join(_MODELS, 'gbm_context_yelp.joblib')
84     if os.path.exists(model_path):
85         data = joblib.load(model_path)
86         self.model = data['model']
87         self.top_categories = data['top_categories']
88         self.features = data['features']
89         self.is_fitted = True
90         self.cat_features = [f'cat_{c}' for c in self.top_categories]
91         print("Gradient Boosting contextual cargado desde disco.")
92         return True
93     return False

```

===== MODELO/src/model_metrics.py ====

```

1 """
2 Comparativa formal de los 3 algoritmos ML y persistencia de métricas / configuración óptima.
3 """
4 import json
5 import logging
6 import os

```

```

7 from math import sqrt
8
9 import numpy as np
10 from sklearn.metrics import mean_absolute_error, mean_squared_error
11
12 from cf import predict_cf_pearson
13
14 _DIR = os.path.dirname(os.path.abspath(__file__))
15 _METRICS_PATH = os.path.join(_DIR, '..', 'models', 'algorithm_metrics.json')
16 logger = logging.getLogger(__name__)
17
18 DEFAULT_METRICS = {
19     'best_algorithm': 'hybrid',
20     'best_alpha': 0.2,
21     'local_blend': {'rf': 0.55, 'gbm': 0.45},
22     'algorithms': {},
23     'ranking': {},
24     'error_distribution': {},
25     'prediction_distribution': {},
26     'data_quality': {},
27 }
28
29
30 def _rmse_mae(actuals, preds):
31     actuals = np.asarray(actuals, dtype=float)
32     preds = np.asarray(preds, dtype=float)
33     return {
34         'rmse': float(sqrt(mean_squared_error(actuals, preds))),
35         'mae': float(mean_absolute_error(actuals, preds)),
36     }
37
38
39 def compare_algorithms(engine, rf_model, gbm_model, sample_size=5000, hybrid_alpha=0.2):
40     """
41     Evalúa CF+KNN, Random Forest y Gradient Boosting; elige el mejor por RMSE.
42     Retorna métricas y pesos recomendados para producción.
43     """
44     from evaluate import _infer_user_context, evaluar_ranking
45     from cf import reset_prediction_stats, get_prediction_stats
46
47     n_eval = min(sample_size, len(engine.test_data))
48     test_sample = engine.test_data.sample(n_eval, random_state=42)
49     reset_prediction_stats()
50
51     actuals, preds_cf, preds_rf, preds_gbm = [], [], [], []
52     user_contexts = {
53         uid: _infer_user_context(uid, engine)
54         for uid in test_sample['user_id'].unique()
55     }
56
57     business_ids = []
58     for _, row in test_sample.iterrows():
59         try:
60             p_cf = predict_cf_pearson(row['user_id'], row['business_id'], engine)
61             ctx = user_contexts.get(row['user_id'])
62             p_rf = float(
63                 rf_model.predict_with_context([row['business_id']], user_context=ctx)[0]
64             )
65             p_gbm = float(
66                 gbm_model.predict_with_context([row['business_id']], user_context=ctx)[0]

```

```

67         )
68         if np.isnan(p_cf) or np.isnan(p_rf) or np.isnan(p_gbm):
69             continue
70         actuals.append(row['stars'])
71         preds_cf.append(p_cf)
72         preds_rf.append(p_rf)
73         preds_gbm.append(p_gbm)
74         business_ids.append(row['business_id'])
75     except Exception:
76         continue
77
78 if len(actuals) < 50:
79     return DEFAULT_METRICS
80
81 actuals = np.array(actuals)
82 preds_cf = np.array(preds_cf)
83 preds_rf = np.array(preds_rf)
84 preds_gbm = np.array(preds_gbm)
85
86 media = float(engine.train_data['stars'].mean())
87 preds_baseline = np.full_like(actuals, media)
88
89 # item_mean: promedio de calificaciones por negocio en train, aplicado al
90 # test set. Ver nota igual en api.py: compute_simple_metrics – es el
91 # candidato más barato (sin entrenamiento) y el más "justo" dado que los
92 # datos sintéticos se generan centrados en el promedio de cada negocio.
93 item_means_train = engine.train_data.groupby('business_id')['stars'].mean()
94 preds_item_mean = np.array(
95     [float(item_means_train.get(bid, media)) for bid in business_ids]
96 )
97
98 metrics = {
99     'baseline': _rmse_mae(actuals, preds_baseline),
100     'item_mean': _rmse_mae(actuals, preds_item_mean),
101     'cf_knn_pearson': _rmse_mae(actuals, preds_cf),
102     'random_forest': _rmse_mae(actuals, preds_rf),
103     'gradient_boosting': _rmse_mae(actuals, preds_gbm),
104 }
105
106 # Alpha grid search constrained to [0.1, 0.9] to guarantee genuine blending.
107 # Unconstrained search can collapse to alpha=1.0 (pure CF) when RF underperforms,
108 # making hybrid indistinguishable from CF in the dashboard.
109 alphas = np.arange(0.1, 0.95, 0.1)
110 best_alpha, best_hybrid_rmse = hybrid_alpha, float('inf')
111 for alpha in alphas:
112     hybrid = alpha * preds_cf + (1 - alpha) * preds_rf
113     rmse = sqrt(mean_squared_error(actuals, hybrid))
114     if rmse < best_hybrid_rmse:
115         best_hybrid_rmse = rmse
116         best_alpha = float(round(alpha, 1))
117
118 hybrid_preds = best_alpha * preds_cf + (1 - best_alpha) * preds_rf
119 metrics['hybrid_cf_rf'] = {
120     **rmse_mae(actuals, hybrid_preds),
121     'alpha': best_alpha,
122 }
123
124 triple = 0.15 * preds_cf + 0.50 * preds_rf + 0.35 * preds_gbm
125 metrics['hybrid_triple'] = _rmse_mae(actuals, triple)
126

```

```

127 # 'baseline'/'item_mean' Sí compiten – antes 'baseline' se excluía a
128 # propósito, lo que garantizaba que el sistema nunca pudiera admitir que
129 # predecir el promedio simple le gana a CF/RF/GBM/híbrido. 'item_mean' es
130 # aún más informativo: si gana, ni CF ni RF están aportando nada por
131 # encima del promedio de calificaciones de cada negocio.
132 candidates = {
133     'baseline': metrics['baseline']['rmse'],
134     'item_mean': metrics['item_mean']['rmse'],
135     'cf_knn_pearson': metrics['cf_knn_pearson']['rmse'],
136     'random_forest': metrics['random_forest']['rmse'],
137     'gradient_boosting': metrics['gradient_boosting']['rmse'],
138     'hybrid_cf_rf': metrics['hybrid_cf_rf']['rmse'],
139     'hybrid_triple': metrics['hybrid_triple']['rmse'],
140 }
141 best_algorithm = min(candidates, key=candidates.get)
142 if best_algorithm in ('baseline', 'item_mean'):
143     logger.warning(
144         f"[compare_algorithms] '{best_algorithm}' (RMSE={metrics[best_algorithm]['rmse']:.4f})
145         "
146         f"le gana a CF/RF/GBM/híbrido – el modelo no aporta valor medible hoy."
147     )
148 rf_rmse = metrics['random_forest']['rmse']
149 gbm_rmse = metrics['gradient_boosting']['rmse']
150 total_inv = (1 / rf_rmse) + (1 / gbm_rmse)
151 w_rf = (1 / rf_rmse) / total_inv
152 w_gbm = (1 / gbm_rmse) / total_inv
153
154 cf_stats = get_prediction_stats()
155 if cf_stats.get('fallback_rate') is not None and cf_stats['fallback_rate'] > 0.5:
156     logger.warning(
157         f"[compare_algorithms] {cf_stats['fallback_rate']*100:.1f}% de las predicciones de "
158         f"CF cayeron en fallback (sin vecinos con señal real) – dataset demasiado disperso."
159     )
160
161 result = {
162     'best_algorithm': best_algorithm,
163     'best_alpha': best_alpha if best_algorithm == 'hybrid_cf_rf' else hybrid_alpha,
164     'production_alpha': hybrid_alpha,
165     'local_blend': {'rf': round(w_rf, 3), 'gbm': round(w_gbm, 3)},
166     'algorithms': metrics,
167     'sample_size': len(actuals),
168     'cf_prediction_sources': cf_stats,
169 }
170
171 # Enrich with error_distribution, prediction_distribution, data_quality
172 _actuals = np.asarray(actuals, dtype=float)
173 errors_abs = np.abs(_actuals - hybrid_preds)
174 ed = {}
175 for t in [0.5, 1.0, 1.5, 2.0]:
176     key = f'within_{str(t).replace(".", "_')}'
177     ed[key] = round(float((errors_abs <= t).mean() * 100), 1)
178 result['error_distribution'] = ed
179
180 actual_buckets = []
181 pred_buckets = []
182 for star in range(1, 6):
183     actual_buckets.append([star, int((_actuals == star).sum())])
184     pred_buckets.append([
185         star,

```

```

186         int(((hybrid_preds >= star - 0.5) & (hybrid_preds < star + 0.5)).sum()),
187     ])
188     result['prediction_distribution'] = {
189         'actual_buckets': actual_buckets,
190         'predicted_buckets': pred_buckets,
191     }
192
193     if engine is not None:
194         # Augment data_quality with real SMARTUR interaction count if available
195         real_count = 0
196         try:
197             from poi_repository import fetch_real_interactions
198             real_df = fetch_real_interactions()
199             real_count = len(real_df) if real_df is not None else 0
200         except Exception:
201             pass
202
203         result['data_quality'] = {
204             'total_interactions': int(len(engine.train_data)) if hasattr(engine, 'train_data') and
engine.train_data is not None else 0,
205             'total_test': int(len(engine.test_data)) if hasattr(engine, 'test_data') and engine.te
st_data is not None else 0,
206             'users_count': int(engine.user_item_matrix.shape[0]) if engine and engine.user_item_ma
trix is not None else 0,
207             'businesses count': int(engine.user_item_matrix.shape[1]) if engine and engine.user_it
em_matrix is not None else 0,
208             'top_categories': getattr(rf_model, 'top_categories', [])[:10],
209             'features_count': len(getattr(rf_model, 'features', [])),
210             'real_smartur_interactions': real_count,
211             'uses_real_data': real_count >= 10,
212         }
213
214     # Ranking metrics (NDCG@5, Precision@5, Hit Rate@10)
215     try:
216         ranking = evaluar_ranking(engine, rf_model, n_users=150, k=5)
217         result['ranking'] = {
218             'ndcg_at_5': round(ranking.get('ndcg', 0.0), 4),
219             'precision_at_5': round(ranking.get('precision', 0.0), 4),
220             'hit_rate_at_10': round(ranking.get('hit_rate', 0.0), 4),
221             'users_evaluated': ranking.get('n_users_evaluated', 0),
222         }
223     except Exception as e:
224         result['ranking'] = {'ndcg_at_5': None, 'precision_at_5': None, 'hit_rate_at_10': None, 'e
rror': str(e)}
225
226     return result
227
228
229 def save_metrics(metrics):
230     os.makedirs(os.path.dirname(_METRICS_PATH), exist_ok=True)
231     with open(_METRICS_PATH, 'w', encoding='utf-8') as f:
232         json.dump(metrics, f, indent=2, ensure_ascii=False)
233
234
235 def load_metrics():
236     if not os.path.exists(_METRICS_PATH):
237         return dict(DEFAULT_METRICS)
238     try:
239         with open(_METRICS_PATH, 'r', encoding='utf-8') as f:
240             data = json.load(f)

```

```

241         merged = dict(DEFAULT_METRICS)
242         merged.update(data)
243         if 'local_blend' not in merged or not merged['local_blend']:
244             merged['local_blend'] = DEFAULT_METRICS['local_blend']
245         return merged
246     except (json.JSONDecodeError, OSError):
247         return dict(DEFAULT_METRICS)

===== MODELO/src/cross_validation.py =====
1  """
2  K-fold cross-validation para los 3 algoritmos principales del motor de
3  recomendación: CF Pearson KNN, Random Forest y Gradient Boosting.
4
5  Complementa (no reemplaza) el train/test split de 80/20 que ya usan
6  model_metrics.py / api.py._compute_simple_metrics. Cada algoritmo se
7  reentrena K veces sobre una muestra del dataset, dejando fuera un fold
8  distinto cada vez para evaluar, y se reporta la media/desviación de
9  RMSE, MAE y tiempo de ejecución por fold.
10 """
11 import logging
12 import time
13
14 import numpy as np
15 import pandas as pd
16 from math import sqrt
17 from sklearn.metrics import mean_absolute_error, mean_squared_error
18 from sklearn.model_selection import KFold
19
20 from cf import predict_cf_pearson
21 from engine import SmarturEngine
22 from evaluate import _infer_user_context
23 from gbm_model import SmarturGbmModel
24 from rf_model import SmarturContextModel
25
26 logger = logging.getLogger(__name__)
27
28 DEFAULT_K = 5
29 DEFAULT_SAMPLE_SIZE = 3000
30
31
32 def _rmse_mae(actuals, preds) -> dict:
33     return {
34         'rmse': float(sqrt(mean_squared_error(actuals, preds))),
35         'mae': float(mean_absolute_error(actuals, preds)),
36     }
37
38
39 def _make_fold_engine(train_fold_df: pd.DataFrame, df_biz: pd.DataFrame) -> SmarturEngine:
40     """
41     Instancia liviana de SmarturEngine que NO lee CSVs de disco (bypassa
42     __init__) - solo sirve para reutilizar prepare_pearson_matrix() y
43     _infer_user_context() sobre los datos de un fold específico.
44     """
45     fold_engine = object.__new__(SmarturEngine)
46     fold_engine.train_data = train_fold_df
47     fold_engine.df_biz = df_biz
48     fold_engine._user_idx_map = None
49     fold_engine._biz_idx_map = None
50     fold_engine.prepare_pearson_matrix()
51     return fold_engine

```

```

52
53
54 def run_kfold_cv(
55     engine: SmarturEngine,
56     k: int = DEFAULT_K,
57     sample_size: int = DEFAULT_SAMPLE_SIZE,
58     random_state: int = 42,
59 ) -> dict:
60     """
61     Ejecuta k-fold cross-validation para cf_knn_pearson, random_forest y
62     gradient_boosting sobre una muestra de `sample_size` interacciones
63     (train + test combinados del engine ya cargado). Retorna un dict listo
64     para persistir/serializar con RMSE/MAE promedio ± desviación estándar
65     y tiempo de ejecución promedio por algoritmo.
66     """
67     full_df = pd.concat([engine.train_data, engine.test_data], ignore_index=True)
68     n = min(sample_size, len(full_df))
69     sample = full_df.sample(n, random_state=random_state).reset_index(drop=True)
70
71     kf = KFold(n_splits=k, shuffle=True, random_state=random_state)
72
73     metrics = {
74         'cf_knn_pearson': {'rmse': [], 'mae': [], 'time_ms': []},
75         'random_forest': {'rmse': [], 'mae': [], 'time_ms': []},
76         'gradient_boosting': {'rmse': [], 'mae': [], 'time_ms': []},
77     }
78
79     fold_num = 0
80     for train_idx, test_idx in kf.split(sample):
81         fold_num += 1
82         train_fold = sample.iloc[train_idx].reset_index(drop=True)
83         test_fold = sample.iloc[test_idx].reset_index(drop=True)
84         logger.info(f"[cv] Fold {fold_num}/{k} - train={len(train_fold)}, test={len(test_fold)}")
85
86         # Engine liviano del fold, compartido por RF/GBM para inferir contexto
87         # de usuario (_infer_user_context) y por CF para el KNN/Pearson.
88         fold_ctx_engine = None
89         try:
90             fold_ctx_engine = object.__new__(SmarturEngine)
91             fold_ctx_engine.train_data = train_fold
92             fold_ctx_engine.df_biz = engine.df_biz
93         except Exception as exc:
94             logger.warning(f"[cv] Fold {fold_num}: no se pudo preparar engine de contexto: {exc}")
95
96         # — CF Pearson KNN —————
97         t0 = time.time()
98         try:
99             fold_cf_engine = _make_fold_engine(train_fold, engine.df_biz)
100             actuals, preds = [], []
101             for _, row in test_fold.iterrows():
102                 p = predict_cf_pearson(row['user_id'], row['business_id'], fold_cf_engine)
103                 if not np.isnan(p):
104                     actuals.append(row['stars'])
105                     preds.append(p)
106             if actuals:
107                 m = _rmse_mae(actuals, preds)
108                 metrics['cf_knn_pearson']['rmse'].append(m['rmse'])
109                 metrics['cf_knn_pearson']['mae'].append(m['mae'])
110             except Exception as exc:
111                 logger.warning(f"[cv] Fold {fold_num} CF falló: {exc}")

```

```

112     metrics['cf_knn_pearson']['time_ms'].append((time.time() - t0) * 1000)
113
114     # — Random Forest —————
115     t0 = time.time()
116     try:
117         rf = SmarturContextModel()
118         rf.train(train_fold, dynamic_override=True)
119         actuals, preds = [], []
120         for _, row in test_fold.iterrows():
121             ctx = _infer_user_context(row['user_id'], fold_ctx_engine) if fold_ctx_engine else
None
122             p = float(rf.predict_with_context([row['business_id']], user_context=ctx)[0])
123             actuals.append(row['stars'])
124             preds.append(p)
125             if actuals:
126                 m = _rmse_mae(actuals, preds)
127                 metrics['random_forest']['rmse'].append(m['rmse'])
128                 metrics['random_forest']['mae'].append(m['mae'])
129     except Exception as exc:
130         logger.warning(f"[cv] Fold {fold_num} RF falló: {exc}")
131     metrics['random_forest']['time_ms'].append((time.time() - t0) * 1000)
132
133     # — Gradient Boosting —————
134     t0 = time.time()
135     try:
136         gbm = SmarturGbmModel()
137         gbm.train(train_fold)
138         actuals, preds = [], []
139         for _, row in test_fold.iterrows():
140             ctx = _infer_user_context(row['user_id'], fold_ctx_engine) if fold_ctx_engine else
None
141             p = float(gbm.predict_with_context([row['business_id']], user_context=ctx)[0])
142             actuals.append(row['stars'])
143             preds.append(p)
144             if actuals:
145                 m = _rmse_mae(actuals, preds)
146                 metrics['gradient_boosting']['rmse'].append(m['rmse'])
147                 metrics['gradient_boosting']['mae'].append(m['mae'])
148     except Exception as exc:
149         logger.warning(f"[cv] Fold {fold_num} GBM falló: {exc}")
150     metrics['gradient_boosting']['time_ms'].append((time.time() - t0) * 1000)
151
152     summary = {'k': k, 'sample_size': n, 'algorithms': {}}
153     for algo, vals in metrics.items():
154         rsmes, maes, times = vals['rmse'], vals['mae'], vals['time_ms']
155         summary['algorithms'][algo] = {
156             'rmse_mean': float(np.mean(rsmes)) if rsmes else None,
157             'rmse_std': float(np.std(rsmes)) if rsmes else None,
158             'mae_mean': float(np.mean(maes)) if maes else None,
159             'mae_std': float(np.std(maes)) if maes else None,
160             'avg_execution_time_ms': float(np.mean(times)) if times else None,
161             'folds_completed': len(rsmes),
162         }
163     return summary

```

===== MODELO/src/evaluate.py =====

```

1 """
2 SMARTUR Evaluación v3
3 Incluye métricas de predicción (RMSE, MAE) y métricas de ranking (NDCG@K, Precision@K, Hit Rate@K)

```

```

4 """
5 import sys
6 import numpy as np
7 from sklearn.metrics import mean_squared_error, mean_absolute_error
8 from math import sqrt, log2
9 from engine import SmarturEngine
10 from rf_model import SmarturContextModel
11 from cf import predict_cf_pearson
12 from fusion import recommend_hybrid
13 from context_encoder import MAPEO_CATEGORIAS
14
15
16 def _infer_user_context(user_id, engine):
17     """
18     Deriva un contexto aproximado del usuario desde su historial de ratings en el
19     set de entrenamiento. Permite evaluar el sistema en condiciones cercanas a las
20     reales (con contexto) en lugar del modo cold-start sin preferencias.
21     """
22     user_train = engine.train_data[engine.train_data['user_id'] == user_id]
23     if user_train.empty:
24         return None
25
26     user_data = user_train.merge(
27         engine.df_biz[['business_id', 'categories', 'price_level', 'is_romantic', 'is_good_for_kid
28         s']],
29         on='business_id', how='left'
30     )
31     liked = user_data[user_data['stars'] >= 4]
32
33     cats_liked = ' '.join(liked['categories'].fillna('').str.lower())
34     tipos = [
35         t for t, cats in MAPEO_CATEGORIAS.items()
36         if any(c.lower() in cats_liked for c in cats)
37     ]
38
39     budget_mean = liked['price_level'].mean() if not liked.empty else 2.0
40     budget_val = int(round(float(budget_mean))) if not np.isnan(float(budget_mean)) else 2
41     budget_val = max(1, min(4, budget_val))
42     budget = {1: 'bajo', 2: 'medio', 3: 'alto', 4: 'premium'}[budget_val]
43
44     likes_romantic = bool(liked['is_romantic'].eq(1).any()) if 'is_romantic' in liked.columns else
45     False
46     likes_kids = bool(liked['is_good_for_kids'].eq(1).any()) if 'is_good_for_kids' in liked.column
47     s else False
48
49     if likes_kids:
50         group = 'familia'
51     elif likes_romantic:
52         group = 'pareja'
53     else:
54         group = 'solo'
55
56     return {
57         'tiposTurismo': tipos or ['cultural'],
58         'presupuesto_bucket': budget,
59         'group_type': group,
60         'edad_range': '25-34',
61         'wants_tours': False,
62         'needs_hotel': False,
63         'pref_food': True,

```

```

61         'requiere_accesibilidad': False,
62         'pref_outdoor': False,
63     }
64
65
66 # -----
67 # Métricas de Ranking
68 # -----
69
70 def dcg_at_k(relevances, k):
71     """Discounted Cumulative Gain para los primeros k elementos."""
72     relevances = np.array(relevances[:k], dtype=float)
73     if len(relevances) == 0:
74         return 0.0
75     discounts = np.array([1.0 / log2(i + 2) for i in range(len(relevances))])
76     return float(np.sum(relevances * discounts))
77
78
79 def ndcg_at_k(relevances, k):
80     """Normalized DCG: DCG@k / IDCG@k (ranking ideal)."""
81     actual_dcg = dcg_at_k(relevances, k)
82     ideal_relevances = sorted(relevances, reverse=True)
83     ideal_dcg = dcg_at_k(ideal_relevances, k)
84     if ideal_dcg == 0:
85         return 0.0
86     return actual_dcg / ideal_dcg
87
88
89 def precision_at_k(recommended_ids, relevant_ids, k):
90     """Fracción de los top-K recomendados que son realmente relevantes."""
91     top_k = recommended_ids[:k]
92     if len(top_k) == 0:
93         return 0.0
94     hits = sum(1 for rid in top_k if rid in relevant_ids)
95     return hits / len(top_k)
96
97
98 def hit_rate_at_k(recommended_ids, relevant_ids, k):
99     """1 si al menos un favorito aparece en top-K, 0 si no."""
100     top_k = set(recommended_ids[:k])
101     return 1.0 if top_k & set(relevant_ids) else 0.0
102
103
104 # -----
105 # Evaluación de predicción punto a punto (RMSE / MAE)
106 # -----
107
108 def evaluar_predicciones(engine, context_model, sample_size=1000):
109     """Evalúa RMSE y MAE para cada componente del sistema."""
110     n_test = len(engine.test_data)
111     n_eval = min(sample_size, n_test)
112     test_sample = engine.test_data.sample(n_eval, random_state=42)
113
114     actuals, preds_cf, preds_rf, preds_hybrid = [], [], [], []
115     errores = 0
116
117     print(f"Evaluando {n_eval} de {n_test} interacciones del test set...")
118     print("  Pre-computando contextos de usuario...")
119     unique_users = test_sample['user_id'].unique()
120     user_contexts = {uid: _infer_user_context(uid, engine) for uid in unique_users}

```

```

121
122 total = len(test_sample)
123 for idx, (_, row) in enumerate(test_sample.iterrows()):
124     if idx % 100 == 0:
125         pct = idx / total * 100
126         sys.stdout.write(f"\r Progreso: {idx}/{total} ({pct:.0f}%)")
127         sys.stdout.flush()
128
129     try:
130         p_cf = predict_cf_pearson(row['user_id'], row['business_id'], engine)
131         user_ctx = user_contexts.get(row['user_id'])
132         p_rf = float(context_model.predict_with_context([row['business_id']], user_context=user
133 r_ctx)[0])
134         if np.isnan(p_cf) or np.isnan(p_rf):
135             errores += 1
136             continue
137         p_hybrid = (0.1 * p_cf) + (0.9 * p_rf)
138
139         actuals.append(row['stars'])
140         preds_cf.append(p_cf)
141         preds_rf.append(p_rf)
142         preds_hybrid.append(p_hybrid)
143     except Exception as e:
144         errores += 1
145         if errores <= 3:
146             print(f"\n [warn] Error en u={row['user_id']}, b={row['business_id']}: {e}")
147
148 print(f"\r Progreso: {total}/{total} (100%)      ")
149
150 actuals = np.array(actuals)
151 preds_cf = np.array(preds_cf)
152 preds_rf = np.array(preds_rf)
153 preds_hybrid = np.array(preds_hybrid)
154
155 media_global = engine.train_data['stars'].mean()
156 preds_baseline = np.full_like(actuals, media_global)
157
158 print(f"\nPredicciones exitosas: {len(actuals)}")
159 print(f"Predicciones fallidas: {errores}")
160
161 print("\n+-----+-----+-----+")
162 print("| Componente | RMSE | MAE |")
163 print("+-----+-----+-----+")
164
165 for nombre, preds in [
166     ("Baseline (media global)", preds_baseline),
167     ("CF (Pearson + KNN)", preds_cf),
168     ("RF contextual (con contexto)", preds_rf),
169     ("Hibrido v3 (a=0.1)", preds_hybrid),
170 ]:
171     rmse = sqrt(mean_squared_error(actuals, preds))
172     mae = mean_absolute_error(actuals, preds)
173     print(f"| {nombre:<30s} | {rmse:.4f} | {mae:.4f} |")
174
175 print("+-----+-----+-----+")
176
177 errors_abs = np.abs(actuals - preds_hybrid)
178 print("\nDistribucion de error absoluto (Hibrido v3):")
179 for threshold in [0.5, 1.0, 1.5, 2.0]:
180     pct = (errors_abs <= threshold).mean() * 100

```

```

180         print(f" |error| <= {threshold}: {pct:.1f}%")
181
182     return {
183         'rmse_hybrid': sqrt(mean_squared_error(actuals, preds_hybrid)),
184         'mae_hybrid': mean_absolute_error(actuals, preds_hybrid),
185     }
186
187
188 # -----
189 # Evaluación de Ranking (NDCG@K, Precision@K, Hit Rate@K)
190 # -----
191
192 def evaluar_ranking(engine, context_model, n_users=100, k=5, relevance_threshold=4):
193     """
194     Evalúa métricas de ranking sobre una muestra de usuarios del test set.
195
196     Un ítem es 'relevante' si el usuario le dio >= relevance_threshold estrellas.
197
198     Args:
199         engine: SmarturEngine inicializado
200         context_model: SmarturContextModel entrenado
201         n_users: número de usuarios a evaluar
202         k: top-K para las métricas
203         relevance_threshold: mínimo de estrellas para considerar un ítem relevante
204     """
205     print(f"\n=== Métricas de Ranking (K={k}) ===\n")
206
207     # Obtener usuarios con suficientes interacciones en test
208     user_counts = engine.test_data.groupby('user_id').size()
209     eligible_users = user_counts[user_counts >= 3].index.tolist()
210
211     if len(eligible_users) == 0:
212         print(" No hay usuarios con suficientes interacciones para evaluar ranking.")
213         return {}
214
215     eval_users = eligible_users[:min(n_users, len(eligible_users))]
216     print(f" Evaluando {len(eval_users)} usuarios con >= 3 interacciones en test...\n")
217
218     ndcg_scores = []
219     precision_scores = []
220     hit_scores = []
221
222     for idx, user_id in enumerate(eval_users):
223         if idx % 20 == 0:
224             sys.stdout.write(f"\r Progreso: {idx}/{len(eval_users)}")
225             sys.stdout.flush()
226
227         try:
228             # Obtener los ítems relevantes reales del usuario en test
229             user_test = engine.test_data[engine.test_data['user_id'] == user_id]
230             relevant_ids = set(
231                 user_test[user_test['stars'] >= relevance_threshold]['business_id'].tolist()
232             )
233
234             if len(relevant_ids) == 0:
235                 continue # Sin ítems relevantes, no contribuye a las métricas
236
237             # Generar recomendaciones con contexto inferido del historial del usuario
238             user_ctx = _infer_user_context(user_id, engine)
239             recs = recommend_hybrid(

```

```

240         user_id, engine, context_model, alpha=0.1, context=user_ctx, top_n=k * 2
241     )
242     rec_ids = [r['item_id'] for r in recs]
243
244     # Construir vector de relevancia para NDCG
245     relevances = []
246     for rid in rec_ids[:k]:
247         user_rating = user_test.loc[
248             user_test['business_id'] == rid, 'stars'
249         ]
250         if len(user_rating) > 0:
251             relevances.append(float(user_rating.values[0]))
252         else:
253             relevances.append(0.0)
254
255     # Calcular métricas
256     ndcg_scores.append(ndcg_at_k(relevances, k))
257     precision_scores.append(precision_at_k(rec_ids, relevant_ids, k))
258     hit_scores.append(hit_rate_at_k(rec_ids, relevant_ids, k * 2))
259
260 except Exception:
261     continue
262
263 print(f"\r Progreso: {len(eval_users)}/{len(eval_users)}    ")
264
265 results = {}
266 if ndcg_scores:
267     results['ndcg'] = float(np.mean(ndcg_scores))
268     results['precision'] = float(np.mean(precision_scores))
269     results['hit_rate'] = float(np.mean(hit_scores))
270     results['n_users_evaluated'] = len(ndcg_scores)
271
272     print(f"\n+-----+-----+")
273     print(f"| Métrica          | Valor |")
274     print(f"+-----+-----+")
275     print(f"| NDCG@{k:<22d} | {results['ndcg']:.4f} |")
276     print(f"| Precision@{k:<17d} | {results['precision']:.4f} |")
277     print(f"| Hit Rate@{k*2:<18d} | {results['hit_rate']:.4f} |")
278     print(f"+-----+-----+")
279     print(f"\n Usuarios evaluados: {len(ndcg_scores)}")
280 else:
281     print(" No se pudieron calcular métricas de ranking.")
282     results['n_users_evaluated'] = 0
283
284 return results
285
286
287 # -----
288 # Evaluación local con datos reales SMARTUR
289 # -----
290
291 def evaluar_ranking_local(
292     context_model,
293     lightfm_model=None,
294     content_model=None,
295     k: int = 5,
296     min_users: int = 20,
297     min_interactions: int = 3,
298 ) -> dict | None:
299     """

```

```

300     Evalúa NDCG@k, Precision@k y Hit Rate@k usando interacciones reales de SMARTUR.
301
302     Retorna None si no hay suficientes datos (señal para usar eval Yelp como fallback).
303     Solo cuenta cuando hay >= min_users usuarios con >= min_interactions cada uno.
304
305     Ventaja sobre evaluar_ranking(): los ítems recomendados (POIs de Altas Montañas)
306     y los ítems del test set son los mismos, por lo que NDCG refleja calidad real.
307     """
308     try:
309         from poi_repository import fetch_real_interactions, fetch_traveler_profile
310         from fusion import recommend_hybrid
311         import numpy as np
312     except ImportError as e:
313         import logging
314         logging.getLogger(__name__).warning(f"[eval_local] Import error: {e}")
315         return None
316
317     df = fetch_real_interactions(min_events=1)
318     if df is None or df.empty:
319         return None
320
321     # Keep only users with enough interactions for a meaningful 80/20 split
322     user_counts = df.groupby('user_id').size()
323     eligible = user_counts[user_counts >= min_interactions].index.tolist()
324
325     if len(eligible) < min_users:
326         import logging
327         logging.getLogger(__name__).info(
328             f"[eval_local] Solo {len(eligible)} usuarios elegibles (mínimo {min_users}) "
329             f"- usando eval Yelp como fallback."
330         )
331         return None
332
333     ndcg_scores, precision_scores, hit_scores, pref_scores = [], [], [], []
334
335     for uid in eligible:
336         user_df = df[df['user_id'] == uid].sort_values('implicit_score', ascending=False)
337         n_total = len(user_df)
338         split = max(1, int(n_total * 0.8))
339
340         # Train items (used to warm context if needed) and test items (ground truth)
341         test_items = set(user_df.iloc[split:]['item_id'].tolist())
342         if not test_items:
343             continue
344
345         # Relevant items: those the user liked (implicit_score >= 4.0)
346         relevant = set(
347             user_df[user_df['implicit_score'] >= 4.0]['item_id'].tolist()
348         ) & test_items
349         if not relevant:
350             # All test items count as relevant (weak signal scenario)
351             relevant = test_items
352
353         # Contexto real del usuario (tiposTurismo/presupuesto/grupo) – sin esto
354         # preference_match_rate no mide nada, porque el boost de preferencia
355         # necesita saber qué declaró el usuario para poder compararlo.
356         try:
357             user_context = fetch_traveler_profile(uid) or {}
358         except Exception:
359             user_context = {}

```

```

360
361     try:
362         recs = recommend_hybrid(
363             user_id=str(uid),
364             engine=None,          # will use local POIs only
365             context_model=context_model,
366             alpha=0.4,
367             context=user_context,
368             top_n=max(k * 2, 10),
369             lightfm_model=lightfm_model,
370             content_model=content_model,
371         )
372     except Exception:
373         continue
374
375     if not recs:
376         continue
377
378     rec_ids = [r['item_id'] for r in recs[:k]]
379     pref_scores.extend(r.get('pred_pref', 0.0) for r in recs[:k])
380
381     # NDCG@k
382     dcg = sum(
383         1.0 / np.log2(pos + 2)
384         for pos, rid in enumerate(rec_ids)
385         if rid in relevant
386     )
387     ideal_hits = min(len(relevant), k)
388     idcg = sum(1.0 / np.log2(pos + 2) for pos in range(ideal_hits))
389     ndcg_scores.append(dcg / idcg if idcg > 0 else 0.0)
390
391     # Precision@k
392     hits_at_k = sum(1 for rid in rec_ids if rid in relevant)
393     precision_scores.append(hits_at_k / k)
394
395     # Hit Rate@k (binary: did at least one relevant item appear?)
396     hit_scores.append(1.0 if hits_at_k > 0 else 0.0)
397
398     if not ndcg_scores:
399         return None
400
401     results = {
402         'ndcg': float(np.mean(ndcg_scores)),
403         'precision': float(np.mean(precision_scores)),
404         'hit_rate': float(np.mean(hit_scores)),
405         # Cuántos usuarios reales entraron a la evaluación – lo usa el relevo
406         # automático de criterio de selección (api._apply_ranking_based_selection)
407         # para decidir si el NDCG ya es confiable o hay que seguir con RMSE.
408         'n_users_evaluated': len(ndcg_scores),
409     }
410
411     if pref_scores:
412         # Qué % de las recomendaciones evaluadas realmente coinciden con lo
413         # que el usuario declaró (tiposTurismo/presupuesto/grupo) – mide
414         # alineación con preferencias, algo que NDCG/precision (basados en
415         # estrellas) no capturan directamente.
416         results['preference_match_rate'] = float(
417             np.mean([1.0 if p >= 0.5 else 0.0 for p in pref_scores])
418         )
419     return results

```

```

420
421 # -----
422 # Main
423 # -----
424
425 def evaluar_modelo(sample_size=1000):
426     print("=== Evaluacion SMARTUR v3 ===\n")
427
428     engine = SmarturEngine(data_source='mexico')
429     engine.prepare_pearson_matrix()
430
431     context_model = SmarturContextModel()
432     context_model.train(engine.train_data)
433
434     # Métricas de predicción (RMSE/MAE)
435     pred_results = evaluar_predicciones(engine, context_model, sample_size)
436
437     # Métricas de ranking (NDCG, Precision, Hit Rate)
438     rank_results = evaluar_ranking(engine, context_model, n_users=100, k=5)
439
440     print("\n=== Evaluacion SMARTUR v3 completada ===")
441     return {**pred_results, **rank_results}
442
443
444 if __name__ == "__main__":
445     evaluar_modelo()

```

```

===== MODELO/src/poi_repository.py =====
1 import logging
2 import os
3 import json
4 import pandas as pd
5 import psycpg2
6
7 _logger = logging.getLogger("smartur-api")
8
9
10 def _get_env(name, default=None):
11     value = os.getenv(name)
12     return value if value not in (None, '') else default
13
14
15 # Category/context mappings for tourist services and traveler profiles
16 _SERVICE_TYPE_CATEGORIES = {
17     'hotel': {
18         'categories_raw': 'hotel, accommodation, Hotels & Travel',
19         'categories_mapped': ['rural'],
20         'outdoor': False,
21     },
22     'restaurant': {
23         'categories_raw': 'restaurant, food, gastronomy, local food, Restaurants',
24         'categories_mapped': ['gastronomy'],
25         'outdoor': False,
26     },
27     'tour': {
28         'categories_raw': 'tour, hiking, nature, adventure, Tours',
29         'categories_mapped': ['nature'],
30         'outdoor': True,
31     },
32 }

```

```

33
34 _INTEREST_MAP = {
35     'naturaleza': 'naturaleza', 'nature': 'naturaleza',
36     'cultura': 'cultural', 'culture': 'cultural', 'cultural': 'cultural',
37     'gastronomía': 'gastronomico', 'gastronomia': 'gastronomico',
38     'gastronomy': 'gastronomico', 'gastronomico': 'gastronomico',
39     'aventura': 'aventura', 'adventure': 'aventura',
40     'rural': 'rural',
41     # Opciones de interés de la app móvil que antes NO se mapeaban y se caían
42     # en silencio (dejando tiposTurismo vacío para muchos usuarios reales):
43     'historia': 'cultural', 'history': 'cultural',
44     'arte': 'cultural', 'art': 'cultural',
45     'deportes': 'aventura', 'sports': 'aventura',
46     'fotografía': 'naturaleza', 'fotografia': 'naturaleza', 'photography': 'naturaleza',
47     'bienestar': 'naturaleza', 'wellness': 'naturaleza',
48     'nightlife': 'nocturno', 'vida nocturna': 'nocturno',
49 }
50
51 _ACTIVITY_BUDGET = {1: 'bajo', 2: 'bajo', 3: 'medio', 4: 'alto', 5: 'premium'}
52
53 # Buckets de presupuesto válidos (coinciden con BUDGET_MAP en context_encoder y
54 # _BUDGET_BUCKET_LEVEL en fusion). Se usan para validar el campo `budget` real.
55 _VALID_BUDGET_BUCKETS = {'bajo', 'medio', 'alto', 'premium'}
56
57 # Mapea el travel_type que guarda la app móvil (Mochilero/Familiar/Lujo/
58 # Aventura/Romántico/De negocios) al vocabulario de GROUP_TYPES que entiende
59 # preference_match_score. Antes 'Familiar' llegaba como 'familiar' (≠ 'familia')
60 # y 'Romántico' como 'romántico' (≠ 'pareja'), así que el bono de grupo
61 # (familia con niños / pareja romántica) NUNCA se activaba con datos reales.
62 # Los valores ya correctos (solo/pareja/familia/amigos, de datos semilla) pasan
63 # de largo por el fallback.
64 _TRAVEL_GROUP_MAP = {
65     'familiar': 'familia',
66     'romántico': 'pareja', 'romantico': 'pareja',
67     'mochilero': 'amigos',
68     'aventura': 'amigos',
69     'lujo': 'solo',
70     'de negocios': 'solo', 'negocios': 'solo',
71 }
72
73
74 def get_poi_connection():
75     return psycopg2.connect(
76         host=_get_env('POI_DB_HOST', 'localhost'),
77         port=int(_get_env('POI_DB_PORT', 5432)),
78         database=_get_env('POI_DB_NAME', 'smartur'),
79         user=_get_env('POI_DB_USER', 'postgres'),
80         password=_get_env('POI_DB_PASSWORD', os.getenv('DB_PASSWORD', '')),
81         options='-c client_encoding=UTF8',
82     )
83
84
85 def fetch_pois(active_only=True):
86     where = 'WHERE is_active = TRUE' if active_only else ''
87     query = f'''
88         SELECT id, name, categories_raw, categories_mapped,
89                price_level, is_accessible, outdoor, latitude, longitude,
90                description
91         FROM point_of_interest {where}
92     '''

```

```

93     with get_poi_connection() as conn:
94         with conn.cursor() as cur:
95             cur.execute(query)
96             cols = [desc[0] for desc in cur.description]
97             rows = cur.fetchall()
98     df = pd.DataFrame(rows, columns=cols)
99
100     if 'categories_raw' in df.columns:
101         df['categories_raw'] = df['categories_raw'].apply(
102             lambda v: v.encode('utf-8', errors='ignore').decode('utf-8')
103             if isinstance(v, str)
104             else (v if isinstance(v, str) else '')
105         )
106
107     if 'categories_mapped' in df.columns:
108         def _normalize(val):
109             if isinstance(val, list):
110                 return val
111             if val is None:
112                 return []
113             if isinstance(val, str):
114                 try:
115                     parsed = json.loads(val)
116                     return parsed if isinstance(parsed, list) else []
117                 except (json.JSONDecodeError, TypeError):
118                     return []
119             return []
120         df['categories_mapped'] = df['categories_mapped'].apply(_normalize)
121     else:
122         df['categories_mapped'] = [[] for _ in range(len(df))]
123
124     df['is_accessible'] = df['is_accessible'].apply(lambda v: 1 if v else 0)
125     df['outdoor'] = df['outdoor'].apply(lambda v: 1 if v else 0)
126     df['price_level'] = df['price_level'].fillna(2).astype(int)
127
128     return df
129
130
131 def fetch_tourist_services(active_only=True):
132     """Tourist services as a POI-compatible DataFrame with prefixed IDs (svc_N)."""
133     where = 'WHERE ts.active = TRUE' if active_only else ''
134     query = f'''
135         SELECT ts.id_service AS id, ts.name, ts.service_type,
136                ts.description, l.latitude, l.longitude
137         FROM tourist_service ts
138         LEFT JOIN location l ON ts.id_location = l.id_location
139         {where}
140     '''
141     with get_poi_connection() as conn:
142         with conn.cursor() as cur:
143             cur.execute(query)
144             cols = [desc[0] for desc in cur.description]
145             rows = cur.fetchall()
146     df = pd.DataFrame(rows, columns=cols)
147     if df.empty:
148         return df
149
150     df['id'] = 'svc_' + df['id'].astype(str)
151     df['categories_raw'] = df['service_type'].map(
152         {k: v['categories_raw'] for k, v in _SERVICE_TYPE_CATEGORIES.items()})

```

```

153     ).fillna('service')
154     df['categories_mapped'] = df['service_type'].map(
155         {k: v['categories_mapped'] for k, v in _SERVICE_TYPE_CATEGORIES.items()}
156     ).apply(lambda x: x if isinstance(x, list) else [])
157     df['outdoor'] = df['service_type'].map(
158         {k: 1 if v['outdoor'] else 0 for k, v in _SERVICE_TYPE_CATEGORIES.items()}
159     ).fillna(0).astype(int)
160     df['price_level'] = 2
161     df['is_accessible'] = 0
162     df['latitude'] = df['latitude'].fillna(0).astype(float)
163     df['longitude'] = df['longitude'].fillna(0).astype(float)
164     df['kind'] = 'svc'
165     return df
166
167
168 def fetch_all_items(active_only=True):
169     """Unified candidate pool: POIs + tourist services."""
170     pois = fetch_pois(active_only)
171     pois['kind'] = 'poi'
172     try:
173         services = fetch_tourist_services(active_only)
174     except Exception:
175         services = pd.DataFrame()
176     if services.empty:
177         return pois
178     return pd.concat([pois, services], ignore_index=True)
179
180
181 def fetch_real_interactions(min_events: int = 1) -> pd.DataFrame:
182     """
183     Builds a user-item implicit rating matrix from live SMARTUR interaction data.
184
185     Signal weighting:
186     explicit star rating (user_rating) -> raw value 1-5 (dominates when present)
187     favorite (user_favorite) -> +4.0
188     visit (user_visit) -> +2.5
189     detail_open (user_interaction) -> +0.4 per open, capped at 5
190     dwell (user_interaction) -> +0.03 per second up to 120 s
191     skip (user_interaction) -> -0.5 per skip, capped at 3
192
193     Returns DataFrame[user_id, item_id, implicit_score] clipped to [1, 5].
194     Returns empty DataFrame on failure so callers can fall back gracefully.
195     """
196     import numpy as np
197
198     sql = """
199     SELECT
200         combined.user_id::text,
201         combined.place_kind || '_' || combined.place_id::text AS item_id,
202         MAX(combined.explicit_rating) AS explicit_rating,
203         SUM(CASE WHEN combined.signal = 'favorite' THEN 1 ELSE 0 END) AS favorites,
204         SUM(CASE WHEN combined.signal = 'visit' THEN 1 ELSE 0 END) AS visits,
205         SUM(CASE WHEN combined.signal = 'detail_open' THEN 1 ELSE 0 END) AS opens,
206         MAX(CASE WHEN combined.signal = 'dwell' THEN combined.dwell_ms ELSE 0 END) AS max_dwell_ms,
207         SUM(CASE WHEN combined.signal = 'skip' THEN 1 ELSE 0 END) AS skips
208     FROM (
209         SELECT user_id, place_kind, place_id,
210             'favorite' AS signal,
211             NULL::smallint AS explicit_rating,

```

```

212         NULL::int      AS dwell_ms
213     FROM user_favorite
214     WHERE is_active = TRUE
215         AND place_kind IN ('svc', 'poi')
216
217     UNION ALL
218
219     SELECT user_id, place_kind, place_id,
220           'visit', NULL, NULL
221     FROM user_visit
222     WHERE place_kind IN ('svc', 'poi')
223
224     UNION ALL
225
226     SELECT user_id, place_kind, place_id,
227           event_type, NULL, dwell_ms
228     FROM user_interaction
229     WHERE event_type IN ('detail_open', 'dwell', 'skip')
230         AND place_kind IN ('svc', 'poi')
231
232     UNION ALL
233
234     SELECT user_id, place_kind, place_id,
235           'rating', rating, NULL
236     FROM user_rating
237 ) combined
238 GROUP BY combined.user_id, combined.place_kind, combined.place_id
239 HAVING COUNT(*) >= %s
240 """
241 try:
242     with get_poi_connection() as conn:
243         with conn.cursor() as cur:
244             cur.execute(sql, (min_events,))
245             cols = [d[0] for d in cur.description]
246             rows = cur.fetchall()
247 except Exception as exc:
248     _logger.warning(f"fetch_real_interactions: DB query failed - {exc}")
249     return pd.DataFrame(columns=["user_id", "item_id", "implicit_score"])
250
251 if not rows:
252     return pd.DataFrame(columns=["user_id", "item_id", "implicit_score"])
253
254 df = pd.DataFrame(rows, columns=cols)
255
256 explicit = df["explicit_rating"].fillna(0).astype(float)
257 favorites = df["favorites"].fillna(0).astype(float).clip(0, 1)
258 visits = df["visits"].fillna(0).astype(float).clip(0, 1)
259 opens = df["opens"].fillna(0).astype(float).clip(0, 5)
260 dwell_sec = (df["max_dwell_ms"].fillna(0).astype(float) / 1000).clip(0, 120)
261 skips = df["skips"].fillna(0).astype(float).clip(0, 3)
262
263 implicit_base = (
264     favorites * 4.0
265     + visits * 2.5
266     + opens * 0.4
267     + dwell_sec * 0.03
268     - skips * 0.5
269 ).clip(1, 5)
270
271 has_explicit = explicit > 0

```

```

272 df["implicit_score"] = np.where(has_explicit, explicit, implicit_base).clip(1, 5)
273
274 return df[["user_id", "item_id", "implicit_score"]]
275
276
277 def fetch_evaluation_scores() -> pd.DataFrame:
278     """
279     Converts admin quality scores (0-100) from service_evaluation into implicit
280     ratings (1-5) that can be merged into the RF/GBM training corpus.
281     This gives the models a signal that well-evaluated tourist services should
282     be ranked higher, even for cold-start users.
283
284     Mapping: total_score >= 80 -> 4.5 | total_score >= 60 -> 3.5 | else -> 2.5
285
286     Returns DataFrame[user_id, business_id, stars]. Returns empty on error.
287     """
288     sql = """
289         SELECT
290             evaluator_id::text          AS user_id,
291             'svc_' || id_service::text  AS business_id,
292             CASE
293                 WHEN total_score >= 80 THEN 4.5
294                 WHEN total_score >= 60 THEN 3.5
295                 ELSE 2.5
296             END::float                  AS stars
297         FROM service_evaluation
298         WHERE is_active = TRUE
299             AND total_score IS NOT NULL
300             AND evaluator_id IS NOT NULL
301     """
302     try:
303         with get_poi_connection() as conn:
304             with conn.cursor() as cur:
305                 cur.execute(sql)
306                 cols = [d[0] for d in cur.description]
307                 rows = cur.fetchall()
308     except Exception as exc:
309         _logger.warning(f"fetch_evaluation_scores: DB query failed – {exc}")
310         return pd.DataFrame(columns=["user_id", "business_id", "stars"])
311
312     if not rows:
313         return pd.DataFrame(columns=["user_id", "business_id", "stars"])
314
315     return pd.DataFrame(rows, columns=cols)
316
317
318 def fetch_traveler_profile(user_id):
319     """
320     Reads traveler_profile for a numeric user_id and returns a context dict
321     compatible with ContextEncoder. Returns None if not found or user_id is non-numeric.
322     """
323     try:
324         uid = int(user_id)
325     except (ValueError, TypeError):
326         return None
327
328     # preferred_place/sustainable_preferences ya se le preguntan al usuario en
329     # el formulario de perfil y se guardan en la tabla, pero preference_match_score
330     # nunca las leía – el motor ignoraba dos señales que el usuario ya dio.
331     query = ''

```



```

392         FROM service_evaluation
393         WHERE is_active = TRUE AND total_score IS NOT NULL
394         GROUP BY id_service
395     """
396     rows = cur.fetchall()
397     cur.close()
398     return {row[0]: float(row[1]) for row in rows}
399 except Exception as exc:
400     import logging
401     logging.getLogger(__name__).warning(f"[fetch_quality_scores] error: {exc}")
402     return {}
403 finally:
404     if conn:
405         conn.close()

===== MODELO/src/synthetic_training.py =====
1 """
2 Modo de entrenamiento SINTÉTICO (opt-in, reversible) – NO se activa nunca por
3 defecto.
4
5 Propósito
6 -----
7 El dataset base de producción (data_reviews_mexico.csv) asigna un user_id nuevo
8 a cada reseña simulada: el 100% de los "usuarios" tiene 1 sola interacción, así
9 que NO existe estructura usuario-ítem que CF/RF puedan aprender (por eso hoy
10 item_mean le gana a todos). Este módulo permite, SOLO cuando se activa a mano,
11 reemplazar los datos de entrenamiento por un conjunto sintético a gran escala en
12 el que SÍ existe una estructura latente aprendible (personas con rasgos fijos
13 que califican coherentemente varios ítems reales del catálogo).
14
15 Con eso, el pipeline completo (train/test + validación cruzada + API + dashboard
16 + persistencia) puede DEMOSTRAR que los algoritmos de ML capturan estructura y
17 superan al baseline cuando la estructura existe – que es lo que pide la rúbrica
18 académica. Cuando lleguen usuarios reales, se apaga y el sistema entrena con
19 datos reales sin ningún residuo.
20
21 Reutiliza los generadores ya validados de synthetic_persona_validation.py
22 (generate_personas / generate_ratings) para no duplicar la lógica de la
23 estructura latente.
24
25 Control (variables de entorno)
26 -----
27 SMARTUR_SYNTH_TRAINING = 1|true|yes|on -> activa el modo (default: OFF)
28 SMARTUR_SYNTH_PERSONAS = <int> -> nº de personas (default: 2500)
29
30 INTEGRIDAD – importantísimo
31 -----
32 Cuando este modo está activo, quien llame DEBE marcar las métricas resultantes
33 con `synthetic_augmented=True` para que el dashboard avise que ese desempeño es
34 sobre una verdad inventada, no sobre turistas reales. Este módulo por sí solo no
35 escribe métricas; solo prepara los datos.
36 """
37 import logging
38 import os
39
40 import pandas as pd
41 from sklearn.model_selection import train_test_split
42
43 from synthetic_persona_validation import (
44     RANDOM_STATE,

```

```

45     generate_personas,
46     generate_ratings,
47 )
48
49 logger = logging.getLogger(__name__)
50
51 _DIR = os.path.dirname(os.path.abspath(__file__))
52 _DATA = os.path.join(_DIR, '..', 'data')
53 # El respaldo se escribe en el volumen de modelos (escribible por el usuario
54 # no-root del contenedor); /app/data es de solo lectura para modeluser.
55 _MODELS = os.path.join(_DIR, '..', 'models')
56
57 # Escala por defecto: bastante mayor que el experimento de validación (800) para
58 # densificar la matriz usuario-ítem sobre el catálogo real y darle al CF vecinos
59 # con solapamiento suficiente. Ajustable con SMARTUR_SYNTH_PERSONAS.
60 _DEFAULT_N_PERSONAS = 2500
61
62 _TRUTHY = {'1', 'true', 'yes', 'on', 'y', 't'}
63
64 # Nombre del respaldo del dataset sintético generado (reproducible/inspeccionable).
65 SYNTH_BACKUP_CSV = os.path.join(_MODELS, 'synthetic_training_backup.csv')
66
67
68 def synth_training_enabled() -> bool:
69     """True si SMARTUR_SYNTH_TRAINING está en un valor afirmativo. Default OFF."""
70     return os.environ.get('SMARTUR_SYNTH_TRAINING', '').strip().lower() in _TRUTHY
71
72
73 def synth_n_personas() -> int:
74     """Nº de personas sintéticas a generar (SMARTUR_SYNTH_PERSONAS o default)."""
75     raw = os.environ.get('SMARTUR_SYNTH_PERSONAS', '').strip()
76     if raw:
77         try:
78             n = int(raw)
79             if n > 0:
80                 return n
81         except ValueError:
82             logger.warning(f"[synth] SMARTUR_SYNTH_PERSONAS inválido ({raw!r}) – usando default {_DEFAULT_N_PERSONAS}.")
83     return _DEFAULT_N_PERSONAS
84
85
86 def build_synthetic_ratings(biz_df: pd.DataFrame, n_personas: int | None = None,
87                             seed: int = RANDOM_STATE, save_backup: bool = True) -> pd.DataFrame:
88     """
89     Genera el dataset sintético completo (user_id, business_id, stars) contra el
90     catálogo REAL de negocios (solo lectura). Cada persona tiene rasgos latentes
91     fijos y califica varios ítems coherentemente – la estructura que CF/RF deben
92     poder recuperar.
93
94     Guarda un CSV de respaldo por defecto para que la corrida sea reproducible e
95     inspeccionable.
96     """
97     if n_personas is None:
98         n_personas = synth_n_personas()
99
100     personas = generate_personas(n_personas, seed=seed)
101     ratings = generate_ratings(personas, biz_df, seed=seed)
102
103     n_users = ratings['user_id'].nunique()

```

```

104     n_items = ratings['business_id'].nunique()
105     pct_repeat = (ratings.groupby('user_id').size() >= 2).mean() * 100 if len(ratings) else 0
106     logger.warning(
107         f"[synth] Dataset sintético: {len(ratings)} ratings, {n_users} personas, "
108         f"{n_items} ítems únicos, {pct_repeat:.0f}% de personas con >=2 ratings "
109         f"(estructura latente aprendible – NO son usuarios reales).")
110     )
111
112     if save_backup:
113         try:
114             ratings.to_csv(SYNTH_BACKUP_CSV, index=False)
115             logger.info(f"[synth] Respaldo del dataset sintético guardado en {SYNTH_BACKUP_CSV}")
116         except Exception as exc:
117             logger.warning(f"[synth] No se pudo guardar el respaldo sintético: {exc}")
118
119     return ratings
120
121
122 def build_synthetic_split(biz_df: pd.DataFrame, test_size: float = 0.2,
123                          n_personas: int | None = None,
124                          seed: int = RANDOM_STATE) -> tuple[pd.DataFrame, pd.DataFrame]:
125     """
126     Igual que build_synthetic_ratings pero devuelve (train_df, test_df) ya
127     particionados 80/20 – listos para asignarse a engine.train_data /
128     engine.test_data y que TODO el pipeline (métricas train/test, validación
129     cruzada, RF/CF/GBM/LightFM) opere sobre estructura aprendible.
130     """
131     ratings = build_synthetic_ratings(biz_df, n_personas=n_personas, seed=seed)
132     train_df, test_df = train_test_split(ratings, test_size=test_size, random_state=seed)
133     return train_df.reset_index(drop=True), test_df.reset_index(drop=True)

```

===== MODELO/src/synthetic_persona_validation.py =====

```

1 """
2 Experimento AISLADO de validación de pipeline – NO es parte del entrenamiento
3 de producción ni se conecta al dashboard.
4
5 Pregunta que responde: si le damos a CF/RF datos donde SÍ existe estructura
6 usuario-ítem real (a diferencia del dataset base de producción, donde el
7 100% de los usuarios tiene 1 sola interacción y no hay nada que aprender),
8 ¿el pipeline es capaz de recuperarla?
9
10 Cómo se genera la estructura sintética – a propósito MUY distinta del
11 generador de datos actual (seed_pois_mexico.py / pre_procesamiento_mexico.py):
12 - Cada "persona" sintética tiene rasgos latentes FIJOS y se reutiliza en
13 varios ítems (afinidad por tipo de turismo + presupuesto preferido).
14 - El generador actual en cambio crea un usuario nuevo por cada reseña y
15 la calificación solo depende del promedio del negocio, nunca de quién
16 califica – por eso el CF no puede aprender nada de ese dataset (ver
17 memoria de la sesión: 248,259 usuarios, 100% con 1 sola interacción).
18 - Aquí la calificación SÍ depende de qué tan bien encaja el ítem con los
19 rasgos de la persona + ruido – la estructura que un CF/RF debería poder
20 recuperar si el pipeline funciona.
21
22 IMPORTANTE – por qué esto NUNCA debe tratarse como una mejora real:
23 Es una verdad sintética que nosotros inventamos. Si CF/RF le "ganan" a
24 item_mean aquí, solo demuestra que el pipeline puede aprender un patrón
25 inventado por nosotros – no dice nada sobre el comportamiento real de un
26 turista en Veracruz. Estos resultados NUNCA se escriben en
27 algorithm_metrics.json, cv_metrics.json ni ningún archivo que el dashboard
28 lea – solo se imprimen/guardan en synthetic_validation_report.json como

```

```

29 diagnóstico técnico.
30
31 Salvaguarda: SmarturContextModel.train() escribe rf_context_yelp.joblib al
32 directorio de modelos de producción sin importar los datos de entrada – acá
33 se redirige ese directorio a una carpeta temporal mientras corre el
34 experimento, así ni por error se pisa el modelo real.
35
36 Uso (manual, nunca automático, nunca dentro del contenedor de producción):
37     python synthetic_persona_validation.py
38 """
39 import json
40 import logging
41 import shutil
42 import tempfile
43 import time
44 from contextlib import contextmanager
45 from math import sqrt
46
47 import numpy as np
48 import pandas as pd
49 from sklearn.metrics import mean_absolute_error, mean_squared_error
50
51 import rf_model as _rf_model_module
52 from context_encoder import MAPEO_CATEGORIAS, TOURISM_TYPES
53 from engine import SmarturEngine
54 from cf import predict_cf_pearson
55
56 logging.basicConfig(level=logging.INFO)
57 logger = logging.getLogger(__name__)
58
59 N_PERSONAS = 800
60 RATINGS_PER_PERSONA = (3, 12) # rango de cuántos lugares califica cada persona
61 RANDOM_STATE = 42
62
63
64 @contextmanager
65 def _sandboxed_model_dir():
66     """Redirige rf_model._MODELS a una carpeta temporal mientras dura el
67     bloque – así SmarturContextModel.train() no puede escribir sobre el
68     rf_context_yelp.joblib real aunque este script se corra por accidente
69     en el servidor. Se restaura y se borra el temporal al salir."""
70     original = _rf_model_module._MODELS
71     tmp_dir = tempfile.mkdtemp(prefix='smartur_synth_experiment_')
72     _rf_model_module._MODELS = tmp_dir
73     try:
74         yield tmp_dir
75     finally:
76         _rf_model_module._MODELS = original
77         shutil.rmtree(tmp_dir, ignore_errors=True)
78
79
80 def _tipos_de_categorias(categorias: str) -> list[str]:
81     """Dado el string de categorías de un ítem, qué tipos de turismo matchea
82     (mismo criterio que preference_match_score en fusion.py)."""
83     cats_lower = (categorias or '').lower()
84     tipos = []
85     for tipo, keywords in MAPEO_CATEGORIAS.items():
86         if any(k.lower() in cats_lower for k in keywords):
87             tipos.append(tipo)
88     return tipos

```

```

89
90
91 def generate_personas(n: int = N_PERSONAS, seed: int = RANDOM_STATE) -> pd.DataFrame:
92     """Cada persona tiene una afinidad [0,1] por cada tipo de turismo
93     (Dirichlet – suman 1, como un perfil de intereses real) y un presupuesto
94     preferido 1-4. Rasgos FIJOS por persona, reutilizados en cada rating que
95     genere – a diferencia del generador de producción, que no tiene ninguna
96     identidad de usuario persistente."""
97     rng = np.random.default_rng(seed)
98     rows = []
99     for i in range(n):
100         afinidades = rng.dirichlet(np.ones(len(TOURISM_TYPES)))
101         row = {'persona_id': f'synth_persona_{i}', 'budget': int(rng.integers(1, 5))}
102         for j, t in enumerate(TOURISM_TYPES):
103             row[f'aff_{t}'] = float(afinidades[j])
104         rows.append(row)
105     return pd.DataFrame(rows)
106
107
108 def generate_ratings(personas: pd.DataFrame, biz_df: pd.DataFrame, seed: int = RANDOM_STATE) -> pd
109     .DataFrame:
110     """Cada persona califica varios ítems reales del catálogo; el rating
111     depende de qué tan bien encaja el ítem con sus rasgos latentes + ruido
112     gaussiano – la estructura aprendible que le falta al dataset actual."""
113     rng = np.random.default_rng(seed)
114     biz = biz_df.copy()
115     biz['_tipos'] = biz['categories'].apply(_tipos_de_categorias)
116     biz = biz[biz['_tipos'].map(len) > 0].reset_index(drop=True)
117     if biz.empty:
118         raise ValueError('Ningún ítem del catálogo tiene categorías mapeables a tipos de turismo.')
119
120     rows = []
121     for _, persona in personas.iterrows():
122         # Afinidad máxima de la persona (Dirichlet suma 1, así que el favorito
123         # ronda 0.3-0.6). Se normaliza cada afinidad contra este máximo para
124         # que un match con el tipo FAVORITO de la persona dé ~1.0 y produzca
125         # calificaciones de 4-5*. Sin esto las afinidades crudas son chicas y
126         # casi ningún rating llega al umbral de relevancia (≥4*), dejando las
127         # métricas de clasificación (Accuracy/Precision/Recall/F1) degeneradas.
128         max_aff = max((persona[f'aff_{t}'] for t in TOURISM_TYPES), default=0.0) or 1.0
129         n_ratings = int(rng.integers(RATINGS_PER_PERSONA[0], RATINGS_PER_PERSONA[1] + 1))
130         n_ratings = min(n_ratings, len(biz))
131         sample_idx = rng.choice(len(biz), size=n_ratings, replace=False)
132         for idx in sample_idx:
133             item = biz.iloc[idx]
134             afinidad_raw = max((persona[f'aff_{t}'] for t in item['_tipos']), default=0.0)
135             afinidad = afinidad_raw / max_aff # 0..1 relativo al favorito de la persona
136             price = item.get('price_level', 2) or 2
137             try:
138                 budget_fit = 1.0 - abs(float(persona['budget']) - float(price)) / 3.0
139             except (TypeError, ValueError):
140                 budget_fit = 0.5
141             true_score = 1.0 + 4.0 * (0.7 * afinidad + 0.3 * max(0.0, budget_fit))
142             noisy = true_score + rng.normal(0, 0.5)
143             stars = int(np.clip(round(noisy), 1, 5))
144             rows.append({
145                 'user_id': persona['persona_id'],
146                 'business_id': item['business_id'],
147                 'stars': stars,

```

```

147         })
148     return pd.DataFrame(rows)
149
150
151 def _rmse_mae(actuals, preds) -> dict:
152     return {
153         'rmse': float(sqrt(mean_squared_error(actuals, preds))),
154         'mae': float(mean_absolute_error(actuals, preds)),
155     }
156
157
158 def _persona_declared_context(persona: pd.Series) -> dict:
159     """
160     Simula lo que la persona 'declararía' en su perfil real: su tipo de
161     turismo favorito (mayor afinidad) y su bucket de presupuesto – igual
162     que fetch_traveler_profile en producción. RF/predict_with_context solo
163     puede usar el CONTEXTO DECLARADO al predecir (no la identidad de
164     entrenamiento – ver rf_model.py:predict_with_context), así que probarlo
165     con user_context=None sería injusto: nunca podría distinguir personas.
166     """
167     afinidades = {t: persona[f'aff_{t}']} for t in TOURISM_TYPES}
168     top_tipo = max(afinidades, key=afinidades.get)
169     budget_map = {1: 'bajo', 2: 'medio', 3: 'alto', 4: 'premium'}
170     return {
171         'tiposTurismo': [top_tipo],
172         'presupuesto_bucket': budget_map.get(int(persona['budget']), 'medio'),
173     }
174
175
176 def _make_fold_engine(train_df: pd.DataFrame, df_biz: pd.DataFrame) -> SmarturEngine:
177     fold_engine = object.__new__(SmarturEngine)
178     fold_engine.train_data = train_df
179     fold_engine.df_biz = df_biz
180     fold_engine._user_idx_map = None
181     fold_engine._biz_idx_map = None
182     fold_engine.prepare_pearson_matrix()
183     return fold_engine
184
185
186 def run_validation(n_personas: int = N_PERSONAS, test_size: float = 0.2) -> dict:
187     from sklearn.model_selection import train_test_split
188
189     logger.info('[synth] Cargando catálogo real de negocios (solo lectura)...')
190     base_engine = SmarturEngine(data_source='mexico')
191     biz_df = base_engine.df_biz
192
193     logger.info(f'[synth] Generando {n_personas} personas sintéticas con rasgos latentes...')
194     personas = generate_personas(n_personas)
195     ratings = generate_ratings(personas, biz_df)
196     logger.info(
197         f'[synth] {len(ratings)} ratings generados – '
198         f'{ratings["user_id"].nunique()} personas, {ratings["business_id"].nunique()} ítems únicos
199         , '
200         f'f'{{(ratings.groupby("user_id").size() >= 2).mean() * 100:.0f}}% de personas con >=2 ratings
201     )
202     train_df, test_df = train_test_split(ratings, test_size=test_size, random_state=RANDOM_STATE)
203
204     actuals = test_df['stars'].values

```

```

205 global_mean = float(train_df['stars'].mean())
206 baseline_preds = np.full(len(test_df), global_mean)
207
208 item_means = train_df.groupby('business_id')['stars'].mean()
209 item_mean_preds = test_df['business_id'].map(item_means).fillna(global_mean).values
210
211 logger.info('[synth] Evaluando CF Pearson KNN...')
212 t0 = time.time()
213 fold_engine = _make_fold_engine(train_df, biz_df)
214 cf_preds = [
215     predict_cf_pearson(row.user_id, row.business_id, fold_engine)
216     for row in test_df.itertuples()
217 ]
218 cf_time_ms = (time.time() - t0) * 1000
219
220 logger.info('[synth] Evaluando Random Forest (modelos en carpeta temporal, no toca producción)
...')
221 t0 = time.time()
222 persona_ctx = {
223     p['persona_id']: _persona_declared_context(p)
224     for _, p in personas.iterrows()
225 }
226 with _sandboxed_model_dir():
227     rf = _rf_model_module.SmarturContextModel()
228     rf.train(train_df, dynamic_override=True)
229     rf_preds = [
230         float(rf.predict_with_context(
231             [row.business_id], user_context=persona_ctx.get(row.user_id))[0])
232         for row in test_df.itertuples()
233     ]
234 rf_time_ms = (time.time() - t0) * 1000
235
236 result = {
237     'n_personas': n_personas,
238     'n_ratings': len(ratings),
239     'n_train': len(train_df),
240     'n_test': len(test_df),
241     'algorithms': {
242         'baseline': _rmse_mae(actuals, baseline_preds),
243         'item_mean': _rmse_mae(actuals, item_mean_preds),
244         'cf_knn_pearson': {*_rmse_mae(actuals, cf_preds), 'time_ms': cf_time_ms},
245         'random_forest': {*_rmse_mae(actuals, rf_preds), 'time_ms': rf_time_ms},
246     },
247     'note': (
248         'DIAGNÓSTICO DE PIPELINE, NO es una métrica de producción. '
249         'La "verdad" aquí es un patrón sintético inventado – que CF/RF le '
250         'ganen a item_mean solo demuestra que el pipeline SÍ puede aprender '
251         'estructura usuario-item cuando existe. No implica nada sobre el '
252         'comportamiento real de turistas en Veracruz.'
253     ),
254 }
255 return result
256
257
258 if __name__ == '__main__':
259     report = run_validation()
260     print(json.dumps(report, indent=2, ensure_ascii=False))
261     best = min(report['algorithms'], key=lambda k: report['algorithms'][k]['rmse'])
262     print(f"\n[synth] Mejor algoritmo sobre estructura inventada: {best} "
263           f"(RMSE={report['algorithms'][best]['rmse']:.4f})")

```

```

264     if best in ('cf_knn_pearson', 'random_forest'):
265         print('[synth] ✓ El pipeline SÍ puede aprender estructura usuario-ítem cuando existe.')
266     else:
267         print('[synth] x Ni con estructura inventada a propósito CF/RF superan a item_mean – revisa
        ar el pipeline.')
```

```

===== MODELO/src/route_optimizer.py =====
1 import math
2 import random
3 from typing import Any, Dict, List, Optional
4
5 # Average city travel speed used to convert km → travel minutes
6 _CITY_SPEED_KMH = 30.0
7 # Large penalty added to effective distance when a stop violates its time window
8 _TIME_VIOLATION_PENALTY = 500.0
9
10
11 def _haversine(lat1: float, lon1: float, lat2: float, lon2: float) -> float:
12     """Distance in km between two lat/lng points (Haversine formula)."""
13     R = 6371.0
14     phi1, phi2 = math.radians(lat1), math.radians(lat2)
15     dphi = math.radians(lat2 - lat1)
16     dlam = math.radians(lon2 - lon1)
17     a = math.sin(dphi / 2) ** 2 + math.cos(phi1) * math.cos(phi2) * math.sin(dlam / 2) ** 2
18     return R * 2 * math.atan2(math.sqrt(a), math.sqrt(1 - a))
19
20
21 def _path_distance(order: List[int], dist_matrix: List[List[float]]) -> float:
22     return sum(dist_matrix[order[i]][order[i + 1]] for i in range(len(order) - 1))
23
24
25 def _travel_minutes(dist_km: float) -> float:
26     """Convert distance in km to travel time in minutes at city speed."""
27     return (dist_km / _CITY_SPEED_KMH) * 60.0
28
29
30 def _effective_distance(
31     order: List[int],
32     dist_matrix: List[List[float]],
33     durations: List[int],
34     open_min: List[Optional[int]],
35     close_min: List[Optional[int]],
36     start_minutes: int,
37 ) -> float:
38     """
39     Compute effective path cost adding time-window violation penalties.
40     Simulates accumulated clock time (travel + dwell) along the path.
41     """
42     total = 0.0
43     clock = float(start_minutes)
44
45     for step in range(len(order)):
46         idx = order[step]
47         if step > 0:
48             seg_km = dist_matrix[order[step - 1]][idx]
49             total += seg_km
50             clock += _travel_minutes(seg_km)
51
52     # Check operating hours
53     if open_min[idx] is not None and clock < open_min[idx]:
```

```

54         # Must wait for opening – waiting is a soft penalty, not a hard skip
55         total += _TIME_VIOLATION_PENALTY * ((open_min[idx] - clock) / 60.0)
56         clock = float(open_min[idx])
57         if close_min[idx] is not None and clock > close_min[idx]:
58             # Already closed – severe penalty
59             total += _TIME_VIOLATION_PENALTY
60
61         clock += durations[idx]
62
63     return total
64
65
66 def optimize_route(
67     stops: List[Dict[str, Any]],
68     *,
69     n_ants: int = 30,
70     n_iterations: int = 80,
71     alpha: float = 1.2,
72     beta: float = 2.5,
73     rho: float = 0.5,
74     q: float = 100.0,
75     seed: int = 42,
76     start_minutes: int = 540, # 09:00 default
77 ) -> Dict[str, Any]:
78     """
79     Ant Colony Optimization for open-path route ordering with optional
80     time-window constraints and stop durations.
81
82     The first stop is treated as the fixed departure point (the ant always
83     starts there); the algorithm optimises the remaining visit order.
84
85     Args:
86         stops: list of dicts with:
87             - ``lat``, ``lng`` (floats, required)
88             - ``duration_minutes`` (int, optional – default 60)
89             - ``open_minutes`` (int, optional – minutes from midnight)
90             - ``close_minutes`` (int, optional – minutes from midnight)
91             start_minutes: departure time in minutes from midnight (default 540 = 09:00).
92
93     Returns:
94         {
95             "optimized_order": list[int],
96             "original_distance_km": float,
97             "optimized_distance_km": float,
98             "savings_pct": int,
99         }
100     """
101     n = len(stops)
102
103     trivial = {
104         "optimized_order": list(range(n)),
105         "original_distance_km": 0.0,
106         "optimized_distance_km": 0.0,
107         "savings_pct": 0,
108     }
109     if n <= 1:
110         return trivial
111
112     # — Per-stop time data —————
113     durations: List[int] = [int(s.get("duration_minutes") or 60) for s in stops]

```

```

114 open_min: List[Optional[int]] = [
115     int(s["open_minutes"]) if s.get("open_minutes") is not None else None
116     for s in stops
117 ]
118 close_min: List[Optional[int]] = [
119     int(s["close_minutes"]) if s.get("close_minutes") is not None else None
120     for s in stops
121 ]
122 has_time_windows = any(o is not None or c is not None for o, c in zip(open_min, close_min))
123
124 # — Distance matrix —————
125 dist: List[List[float]] = [[0.0] * n for _ in range(n)]
126 for i in range(n):
127     li, oi = stops[i].get("lat", 0.0), stops[i].get("lng", 0.0)
128     for j in range(i + 1, n):
129         lj, oj = stops[j].get("lat", 0.0), stops[j].get("lng", 0.0)
130         d = _haversine(li, oi, lj, oj)
131         dist[i][j] = d
132         dist[j][i] = d
133
134 original_order = list(range(n))
135 original_dist = _path_distance(original_order, dist)
136
137 # Cost function: pure distance or time-window-penalised distance
138 def path_cost(order: List[int]) -> float:
139     if has_time_windows:
140         return _effective_distance(order, dist, durations, open_min, close_min, start_minutes)
141     return _path_distance(order, dist)
142
143 original_cost = path_cost(original_order)
144
145 if n == 2:
146     return {
147         "optimized_order": original_order,
148         "original_distance_km": round(original_dist, 2),
149         "optimized_distance_km": round(original_dist, 2),
150         "savings_pct": 0,
151     }
152
153 # — ACO setup —————
154 tau_init = 1.0 / (n * max(original_cost, 0.01))
155 tau: List[List[float]] = [[tau_init] * n for _ in range(n)]
156
157 # Heuristic visibility: 1/dist (avoid division by zero)
158 eta: List[List[float]] = [[0.0] * n for _ in range(n)]
159 for i in range(n):
160     for j in range(n):
161         eta[i][j] = 1.0 / max(dist[i][j], 0.01)
162
163 best_order = original_order[:]
164 best_cost = original_cost
165 rng = random.Random(seed)
166
167 for _ in range(n_iterations):
168     iteration_paths: List[tuple] = []
169
170     for _ant in range(n_ants):
171         visited = [False] * n
172         path = [0] # fixed departure = first stop
173         visited[0] = True

```

```

174
175         for _step in range(n - 1):
176             cur = path[-1]
177             candidates = [j for j in range(n) if not visited[j]]
178
179             # Probability proportional to tau^alpha * eta^beta
180             weights = [
181                 (tau[cur][j] ** alpha) * (eta[cur][j] ** beta)
182                 for j in candidates
183             ]
184             total = sum(weights)
185
186             if total <= 0:
187                 nxt = candidates[0]
188             else:
189                 r = rng.random() * total
190                 cumulative = 0.0
191                 nxt = candidates[-1]
192                 for idx, w in enumerate(weights):
193                     cumulative += w
194                     if r <= cumulative:
195                         nxt = candidates[idx]
196                         break
197
198             path.append(nxt)
199             visited[nxt] = True
200
201             pc = path_cost(path)
202             pd = _path_distance(path, dist)
203             iteration_paths.append((path, pc, pd))
204             if pc < best_cost:
205                 best_cost = pc
206                 best_order = path[:]
207
208         # — Pheromone update —————
209         for i in range(n):
210             for j in range(n):
211                 tau[i][j] *= (1.0 - rho)
212                 if tau[i][j] < 1e-10:
213                     tau[i][j] = 1e-10
214
215         for path, pc, _ in iteration_paths:
216             deposit = q / max(pc, 0.01)
217             for k in range(len(path) - 1):
218                 tau[path[k]][path[k + 1]] += deposit
219                 tau[path[k + 1]][path[k]] += deposit
220
221         # — Result —————
222         best_dist = _path_distance(best_order, dist)
223
224         if best_cost >= original_cost:
225             best_order = original_order
226             best_dist = original_dist
227             savings_pct = 0
228         else:
229             savings_pct = max(0, round((1.0 - best_dist / original_dist) * 100))
230
231         return {
232             "optimized_order": best_order,
233             "original_distance_km": round(original_dist, 2),

```

```

234         "optimized_distance_km": round(best_dist, 2),
235         "savings_pct": savings_pct,
236     }

===== MODELO/src/api.py =====
1 import asyncio
2 import logging
3 import os
4 import threading
5 import time
6 from datetime import datetime, timezone
7 import pandas as pd
8 from concurrent.futures import ThreadPoolExecutor
9 from contextlib import asynccontextmanager
10 from typing import List, Dict, Any, Optional
11
12 from fastapi import FastAPI, HTTPException, Query, BackgroundTasks
13 from fastapi.middleware.cors import CORSMiddleware
14 from pydantic import BaseModel, Field
15
16 """
17 SMARTUR API Base File
18 Define los Endpoints de recomendación mediante FastAPI.
19 Conecta los flujos entre el Engine de Pearson y el Modelo Contextual de RF.
20 """
21
22 from engine import SmarturEngine
23 from rf_model import SmarturContextModel, _MODELS
24 from fusion import recommend_hybrid
25 from route_optimizer import optimize_route as _aco_optimize
26 import json
27 from pathlib import Path
28 from poi_repository import fetch_pois, get_poi_connection, fetch_traveler_profile
29 from restmex_repository import fetch_restmex_data
30
31 logging.basicConfig(level=logging.INFO)
32 logger = logging.getLogger("smartur-api")
33
34 engine = None
35 context_model = None
36 lightfm_model = None # LightFM: cold-start-aware matrix factorization (WARP loss)
37 content_model_cb = None # ContentModel: TF-IDF fallback for cold-start
38 quality_scores: dict = {} # {business_id: normalized quality ∈ [0,1]} from service_evaluation
39 # data_warmth ∈ [0,1]: qué tan "caliente" está el sistema en datos reales.
40 # Controla el blend dinámico en fusion.recommend_hybrid – con warmth=0 la
41 # preferencia declarada domina; con warmth=1 los modelos aprendidos toman el
42 # control. Se recalcula al arrancar y tras cada reentrenamiento.
43 _data_warmth: float = 0.0
44 # Umbral de "madurez": nº de usuarios reales con historial suficiente (>=3 items
45 # interactuados) para confiar al 100% en los modelos aprendidos. Se mide por
46 # USUARIOS con historial, no por interacciones crudas – 1000 eventos de 15
47 # usuarios no dan señal colaborativa; lo que CF/RF necesitan es DIVERSIDAD de
48 # usuarios con historial repetido. Constante nombrada y ajustable con el tiempo.
49 _WARMTH_TARGET_USERS = 100
50 _WARMTH_MIN_ITEMS_PER_USER = 3
51 _scheduler = None # APScheduler instance – module-level so endpoints can read/update it
52 _restmex_cache = None # tuple of (reviews_df, biz_df)
53 _training_in_progress = False # True mientras se entrena en background al arrancar
54 _models_lock = threading.RLock() # Guards global model references during swap
55

```

```

56 # Connection pool for /metrics - reuses DB connections instead of opening one per request
57 _db_pool = None
58 _db_pool_lock = threading.Lock()
59
60 # Default context for cold-start users with no profile in DB (Altas Montañas region, Veracruz)
61 _REGION_DEFAULT_CONTEXT = {
62     "presupuesto_bucket": "medio",
63     "edad_range": "25-34",
64     "tiposTurismo": ["cultural"],
65     "group_type": "pareja",
66     "wants_tours": False,
67     "needs_hotel": False,
68     "pref_food": True,
69     "requiere_accesibilidad": False,
70     "pref_outdoor": False,
71     "has_visited_region": False,
72 }
73
74
75 def _get_db_pool():
76     global _db_pool
77     if _db_pool is None:
78         with _db_pool_lock:
79             if _db_pool is None:
80                 import psycopg2.pool as _pg_pool
81                 _db_pool = _pg_pool.ThreadedConnectionPool(
82                     minconn=1,
83                     maxconn=5,
84                     host=os.environ.get("POI_DB_HOST", "localhost"),
85                     port=int(os.environ.get("POI_DB_PORT", 5432)),
86                     dbname=os.environ.get("POI_DB_NAME", "smartur"),
87                     user=os.environ.get("POI_DB_USER", "postgres"),
88                     password=os.environ.get("POI_DB_PASSWORD", os.environ.get("DB_PASSWORD", "")),
89                 )
90     return _db_pool
91
92 def _get_restmex_data():
93     global _restmex_cache
94     if _restmex_cache is None:
95         _restmex_cache = fetch_restmex_data()
96     return _restmex_cache
97
98 def _merge_restmex(train_data, peso=0.3):
99     reviews, biz = _get_restmex_data()
100     if reviews is not None and len(reviews) > 100:
101         tile_count = max(1, int(peso * len(train_data) / len(reviews)))
102         tiles = [reviews] * tile_count
103         train_data = pd.concat([train_data] + tiles, ignore_index=True)
104         logger.info(f"Rest-Mex: {len(reviews)} reseñas x{tile_count}")
105         return train_data, biz
106     return train_data, None
107
108
109 def _refresh_data_warmth() -> float:
110     """Recalcula data_warmth desde la DIVERSIDAD de usuarios reales con
111     historial suficiente (no desde interacciones crudas).
112     warmth = min(1, usuarios_con_>=N_items / WARMTH_TARGET_USERS)
113     Si falla la consulta, deja el warmth en 0.0 (lado seguro: la preferencia
114     declarada domina)."""
115     global _data_warmth

```

```

116     try:
117         from poi_repository import fetch_real_interactions
118         df = fetch_real_interactions(min_events=1)
119         if df is None or df.empty:
120             eligible = 0
121         else:
122             counts = df.groupby('user_id').size()
123             eligible = int((counts >= _WARMTH_MIN_ITEMS_PER_USER).sum())
124             data_warmth = min(1.0, eligible / _WARMTH_TARGET_USERS)
125             logger.info(
126                 f"[warmth] {eligible} usuarios con >={_WARMTH_MIN_ITEMS_PER_USER} items "
127                 f"(de {_WARMTH_TARGET_USERS} objetivo) -> data_warmth={_data_warmth:.3f}"
128             )
129     except Exception as exc:
130         logger.warning(f"[warmth] No se pudo calcular data_warmth: {exc}")
131     _data_warmth = 0.0
132     return _data_warmth
133
134
135 def _load_or_train_models(do_train: bool = False) -> None:
136     """
137     Carga los modelos desde disco. Si do_train=True y alguno falta, lo entrena.
138     Diseñado para ejecutarse en un ThreadPoolExecutor (CPU-bound).
139     """
140     global engine, context_model, lightfm_model, content_model_cb, quality_scores, _training_in_progress
141
142     try:
143         logger.info("[boot] Cargando Motor de Pearson + SVD (Engine)...")
144         engine = SmarturEngine(data_source='mexico')
145         engine.prepare_pearson_matrix()
146         # Seed data (seed_pois_mexico.py) ya incluye REST-MEX 2025/2022.
147         # No llamar _merge_restmex para evitar duplicar 208K reseñas con
148         # business_ids inconsistentes (idx%40 genera 40 IDs por Town+Type).
149         restmex_biz = None
150
151         logger.info("[boot] Cargando Modelo de Contexto (Random Forest)...")
152         context_model = SmarturContextModel()
153         if not context_model.load():
154             if do_train:
155                 logger.info("[boot] Modelo RF no encontrado – entrenando en segundo plano...")
156                 context_model.train(engine.train_data, df_biz_extra=restmex_biz)
157             else:
158                 context_model = None
159
160         logger.info("[boot] Cargando LightFM (cold-start WARP)...")
161     try:
162         from lightfm_model import SmarturLightFMModel
163         lightfm_model = SmarturLightFMModel()
164         if not lightfm_model.load():
165             if do_train and context_model is not None:
166                 ok = lightfm_model.train(engine.train_data, context_model.df_biz)
167                 if not ok:
168                     lightfm_model = None
169             else:
170                 lightfm_model = None
171     except Exception as lfm_err:
172         lightfm_model = None
173         logger.warning(f"[boot] LightFM no disponible: {lfm_err}")
174

```

```

175     logger.info("[boot] Cargando ContentModel (TF-IDF)...")
176     try:
177         from content_model import SmarturContentModel
178         content_model_cb = SmarturContentModel()
179         if context_model is not None:
180             content_model_cb.fit(context_model.df_biz)
181     except Exception as cm_err:
182         content_model_cb = None
183         logger.warning(f"[boot] ContentModel no disponible: {cm_err}")
184
185     try:
186         from poi_repository import fetch_quality_scores
187         quality_scores = fetch_quality_scores()
188         logger.info(f"[boot] Quality scores: {len(quality_scores)} servicios")
189     except Exception:
190         quality_scores = {}
191
192     # Estado inicial del blend dinámico (frío hasta que la BD diga otra cosa).
193     _refresh_data_warmth()
194
195     lfm_st = "LightFM✓" if lightfm_model else "LightFMx"
196     cm_st = "ContentModel✓" if content_model_cb else "ContentModelx"
197     logger.info(f"[boot] SMARTUR v4 listo (RF + SVD/CF + {lfm_st} + {cm_st}).")
198 except Exception as e:
199     logger.error(f"[boot] Error cargando modelos: {e}")
200 finally:
201     _training_in_progress = False
202
203
204 @asynccontextmanager
205 async def lifespan(app: FastAPI):
206     """
207     Arranque no bloqueante: carga modelos desde disco (ms) y, si falta alguno,
208     lanza el entrenamiento en un hilo separado sin bloquear el servidor.
209     """
210     global _training_in_progress
211
212     if os.getenv("SKIP_MODEL_BOOT") == "1":
213         logger.warning("SKIP_MODEL_BOOT=1: saltando carga de modelos en arranque")
214         yield
215         return
216
217     # Intentar cargar modelos desde disco sin entrenar (rápido)
218     _load_or_train_models(do_train=False)
219
220     # Si algún modelo no está listo, entrenar en background (no bloquea el API)
221     models_ready = (
222         engine is not None and
223         context_model is not None
224     )
225     if not models_ready:
226         missing = [n for n, v in [("engine", engine), ("rf", context_model)] if v is None]
227         logger.warning(
228             f"[boot] ⚠ API arrancando con modelos faltantes: {missing}. "
229             "Los endpoints /recommend devolverán 503 hasta que el entrenamiento termine."
230         )
231         logger.info("[boot] Modelos faltantes – iniciando entrenamiento en hilo de fondo. "
232             "API disponible en modo degradado.")
233         _training_in_progress = True
234         loop = asyncio.get_event_loop()

```

```

235         executor = ThreadPoolExecutor(max_workers=1, thread_name_prefix="smartur-train")
236         loop.run_in_executor(executor, _load_or_train_models, True)
237
238     # — Scheduled nightly retraining —————
239     # APScheduler runs _run_full_training() every day at 02:00 UTC so that
240     # new user interactions accumulated during the day are incorporated
241     # automatically – no admin button needed.
242     global _scheduler
243     try:
244         from apscheduler.schedulers.background import BackgroundScheduler
245         _scheduler = BackgroundScheduler(timezone='UTC')
246         _scheduler.add_job(
247             _run_full_training,
248             trigger='cron',
249             hour=2, minute=0,
250             id='nightly_retrain',
251             replace_existing=True,
252             misfire_grace_time=3600, # allow up to 1h late start (e.g. if container was down)
253         )
254         _scheduler.start()
255         logger.info("[scheduler] Reentrenamiento nocturno programado (02:00 UTC diario)")
256     except Exception as sched_err:
257         logger.warning(f"[scheduler] No disponible: {sched_err}")
258
259     yield
260
261     # — Teardown —————
262     if _scheduler is not None:
263         try:
264             _scheduler.shutdown(wait=False)
265         except Exception:
266             pass
267
268 app = FastAPI(title="SMARTUR Recommender API v2", version="2.1", lifespan=lifespan)
269
270 _CORS_ORIGINS = [o.strip() for o in os.getenv("CORS_ORIGINS", "http://localhost:3000").split(",")
271                  if o.strip()]
272
273 app.add_middleware(
274     CORSMiddleware,
275     allow_origins=_CORS_ORIGINS,
276     allow_credentials=True,
277     allow_methods=["GET", "POST", "OPTIONS"],
278     allow_headers=["*"],
279 )
280
281
282 class RecItem(BaseModel):
283     item_id: str
284     title: str
285     description: str = ''
286     category: str = ''
287     score: float
288     pred_cf: float
289     # Fuente real de pred_cf: 'knn'/'svd' = el CF-KNN aportó señal real;
290     # 'quality_proxy'/'global_mean' = todavía sin interacciones reales sobre
291     # este lugar, se usó la evaluación del admin o el promedio.
292     cf_signal: str = 'none'
293     pred_rf: float

```

```

294     pred_pref: float = 0.0
295     kind: str = 'poi'
296     reason_tags: list[str] = [] # human-readable explanation tags, e.g. ["Coincide con naturaleza
    ", "A 3.2 km"]
297
298
299 class RecommendationResponse(BaseModel):
300     user_id: str
301     recommendations: List[RecItem]
302     alpha: float
303
304
305 class RecommendRequest(BaseModel):
306     alpha: float = 0.4
307     context: Optional[Dict[str, Any]] = None
308     top_n: int = 5
309
310
311 @app.get("/health")
312 def health():
313     """
314     Endpoint pasivo para sondear la disponibilidad y estado de carga de la API.
315     Aporta métricas rápidas del estado interno del Engine y el RandomForest en RAM.
316     """
317     return {
318         "status": "ok",
319         "engine_ready":      engine is not None and engine.user_item_matrix is not None,
320         "rf_ready":          context_model is not None and getattr(context_model, 'is_fitted', F
    else),
321         "svd_ready":         engine is not None and hasattr(engine, 'user_latent'),
322         "lightfm_ready":     lightfm_model is not None and getattr(lightfm_model, 'is_fitted', F
    else),
323         "content_ready":     content_model_cb is not None and getattr(content_model_cb, 'is_fitt
    ed', False),
324         "users_count":       engine.user_item_matrix.shape[0] if engine and engine.user_item_mat
    rix is not None else 0,
325         "training_in_progress": _training_in_progress,
326         "skip_model_boot":    os.getenv("SKIP_MODEL_BOOT") == "1",
327         # Estado del blend dinámico: warmth=0 → la preferencia declarada domina;
328         # warmth=1 → los modelos aprendidos (CF/RF/LightFM) toman el control.
329         "data_warmth":        round(_data_warmth, 3),
330         "pref_weight":        round(0.65 - 0.45 * _data_warmth, 3),
331     }
332
333
334 @app.get("/health/poi-db")
335 def health_poi_db():
336     """Chequea conectividad a Postgres para POIs locales."""
337     try:
338         with get_poi_connection() as conn:
339             with conn.cursor() as cur:
340                 cur.execute("SELECT 1")
341                 cur.fetchone()
342             return {"status": "ok"}
343     except Exception as e:
344         raise HTTPException(status_code=503, detail=f"POI DB unavailable: {e}")
345
346
347 # -----
348 # Scheduler config – readable and writable from the admin dashboard

```

```

349 # -----
350
351 @app.get("/scheduler")
352 def get_scheduler_config():
353     """
354     Returns the current nightly retraining schedule.
355     Readable by the PLATAFORMA admin dashboard.
356     """
357     if _scheduler is None:
358         return {"enabled": False, "hour": 2, "minute": 0, "next_run": None}
359     job = _scheduler.get_job('nightly_retrain')
360     if job is None:
361         return {"enabled": False, "hour": 2, "minute": 0, "next_run": None}
362     # APScheduler CronTrigger stores fields in a list; find hour/minute by name
363     try:
364         trigger = job.trigger
365         field_map = {f.name: f for f in trigger.fields}
366         hour = int(str(field_map['hour'].expressions[0]))
367         minute = int(str(field_map['minute'].expressions[0]))
368     except Exception:
369         hour, minute = 2, 0
370     next_run = job.next_run_time.isoformat() if job.next_run_time else None
371     return {"enabled": True, "hour": hour, "minute": minute, "next_run": next_run}
372
373
374 class SchedulerConfigRequest(BaseModel):
375     enabled: bool
376     hour: int = Field(default=2, ge=0, le=23)
377     minute: int = Field(default=0, ge=0, le=59)
378
379
380 @app.put("/scheduler")
381 def put_scheduler_config(payload: SchedulerConfigRequest):
382     """
383     Enable/disable and/or reschedule nightly retraining.
384     Changes take effect immediately without restarting the container.
385     Called by the PLATAFORMA admin dashboard scheduler card.
386     """
387     if _scheduler is None:
388         raise HTTPException(status_code=503, detail="Scheduler no disponible en este entorno.")
389
390     if not payload.enabled:
391         try:
392             _scheduler.remove_job('nightly_retrain')
393         except Exception:
394             pass # already removed – not an error
395         logger.info("[scheduler] Reentrenamiento automático DESHABILITADO desde el dashboard")
396         return {"ok": True, "enabled": False, "next_run": None}
397
398     _scheduler.add_job(
399         _run_full_training,
400         trigger='cron',
401         hour=payload.hour,
402         minute=payload.minute,
403         id='nightly_retrain',
404         replace_existing=True,
405         misfire_grace_time=3600,
406     )
407     job = _scheduler.get_job('nightly_retrain')
408     next_run = job.next_run_time.isoformat() if job and job.next_run_time else None

```

```

409     logger.info(
410         f"[scheduler] Reprogramado a {payload.hour:02d}:{payload.minute:02d} UTC "
411         "desde el dashboard"
412     )
413     return {
414         "ok": True, "enabled": True,
415         "hour": payload.hour, "minute": payload.minute,
416         "next_run": next_run,
417     }
418
419
420 @app.get("/recommend/{user_id}", response_model=RecommendationResponse)
421 def get_recommendation(
422     user_id: str,
423     alpha: float = Query(0.4, ge=0.0, le=1.0),
424     top_n: int = Query(5, ge=1, le=50),
425 ):
426     if engine is None or context_model is None:
427         raise HTTPException(status_code=503, detail="Modelos no cargados.")
428
429     try:
430         recs = recommend_hybrid(
431             user_id, engine, context_model,
432             alpha=alpha, top_n=top_n,
433             lightfm_model=lightfm_model, content_model=content_model_cb,
434             quality_scores=quality_scores, data_warmth=data_warmth,
435         )
436         return RecommendationResponse(
437             user_id=user_id,
438             recommendations=[RecItem(**r) for r in recs],
439             alpha=alpha,
440         )
441     except Exception as e:
442         logger.error(f"Error en GET recommend: {e}")
443         raise HTTPException(status_code=500, detail=str(e))
444
445
446 @app.post("/recommend/{user_id}", response_model=RecommendationResponse)
447 def post_recommendation(user_id: str, payload: RecommendRequest):
448     if engine is None or context_model is None:
449         raise HTTPException(status_code=503, detail="Modelos no cargados.")
450
451     try:
452         logger.info(f"Recomendación POST para usuario: {user_id}")
453
454         # Smart merge: DB profile fills gaps; request overrides only non-empty fields.
455         # This prevents an empty request context from masking a rich DB profile,
456         # which was causing cold-start users to get flat (all-zero) feature vectors.
457         try:
458             profile_ctx = fetch_traveler_profile(user_id) or {}
459         except Exception as e:
460             profile_ctx = {}
461         logger.warning(f"No se pudo cargar perfil de viajero para {user_id}: {e}")
462
463         request_ctx = dict(payload.context) if payload.context else {}
464         merged_context = dict(profile_ctx) # base: DB profile
465         for key, val in request_ctx.items():
466             # Request field wins only if it's actually set (non-empty/non-null)
467             if val is not None and val != [] and val != '':
468                 merged_context[key] = val

```

```

469     if profile_ctx:
470         logger.info(f"Perfil de viajero cargado para usuario {user_id}")
471
472     # If request sent raw 'interests' but no 'tiposTurismo' exists yet (cold-start with no
473     # DB profile), convert them so filtrar_candidatos_por_contexto can use them.
474     if 'tiposTurismo' not in merged_context and merged_context.get('interests'):
475         from poi_repository import _INTEREST_MAP
476         tipos = []
477         for interest in merged_context['interests']:
478             mapped = _INTEREST_MAP.get(str(interest).lower().strip())
479             if mapped and mapped not in tipos:
480                 tipos.append(mapped)
481         if tipos:
482             merged_context['tiposTurismo'] = tipos
483
484     # Cold-start fallback: use region defaults when no profile or form context is available
485     if not merged_context:
486         merged_context = dict(_REGION_DEFAULT_CONTEXT)
487     logger.info(f"[cold-start] Sin contexto para usuario {user_id} – usando defaults de re
488     gión")
489
490     with _models_lock:
491         _engine = engine
492         _ctx = context_model
493         _lfm = lightfm_model
494         _cb = content_model_cb
495         _qs = quality_scores
496
497     recs = recommend_hybrid(
498         user_id=user_id,
499         engine=_engine,
500         context_model=_ctx,
501         alpha=payload.alpha,
502         context=merged_context,
503         top_n=payload.top_n,
504         lightfm_model=_lfm,
505         content_model=_cb,
506         quality_scores=_qs,
507         data_warmth=_data_warmth,
508     )
509     return RecommendationResponse(
510         user_id=user_id,
511         recommendations=[
512             RecItem(
513                 item_id=r.get('item_id', ''),
514                 title=r.get('title', ''),
515                 description=r.get('description', ''),
516                 category=r.get('category', ''),
517                 score=r.get('score', 0.0),
518                 pred_cf=r.get('pred_cf', 0.0),
519                 cf_signal=r.get('cf_signal', 'none'),
520                 pred_rf=r.get('pred_rf', 0.0),
521                 pred_pref=r.get('pred_pref', 0.0),
522                 kind=r.get('kind', 'poi'),
523                 reason_tags=r.get('reason_tags', []),
524             )
525             for r in recs
526         ],
527         alpha=payload.alpha,

```

```

528 except Exception as e:
529     logger.error(f"Error en POST recommend: {e}")
530     raise HTTPException(status_code=500, detail=str(e))
531
532
533 @app.get("/metrics")
534 def get_metrics():
535     """
536     Returns the latest stored algorithm metrics for the admin dashboard.
537     Priority:
538     1. DB table ml_model_metrics (most recent row)
539     2. models/algorithm_metrics.json (written by _run_full_training)
540     Includes feature_importances, hyperparameters, and training_history.
541     """
542     _BASE = Path(_MODELS)
543     data = {}
544
545     # 1. Try DB first (pooled connection – no new TCP handshake per request)
546     try:
547         pool = _get_db_pool()
548         conn = pool.getconn()
549         try:
550             with conn.cursor() as cur:
551                 cur.execute(
552                     "SELECT metrics_json FROM ml_model_metrics ORDER BY created_at DESC LIMIT 1"
553                 )
554                 row = cur.fetchone()
555             finally:
556                 pool.putconn(conn)
557             if row:
558                 data = row[0] # psycopg2 returns jsonb as dict automatically
559         except Exception as db_err:
560             logger.warning(f"[metrics] DB fallback: {db_err}")
561
562     # 2. Fall back to JSON file in the models volume
563     if not data:
564         metrics_path = _BASE / "algorithm_metrics.json"
565         if not metrics_path.exists():
566             raise HTTPException(status_code=404, detail="No hay métricas almacenadas todavía.")
567         try:
568             with open(metrics_path, encoding="utf-8") as f:
569                 data = json.load(f)
570             if "status" in data and "note" in data and len(data) == 2:
571                 raise HTTPException(status_code=404, detail="Métricas aún no calculadas. Llama a /
train primero.")
572         except HTTPException:
573             raise
574         except Exception as e:
575             logger.error(f"Error leyendo métricas: {e}")
576             raise HTTPException(status_code=500, detail="Error al leer métricas.")
577
578     # 3. Enrich with feature importances and hyperparameters
579     fi_path = _BASE / "rf_feature_importances.json"
580     if fi_path.exists():
581         try:
582             with open(fi_path) as f:
583                 data["feature_importances"] = json.load(f)
584         except Exception as e:
585             logger.warning(f"[metrics] No se pudieron cargar feature importances: {e}")
586

```

```

587 hp_path = _BASE / "rf_hyperparams.json"
588 if hp_path.exists():
589     try:
590         with open(hp_path) as f:
591             data["hyperparameters"] = json.load(f)
592     except Exception as e:
593         logger.warning(f"[metrics] No se pudieron cargar hyperparams: {e}")
594
595 # 4. Enrich with GBM feature importances and hyperparams
596 gbm_fi_path = _BASE / "gbm_feature_importances.json"
597 if gbm_fi_path.exists():
598     try:
599         with open(gbm_fi_path) as f:
600             data["gbm_feature_importances"] = json.load(f)
601     except Exception as e:
602         logger.warning(f"[metrics] No se pudieron cargar GBM feature importances: {e}")
603
604 gbm_hp_path = _BASE / "gbm_hyperparams.json"
605 if gbm_hp_path.exists():
606     try:
607         with open(gbm_hp_path) as f:
608             data["gbm_hyperparameters"] = json.load(f)
609     except Exception as e:
610         logger.warning(f"[metrics] No se pudieron cargar GBM hyperparams: {e}")
611
612 # 5. Enrich with training history
613 history_path = _BASE / "training_history.json"
614 if history_path.exists():
615     try:
616         with open(history_path) as f:
617             data["training_history"] = json.load(f)
618     except Exception as e:
619         logger.warning(f"[metrics] No se pudo cargar training history: {e}")
620
621 return data
622
623
624 def _enrich_metrics(actuals, preds, engine_obj, context_model):
625     """Adds error distribution, prediction distribution, and data_quality to a metrics dict."""
626     import numpy as np
627     actuals = np.asarray(actuals, dtype=float)
628     preds = np.asarray(preds, dtype=float)
629     errors_abs = np.abs(actuals - preds)
630
631     # Error distribution
632     ed = {}
633     for t in [0.5, 1.0, 1.5, 2.0]:
634         ed[f'within_{str(t).replace(".", "_')}'] = round(float((errors_abs <= t).mean() * 100), 1)
635     error_distribution = ed
636
637     # Prediction distribution (actual vs predicted counts per star level)
638     actual_buckets = []
639     pred_buckets = []
640     for star in range(1, 6):
641         actual_buckets.append([star, int((actuals == star).sum())])
642         pred_buckets.append([star, int(((preds >= star - 0.5) & (preds < star + 0.5)).sum())])
643     prediction_distribution = {
644         'actual_buckets': actual_buckets,
645         'predicted_buckets': pred_buckets,
646     }

```

```

647
648 # Data quality
649 data_quality = {
650     'total_interactions': int(len(engine_obj.train_data)) if hasattr(engine_obj, 'train_data')
        and engine_obj.train_data is not None else 0,
651     'total_test': int(len(engine_obj.test_data)) if hasattr(engine_obj, 'test_data') and engine_obj.test_data is not None else 0,
652     'users_count': int(engine_obj.user_item_matrix.shape[0]) if engine_obj and engine_obj.user_item_matrix is not None else 0,
653     'businesses_count': int(engine_obj.user_item_matrix.shape[1]) if engine_obj and engine_obj.user_item_matrix is not None else 0,
654     'top_categories': getattr(context_model, 'top_categories', [])[:10],
655     'features_count': len(getattr(context_model, 'features', [])),
656 }
657
658 return {
659     'error_distribution': error_distribution,
660     'prediction_distribution': prediction_distribution,
661     'data_quality': data_quality,
662 }
663
664
665 def _compute_simple_metrics(engine_obj, rf_model, sample_size: int = 800) -> dict:
666     """
667     Computes CF-Pearson + Random-Forest metrics on held-out test data.
668     Does NOT require a GBM model (simplified vs. compare_algorithms).
669     Returns a dict compatible with the ml_model_metrics schema.
670
671     NOTE: This function is kept as a fallback for when GBM is not loaded.
672     Prefer compare_algorithms() when gbm_model is available.
673     """
674     import numpy as np
675     from math import sqrt
676     from sklearn.metrics import mean_absolute_error, mean_squared_error
677     from cf import predict_cf_pearson, reset_prediction_stats, get_prediction_stats
678     from evaluate import _infer_user_context
679     from model_metrics import DEFAULT_METRICS
680
681     try:
682         test_data = getattr(engine_obj, 'test_data', None)
683         if test_data is None or len(test_data) == 0:
684             logger.warning("[metrics] No hay datos de prueba – usando métricas por defecto.")
685             return dict(DEFAULT_METRICS)
686
687         n_eval = min(sample_size, len(test_data))
688         test_sample = test_data.sample(n_eval, random_state=42)
689         reset_prediction_stats()
690
691         # Pre-compute user contexts so predict_with_context gets real signal
692         user_contexts = {
693             uid: _infer_user_context(uid, engine_obj)
694             for uid in test_sample['user_id'].unique()
695         }
696
697         actuals, preds_cf, preds_rf, business_ids = [], [], [], []
698         for _, row in test_sample.iterrows():
699             try:
700                 p_cf = predict_cf_pearson(row['user_id'], row['business_id'], engine_obj)
701                 ctx = user_contexts.get(row['user_id'])
702                 p_rf = float(rf_model.predict_with_context([row['business_id']], user_context=ctx))

```

[0])

```

703
704     # Only skip if RF is invalid (CF always returns a fallback mean)
705     if np.isnan(p_rf):
706         continue
707     if np.isnan(p_cf):
708         p_cf = p_rf # graceful CF fallback when KNN has no neighbors
709
710     actuals.append(row['stars'])
711     preds_cf.append(p_cf)
712     preds_rf.append(p_rf)
713     business_ids.append(row['business_id'])
714 except Exception:
715     continue
716
717 if len(actuals) < 30:
718     logger.warning(f"[metrics] Solo {len(actuals)} muestras válidas – usando métricas por
defecto.")
719     return dict(DEFAULT_METRICS)
720
721 actuals = np.asarray(actuals, dtype=float)
722 preds_cf = np.asarray(preds_cf, dtype=float)
723 preds_rf = np.asarray(preds_rf, dtype=float)
724
725 # Umbral de relevancia: un lugar es "relevante" si su calificación
726 # (real o predicha) es >= 4 estrellas. Esto convierte la regresión de
727 # rating en una CLASIFICACIÓN binaria (relevante / no relevante), lo
728 # que permite reportar Accuracy/Precision/Recall/F1 – las métricas
729 # estándar para evaluar recomendadores como clasificador de relevancia.
730 _REL_THRESHOLD = 4.0
731
732 def _classification_metrics(a, p, thr=_REL_THRESHOLD):
733     a_rel = np.asarray(a, dtype=float) >= thr
734     p_rel = np.asarray(p, dtype=float) >= thr
735     tp = int(np.sum(a_rel & p_rel))
736     fp = int(np.sum(~a_rel & p_rel))
737     fn = int(np.sum(a_rel & ~p_rel))
738     tn = int(np.sum(~a_rel & ~p_rel))
739     total = tp + fp + fn + tn
740     accuracy = (tp + tn) / total if total else 0.0
741     precision = tp / (tp + fp) if (tp + fp) else 0.0
742     recall = tp / (tp + fn) if (tp + fn) else 0.0
743     f1 = (2 * precision * recall / (precision + recall)) if (precision + recall) else 0.0
744     return {
745         'accuracy': round(float(accuracy), 4),
746         'precision': round(float(precision), 4),
747         'recall': round(float(recall), 4),
748         'f1': round(float(f1), 4),
749     }
750
751 def _rmse_mae(a, p):
752     return {
753         'rmse': float(sqrt(mean_squared_error(a, p))),
754         'mae': float(mean_absolute_error(a, p)),
755         **_classification_metrics(a, p),
756     }
757
758 # Find best blending alpha (CF weight) for hybrid
759 best_alpha, best_rmse_h = 0.2, float('inf')
760 for alpha in np.arange(0.0, 1.05, 0.1):

```

```

761         hybrid = alpha * preds_cf + (1.0 - alpha) * preds_rf
762         rmse_h = sqrt(mean_squared_error(actuals, hybrid))
763         if rmse_h < best_rmse_h:
764             best_rmse_h = rmse_h
765             best_alpha = float(alpha)
766
767     hybrid_preds = best_alpha * preds_cf + (1.0 - best_alpha) * preds_rf
768     baseline = np.full_like(actuals, actuals.mean())
769
770     # item_mean: promedio de calificaciones por negocio en el TRAIN set,
771     # aplicado al test set (fallback a la media global si el negocio no
772     # aparece en train). Candidato mucho más barato que CF/RF/híbrido –
773     # cero entrenamiento, solo un groupby. Es un baseline "justo" para
774     # datos sintéticos generados centrados en el promedio de cada
775     # negocio (ver seed_pois_mexico.py/pre_procesamiento_mexico.py): si
776     # item_mean le gana a CF/RF, es señal de que la personalización no
777     # está aportando nada por encima de "cuánto le gusta este lugar a
778     # la gente en general", sin necesidad de retrain para saberlo.
779     item_means_train = engine_obj.train_data.groupby('business_id')['stars'].mean()
780     global_mean = float(actuals.mean())
781     preds_item_mean = np.array(
782         [float(item_means_train.get(bid, global_mean)) for bid in business_ids]
783     )
784
785     algorithms = {
786         'baseline': _rmse_mae(actuals, baseline),
787         'item_mean': _rmse_mae(actuals, preds_item_mean),
788         'cf_knn_pearson': _rmse_mae(actuals, preds_cf),
789         'random_forest': _rmse_mae(actuals, preds_rf),
790         'hybrid_cf_rf': (**_rmse_mae(actuals, hybrid_preds), 'alpha': best_alpha),
791     }
792
793     # 'baseline'/'item_mean' Sí compiten – antes 'baseline' se excluía a
794     # propósito de "candidates", lo que garantizaba que el sistema nunca
795     # pudiera admitir que predecir el promedio simple le gana a
796     # CF/RF/híbrido. Eso ocultaba el problema real (el modelo no aporta
797     # valor) en vez de reportarlo. 'item_mean' es aún más informativo:
798     # si gana, significa que ni siquiera hace falta CF/RF – con el
799     # promedio de calificaciones de cada negocio alcanza.
800     candidates = {
801         k: algorithms[k]['rmse']
802         for k in ('baseline', 'item_mean', 'cf_knn_pearson', 'random_forest', 'hybrid_cf_rf')
803     }
804     best_algorithm = min(candidates, key=candidates.get)
805     if best_algorithm in ('baseline', 'item_mean'):
806         logger.warning(
807             f"[metrics] '{best_algorithm}' (RMSE={algorithms[best_algorithm]['rmse']:.4f}) "
808             f"le gana a CF/RF/híbrido – el modelo no está aportando valor medible hoy."
809         )
810
811     cf_stats = get_prediction_stats()
812     if cf_stats.get('fallback_rate') is not None and cf_stats['fallback_rate'] > 0.5:
813         logger.warning(
814             f"[metrics] {cf_stats['fallback_rate']*100:.1f}% de las predicciones de CF "
815             f"cayeron en fallback (sin vecinos con señal real) – dataset demasiado disperso."
816         )
817
818     # Razón legible de por qué se eligió este modelo – para mostrar en el
819     # dashboard junto a la tabla comparativa.
820     _best_rmse = algorithms[best_algorithm]['rmse']

```

```

821     if best_algorithm in ('baseline', 'item_mean'):
822         _rationale = (
823             f"Seleccionado por menor RMSE ({_best_rmse:.3f}). Que un método sin "
824             f"aprendizaje ({best_algorithm}) gane indica que CF/RF aún no aportan "
825             f"valor sobre estos datos – el sistema lo reporta con honestidad en vez "
826             f"de forzar un modelo peor."
827         )
828     else:
829         _rationale = (
830             f"Seleccionado por menor RMSE ({_best_rmse:.3f}) entre todos los "
831             f"algoritmos evaluados sobre el mismo conjunto de prueba."
832         )
833
834     result = {
835         'best_algorithm': best_algorithm,
836         'best_alpha':     best_alpha if best_algorithm == 'hybrid_cf_rf' else 0.2,
837         'local_blend':    {'rf': 1.0, 'gbm': 0.0},
838         'algorithms':     algorithms,
839         'sample_size':    len(actuals),
840         'cf_prediction_sources': cf_stats,
841         # Info de selección de modelos para el dashboard:
842         'selection_metric': 'rmse', # criterio de elección (menor = mejor)
843         'relevance_threshold': _REL_THRESHOLD, # umbral para Accuracy/Precision/Recall/F1
844         'selection_rationale': _rationale,
845         'data_warmth':       round(_data_warmth, 3),
846     }
847     enrich = _enrich_metrics(actuals, hybrid_preds, engine_obj, rf_model)
848     result.update(enrich)
849     return result
850 except Exception as exc:
851     logger.error(f"[metrics] Error calculando métricas: {exc}")
852     return dict(DEFAULT_METRICS)
853
854
855 # Mínimo de usuarios reales evaluados para confiar en el NDCG como criterio
856 # primario (coincide con evaluar_ranking_local.min_users). Por debajo de esto,
857 # el ranking es demasiado ruidoso y se mantiene el RMSE como proxy estable.
858 _RANKING_TRUST_MIN_USERS = 20
859
860
861 def _apply_ranking_based_selection(metrics: dict, min_users: int = _RANKING_TRUST_MIN_USERS) -> dict:
862     """
863     Relevo automático del criterio de selección (misma filosofía que data_warmth,
864     aplicada a la métrica-norte). Cuando las métricas de ranking son CONFIABLES
865     –evaluación local sobre >= min_users usuarios reales con historial–, el
866     criterio primario con el que se juzga y presenta el sistema pasa de RMSE
867     (error de predicción de estrellas) a NDCG (calidad de ORDENAMIENTO, que es
868     lo que de verdad le importa a una recomendación). Si no hay datos
869     suficientes, se conserva el RMSE como proxy estable.
870
871     No cambia `best_algorithm` (qué predictor de rating se mezcla en el blend);
872     cambia `selection_metric`/`selection_rationale` – la métrica con la que el
873     sistema se autoevalúa. En modo sintético NO se aplica: ese ranking no
874     representa usuarios reales.
875     """
876     if metrics.get('synthetic_augmented'):
877         return metrics
878
879     ranking = metrics.get('ranking')

```

```

880     trustworthy = (
881         isinstance(ranking, dict)
882         and not ranking.get('error')
883         and ranking.get('ndcg') is not None
884         and (ranking.get('n_users_evaluated') or 0) >= min_users
885     )
886     if not trustworthy:
887         metrics.setdefault('selection_metric', 'rmse')
888         return metrics
889
890     ndcg = float(ranking['ndcg'])
891     n_users = int(ranking.get('n_users_evaluated') or 0)
892     pref = ranking.get('preference_match_rate')
893     pref_txt = (
894         f"Coincidencia con preferencias declaradas: {pref:.1%}."
895         if isinstance(pref, (int, float)) else ""
896     )
897     metrics['selection_metric'] = 'ndcg'
898     metrics['selection_rationale'] = (
899         f"Criterio primario: NDCG@5 = {ndcg:.3f} sobre {n_users} usuarios reales con "
900         f"historial suficiente. El sistema ya tiene datos para juzgarse por CALIDAD DE "
901         f"ORDENAMIENTO (poner lo mejor arriba) en vez de por error de predicción de "
902         f"estrellas (RMSE)." + pref_txt
903     )
904     return metrics
905
906 def _persist_metrics_to_db(metrics_json: dict) -> bool:
907     """
908     Inserts ML training metrics into the ml_model_metrics table
909     using the same PostgreSQL connection as poi_repository.
910     """
911     try:
912         import psycopg2
913         import json as _json
914
915         conn = psycopg2.connect(
916             host=os.getenv('POI_DB_HOST', 'localhost'),
917             port=int(os.getenv('POI_DB_PORT', 5432)),
918             database=os.getenv('POI_DB_NAME', 'smartur'),
919             user=os.getenv('POI_DB_USER', 'postgres'),
920             password=os.getenv('POI_DB_PASSWORD', os.getenv('DB_PASSWORD', '')),
921             options='-c client_encoding=UTF8',
922         )
923         with conn:
924             with conn.cursor() as cur:
925                 cur.execute(
926                     "INSERT INTO ml_model_metrics (metrics_json) VALUES (%s)",
927                     (_json.dumps(metrics_json, ensure_ascii=False),),
928                 )
929             conn.close()
930             return True
931     except Exception as exc:
932         logger.error(f"[train] No se pudo persistir métricas en DB: {exc}")
933         return False
934
935 def _persist_cv_metrics_to_db(cv_json: dict) -> bool:
936     """Inserts k-fold cross-validation results into ml_cross_validation_metrics."""
937     try:

```

```

940     import psycopg2
941     import json as _json
942
943     conn = psycopg2.connect(
944         host=os.getenv('POI_DB_HOST', 'localhost'),
945         port=int(os.getenv('POI_DB_PORT', 5432)),
946         database=os.getenv('POI_DB_NAME', 'smartur'),
947         user=os.getenv('POI_DB_USER', 'postgres'),
948         password=os.getenv('POI_DB_PASSWORD', os.getenv('DB_PASSWORD', '')),
949         options='-c client_encoding=UTF8',
950     )
951     with conn:
952         with conn.cursor() as cur:
953             cur.execute(
954                 "INSERT INTO ml_cross_validation_metrics (cv_json) VALUES (%s)",
955                 (_json.dumps(cv_json, ensure_ascii=False),),
956             )
957     conn.close()
958     return True
959 except Exception as exc:
960     logger.error(f"[cv] No se pudo persistir cross-validation en DB: {exc}")
961     return False
962
963
964 def _compute_enrich_from_engine(engine_obj, rf_model):
965     """Computes enrichment metrics using a sample of test data."""
966     import numpy as np
967     from math import sqrt
968     from sklearn.metrics import mean_absolute_error, mean_squared_error
969     from cf import predict_cf_pearson
970     from evaluate import _infer_user_context
971
972     try:
973         test_data = getattr(engine_obj, 'test_data', None)
974         if test_data is None or len(test_data) == 0:
975             return {}
976         sample = test_data.sample(min(800, len(test_data)), random_state=42)
977         contexts = {uid: _infer_user_context(uid, engine_obj) for uid in sample['user_id'].unique()}
978
979         actuals, preds = [], []
980         for _, row in sample.iterrows():
981             try:
982                 p_cf = predict_cf_pearson(row['user_id'], row['business_id'], engine_obj)
983                 ctx = contexts.get(row['user_id'])
984                 p_rf = float(rf_model.predict_with_context([row['business_id']], user_context=ctx)
985                     [0])
986                 if np.isnan(p_rf):
987                     continue
988                 if np.isnan(p_cf):
989                     p_cf = p_rf
990                 hybrid = 0.2 * p_cf + 0.8 * p_rf
991                 actuals.append(row['stars'])
992                 preds.append(hybrid)
993             except Exception:
994                 continue
995         if len(actuals) < 30:
996             return {}
997         return _enrich_metrics(actuals, preds, engine_obj, rf_model)
998 except Exception:
999     return {}

```

```

998
999
1000 def _run_full_training():
1001     """
1002     Background worker: retrains all models (RF + GBM + LightFM), refreshes
1003     Pearson/SVD matrix, computes full algorithm comparison metrics, appends
1004     ranking metrics, and persists to DB.
1005     """
1006     global engine, context_model, lightfm_model, content_model_cb, quality_scores
1007     try:
1008         import pandas as pd
1009         from model_metrics import save_metrics, compare_algorithms
1010         from poi_repository import fetch_real_interactions, fetch_evaluation_scores
1011
1012         # Todo lo que muta engine/context_model/lightfm_model/content_model_cb
1013         # EN VIVO (in-place, no reasignación de referencia) va bajo _models_lock.
1014         # Antes el lock solo protegía la lectura (snapshot de referencias en
1015         # /recommend), pero como el retrain modifica los mismos objetos en
1016         # lugar de construir unos nuevos y reasignarlos, una request podía
1017         # quedarse con una referencia al motor mientras este se reescribía a
1018         # mitad de camino (matriz Pearson reconstruida, RF reentrenado, etc.).
1019         # — 0. ¿Modo sintético (opt-in, reversible)? —————
1020         # Cuando SMARTUR SYNTH TRAINING está activo, se REEMPLAZA train/test por
1021         # un dataset sintético a gran escala con estructura latente aprendible,
1022         # para demostrar (train/test + CV + API + dashboard + persistencia) que
1023         # el ML supera al baseline cuando existe estructura. OFF por defecto:
1024         # en ese caso el flujo es idéntico al de siempre (datos reales/CSV).
1025         # Las métricas se marcan con synthetic_augmented para no confundir este
1026         # desempeño con el de usuarios reales.
1027         from synthetic_training import synth_training_enabled, build_synthetic_split
1028         _synth_active = synth_training_enabled()
1029
1030         with _models_lock:
1031             restmex.biz = None
1032             if _synth_active:
1033                 # — 1s. Reemplazo sintético (NO son usuarios reales) —————
1034                 logger.warning(
1035                     "[train][SYNTH] SMARTUR SYNTH TRAINING activo – entrenando sobre "
1036                     "estructura latente INVENTADA. Las métricas NO representan turistas reales."
1037                 )
1038                 syn_train, syn_test = build_synthetic_split(engine.df_biz)
1039                 engine.train_data = syn_train
1040                 engine.test_data = syn_test
1041             else:
1042                 # — 1. Merge real DB interactions (if any accumulated) —————
1043                 # Threshold lowered 50->10 so early adopters immediately influence training.
1044                 # Weight scaled adaptively (3x min -> 10x max) to compensate for the 266K
1045                 # Yelp corpus – without this, SMARTUR signals are drowned out.
1046                 _MIN_REAL = 10
1047                 try:
1048                     real_df = fetch_real_interactions(min_events=1)
1049                     if real_df is not None and len(real_df) >= _MIN_REAL:
1050                         real_df = real_df.rename(columns={'item_id': 'business_id', 'implicit_score': 'stars'})
1051                         real_df['user_id'] = real_df['user_id'].astype(str)
1052                         n_repeats = max(3, min(10, len(real_df) // 5))
1053                         tiles = [real_df] * n_repeats
1054                         engine.train_data = pd.concat([engine.train_data] + tiles, ignore_index=True)
1055                         logger.info(

```

```

1056         f"[train] Datos reales: {len(real_df)} interacciones, ponderadas {n_re
1057         peats}x"
1058     )
1059     else:
1060         n_real = len(real_df) if real_df is not None else 0
1061         logger.info(f"[train] Solo {n_real} interacciones reales (mínimo {_MIN_REA
1062         L}) - usando solo CSV.")
1063     except Exception as exc:
1064         logger.warning(f"[train] Interacciones reales no disponibles: {exc}")
1065
1066     # — 1b. Merge admin evaluation scores —————
1067     try:
1068         eval_df = fetch_evaluation_scores()
1069         if eval_df is not None and len(eval_df) > 0:
1070             # evaluation scores only cover tourist services (svc_N IDs)
1071             # Merge columns must match training schema
1072             eval_df['user_id'] = eval_df['user_id'].astype(str)
1073             eval_df = eval_df.rename(columns={'business_id': 'business_id', 'stars': '
1074             stars'})
1075             engine.train_data = pd.concat(
1076                 [engine.train_data, eval_df], ignore_index=True
1077             )
1078             logger.info(f"[train] {len(eval_df)} scores de evaluación admin integrados
1079             al entrenamiento.")
1080         except Exception as exc:
1081             logger.warning(f"[train] Scores de evaluación admin no disponibles: {exc}")
1082
1083     # — 1c. Rest-Mex ya incluido en seed_pois_mexico.py —————
1084     restmex_biz = None
1085
1086     # — 2. Rebuild Pearson + SVD matrix —————
1087     logger.info("[train] Actualizando matriz de Pearson + SVD...")
1088     engine.prepare_pearson_matrix()
1089
1090     # — 3. Retrain RF —————
1091     logger.info("[train] Reentrenando Random Forest...")
1092     context_model.train(engine.train_data, df_biz_extra=restmex_biz)
1093
1094     # — 4b. Retrain LightFM —————
1095     # Si LightFM falló al cargar en el boot (pickle incompatible, archivo
1096     # faltante, etc.) queda en None - sin este fallback, el reentrenamiento
1097     # nocturno lo saltaba para siempre porque solo reentrenaba una instancia
1098     # que ya existiera. Se crea una instancia nueva aquí mismo si hace falta,
1099     # así el sistema se autorecupera en el siguiente ciclo de entrenamiento.
1100     if lightfm_model is None:
1101         try:
1102             from lightfm_model import SmarturLightFMModel
1103             lightfm_model = SmarturLightFMModel()
1104             logger.info("[train] LightFM estaba caído - instancia nueva creada para reinte
1105             ntar.")
1106         except Exception as lfm_init_err:
1107             logger.warning(f"[train] No se pudo instanciar LightFM: {lfm_init_err}")
1108             lightfm_model = None
1109
1110     if lightfm_model is not None:
1111         logger.info("[train] Reentrenando LightFM...")
1112         try:
1113             lightfm_model.train(engine.train_data, context_model.df_biz)
1114         except Exception as lfm_err:
1115             logger.warning(f"[train] LightFM reentrenamiento falló: {lfm_err}")

```

```

1111
1112 # — 4c. Refit ContentModel —————
1113 if content_model_cb is not None:
1114     try:
1115         content_model_cb.fit(context_model.df_biz)
1116     except Exception as cm_err:
1117         logger.warning(f"[train] ContentModel refit falló: {cm_err}")
1118
1119 # — 5. Algorithm metrics (RF + CF) —————
1120 logger.info("[train] Calculando métricas de algoritmos...")
1121 metrics = _compute_simple_metrics(engine, context_model)
1122
1123 # Integridad: si el entrenamiento corrió en modo sintético, marcarlo para
1124 # que el dashboard avise que este desempeño es sobre una verdad inventada,
1125 # no sobre turistas reales. Nunca se debe presentar como métrica real.
1126 if _synth_active:
1127     metrics['synthetic_augmented'] = True
1128     _prev_rationale = metrics.get('selection_rationale', '')
1129     metrics['selection_rationale'] = (
1130         "⚠ ENTRENAMIENTO SINTÉTICO (SMARTUR SYNTH_TRAINING activo): estos "
1131         "resultados son sobre personas sintéticas con estructura latente "
1132         "inventada, para validar que el pipeline de ML aprende cuando existe "
1133         "estructura. NO representan el comportamiento de turistas reales. "
1134         + _prev_rationale
1135     )
1136
1137 # — 5b. Enrich with error_dist, data_quality, prediction_distribution —
1138 if 'error_distribution' not in metrics:
1139     try:
1140         enrich = _compute_enrich_from_engine(engine, context_model)
1141         metrics.update(enrich)
1142     except Exception as exc:
1143         logger.warning(f"[train] Enrichment no disponible: {exc}")
1144
1145 # — 6. Ranking metrics (NDCG@5, Precision@5, Hit Rate@10) —————
1146 # Prefer local SMARTUR evaluation when we have enough real interactions;
1147 # Yelp eval always returns NDCG=0 because recommender returns Veracruz POIs.
1148 try:
1149     from evaluate import evaluar_ranking, evaluar_ranking_local
1150     ranking = evaluar_ranking_local(
1151         context_model=context_model,
1152         lightfm_model=lightfm_model,
1153         content_model=content_model_cb,
1154         k=5,
1155     )
1156     _ranking_is_local = False
1157     if ranking is not None:
1158         metrics['ranking'] = ranking
1159         _ranking_is_local = True # evaluación sobre POIs + usuarios reales
1160         logger.info(
1161             f"[train] Ranking local SMARTUR – NDCG@5={ranking.get('ndcg', 0):.4f}, "
1162             f"P@5={ranking.get('precision', 0):.4f}, "
1163             f"HR@10={ranking.get('hit_rate', 0):.4f}, "
1164             f"PrefMatch={ranking.get('preference_match_rate', 0):.4f}"
1165         )
1166     else:
1167         # Fallback: Yelp-based eval (will show NDCG=0 by design)
1168         ranking = evaluar_ranking(engine, context_model, n_users=50, k=5)
1169         if ranking:
1170             metrics['ranking'] = ranking

```

```

1171         logger.info("[train] Ranking Yelp (fallback – sin suficientes datos reales SMA
RTUR)")
1172
1173         # Relevo automático RMSE → NDCG: solo cuando el ranking viene de la
1174         # evaluación LOCAL (POIs + usuarios reales). El fallback Yelp da
1175         # NDCG=0 por diseño y nunca debe convertirse en criterio primario.
1176         if _ranking_is_local:
1177             _apply_ranking_based_selection(metrics)
1178     except Exception as exc:
1179         logger.warning(f"[train] Ranking evaluation omitida: {exc}")
1180
1181     # — 7. Persist locally —————
1182     save_metrics(metrics)
1183     _best_algo = metrics.get('best_algorithm', '-')
1184     _rmse_val = metrics.get('algorithms', {}).get(_best_algo, {}).get('rmse')
1185     _rmse_str = f"{_rmse_val:.4f}" if isinstance(_rmse_val, (int, float)) else '-'
1186     logger.info(f"[train] Métricas guardadas – mejor algoritmo: {_best_algo} (RMSE={_rmse_str}
)")
1187
1188     # — 7b. Append to training history —————
1189     try:
1190         from datetime import datetime, timezone
1191         history_path = Path(MODELS) / "training_history.json"
1192         history = []
1193         if history_path.exists():
1194             with open(history_path) as fh:
1195                 history = json.load(fh)
1196         entry = {
1197             'timestamp': datetime.now(timezone.utc).isoformat(),
1198             'best_algorithm': _best_algo,
1199             'sample_size': metrics.get('sample_size', 0),
1200         }
1201         best_key = metrics.get('best_algorithm', '')
1202         best_rmse = metrics.get('algorithms', {}).get(best_key, {}).get('rmse')
1203         best_mae = metrics.get('algorithms', {}).get(best_key, {}).get('mae')
1204         if best_rmse is not None:
1205             entry['rmse'] = round(float(best_rmse), 4)
1206         if best_mae is not None:
1207             entry['mae'] = round(float(best_mae), 4)
1208         if 'error_distribution' in metrics:
1209             entry['error_within_1'] = metrics['error_distribution'].get('within_1_0', 0)
1210         if 'ranking' in metrics:
1211             entry['ndcg'] = metrics['ranking'].get('ndcg', 0)
1212             entry['hit_rate'] = metrics['ranking'].get('hit_rate', 0)
1213             if 'preference_match_rate' in metrics['ranking']:
1214                 entry['preference_match_rate'] = metrics['ranking']['preference_match_rate']
1215         history.append(entry)
1216         # Keep last 20 entries
1217         history = history[-20:]
1218         with open(history_path, 'w') as fh:
1219             json.dump(history, fh, indent=2, ensure_ascii=False)
1220         logger.info(f"[train] Training history actualizado ({len(history)} entradas)")
1221     except Exception as h_err:
1222         logger.warning(f"[train] No se pudo persistir training history: {h_err}")
1223
1224     # — 8. Persist to DB (only if we have real algorithm data) —————
1225     if not metrics.get('algorithms'):
1226         logger.warning(f"[train] Métricas vacías – no se persiste en DB (datos insuficientes).")
1227
1228     elif _persist_metrics_to_db(metrics):

```

```

1228         logger.info("[train] Métricas persistidas en DB correctamente.")
1229     else:
1230         logger.warning("[train] Métricas solo disponibles en archivo local (DB no actualizada)
1231     .")
1232
1232     # — 9. Refresh quality scores in-memory (new evaluations during training) —
1233     try:
1234         from poi_repository import fetch_quality_scores
1235         new_quality_scores = fetch_quality_scores()
1236         with _models_lock:
1237             quality_scores = new_quality_scores
1238         logger.info(f"[train] Quality scores refrescados: {len(quality_scores)} servicios")
1239     except Exception as exc:
1240         logger.warning(f"[train] Quality scores no actualizados: {exc}")
1241
1242     # Recalcula el warmth: tras entrenar puede haber más interacciones reales.
1243     _refresh_data_warmth()
1244
1245     logger.info(f"[train] Entrenamiento completado. Muestra: {metrics.get('sample_size', '?')}
1246     predicciones.")
1247 except Exception as e:
1248     logger.error(f"[train] Error en background training: {e}", exc_info=True)
1249
1250 @app.post("/train")
1251 def train_full(background_tasks: BackgroundTasks):
1252     """
1253     Inicia el reentrenamiento completo en background (fire-and-forget).
1254     Actualiza la matriz de Pearson y el modelo Random Forest.
1255     Returns immediately with { ok: true }.
1256     """
1257     if engine is None or context_model is None:
1258         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1259     background_tasks.add_task(_run_full_training)
1260     return {"ok": True, "message": "Entrenamiento iniciado en background"}
1261
1262
1263 _cv_in_progress = False # True mientras corre la validación cruzada en background
1264
1265
1266 def _run_cross_validation(k: int = 5, sample_size: int = 3000) -> None:
1267     """
1268     Background worker: k-fold cross-validation de CF/RF/GBM (ver
1269     cross_validation.py). Complementa el train/test split que ya calcula
1270     _run_full_training/_compute_simple_metrics – responde al requisito de
1271     que cada algoritmo se evalúe tanto con validación cruzada como con
1272     división train/test.
1273     """
1274     global _cv_in_progress
1275     _cv_in_progress = True
1276     try:
1277         from cross_validation import run_kfold_cv
1278         logger.info(f"[cv] Iniciando k-fold cross-validation (k={k}, sample_size={sample_size})...
1279     ")
1280     t0 = time.time()
1281     cv_result = run_kfold_cv(engine, k=k, sample_size=sample_size)
1282     cv_result['total execution time_ms'] = (time.time() - t0) * 1000
1283     cv_result['timestamp'] = datetime.now(timezone.utc).isoformat()
1284     cv_path = Path(_MODELS) / "cv_metrics.json"

```

```

1285         with open(cv_path, 'w', encoding='utf-8') as f:
1286             json.dump(cv_result, f, ensure_ascii=False, indent=2)
1287
1288         _persist_cv_metrics_to_db(cv_result)
1289
1290         for algo, m in cv_result['algorithms'].items():
1291             logger.info(
1292                 f"[cv] {algo}: RMSE={m['rmse_mean']:.4f}±{m['rmse_std']:.4f}, "
1293                 f"MAE={m['mae_mean']:.4f}±{m['mae_std']:.4f}, "
1294                 f"tiempo={m['avg_execution_time_ms']:.0f}ms/fold"
1295             )
1296         logger.info(f"[cv] Cross-validation completada en {cv_result['total_execution_time_ms']:.0f}ms.")
1297     except Exception as exc:
1298         logger.error(f"[cv] Error en cross-validation: {exc}", exc_info=True)
1299     finally:
1300         _cv_in_progress = False
1301
1302
1303 @app.post("/cross-validation")
1304 def cross_validation_endpoint(background_tasks: BackgroundTasks, k: int = 5, sample_size: int = 3000):
1305     """
1306     Inicia k-fold cross-validation de CF/RF/GBM en background. Guarda el
1307     resultado en models/cv_metrics.json y en la tabla
1308     ml_cross_validation_metrics. Consultar el resultado con GET /cross-validation.
1309     """
1310     if engine is None:
1311         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1312     if _cv_in_progress:
1313         return {"ok": False, "message": "Cross-validation ya en curso."}
1314     background_tasks.add_task(_run_cross_validation, k, sample_size)
1315     return {"ok": True, "message": f"Cross-validation iniciada en background (k={k}, sample_size={sample_size})"}
1316
1317
1318 @app.get("/cross-validation")
1319 def get_cross_validation():
1320     """Devuelve el último resultado de cross-validation guardado en disco."""
1321     cv_path = Path(_MODELS) / "cv_metrics.json"
1322     if not cv_path.exists():
1323         raise HTTPException(status_code=404, detail="Sin resultados de cross-validation aún. Ejecuta POST /cross-validation primero.")
1324     with open(cv_path, 'r', encoding='utf-8') as f:
1325         return json.load(f)
1326
1327
1328 @app.post("/train-rf")
1329 def train_rf():
1330     """
1331     Llamado bajo demanda de re-entrenamiento del Random Forest contextual.
1332     Esto vuelve a generar el árbol utilizando datos actuales de la Base e ignora el archivo local
1333     previo,
1334     creando un fichero `.joblib` en disco en los procesos intermedios que quedará para los futuros
1335     arranques.
1336     """
1337     if engine is None or context_model is None:
1338         raise HTTPException(status_code=503, detail="Modelos no cargados.")
1339     try:
1340         logger.info("Forzando entrenamiento del modelo Random Forest...")

```

```

1339         __, restmex_biz = _get_restmex_data()
1340         context_model.train(engine.train_data, df_biz_extra=restmex_biz)
1341         return {"status": "ok", "message": "Modelo entrenado y guardado correctamente."}
1342     except Exception as e:
1343         logger.error(f"Error entrenando modelo: {e}")
1344         raise HTTPException(status_code=500, detail=str(e))
1345
1346
1347 # — Route Optimizer (ACO) —————
1348
1349 class OptimizeStop(BaseModel):
1350     lat: float
1351     lng: float
1352     name: str = ''
1353     duration_minutes: Optional[int] = None    # dwell time at this stop
1354     open_minutes: Optional[int] = None        # opening time in minutes from midnight
1355     close_minutes: Optional[int] = None       # closing time in minutes from midnight
1356
1357 class OptimizeRouteRequest(BaseModel):
1358     stops: List[OptimizeStop]
1359     start_minutes: int = 540 # departure time in minutes from midnight (default 09:00)
1360
1361
1362 @app.post("/optimize-route")
1363 def optimize_route_endpoint(req: OptimizeRouteRequest):
1364     """
1365     Ant Colony Optimization for itinerary stop ordering with time-window support.
1366     Keeps the first stop fixed as departure and finds the shortest open path
1367     through the remaining stops.
1368
1369     Optional per-stop fields: duration_minutes, open_minutes, close_minutes.
1370     Optional request field: start_minutes (default 540 = 09:00).
1371
1372     Returns:
1373         optimized_order (indices into input stops),
1374         original_distance_km, optimized_distance_km, savings_pct.
1375     """
1376     if len(req.stops) < 2:
1377         raise HTTPException(
1378             status_code=422,
1379             detail="Se necesitan al menos 2 paradas para optimizar",
1380         )
1381     stops = [
1382         {
1383             "lat": s.lat,
1384             "lng": s.lng,
1385             "name": s.name,
1386             "duration_minutes": s.duration_minutes,
1387             "open_minutes": s.open_minutes,
1388             "close_minutes": s.close_minutes,
1389         }
1390         for s in req.stops
1391     ]
1392     result = _aco_optimize(stops, start_minutes=req.start_minutes)
1393     return result
1394
1395
1396 # — WellTur – Wellness Tourism Endpoints —————
1397
1398

```

```

1399 class WellnessAssessmentRequest(BaseModel):
1400     q1: int = Field(..., ge=1, le=4, description="Energía cognitiva (1=muy alta, 4=muy baja)")
1401     q2: int = Field(..., ge=1, le=4, description="Tensión física")
1402     q3: int = Field(..., ge=1, le=4, description="Rumiación / pensamientos recurrentes")
1403     q4: int = Field(..., ge=1, le=4, description="Activación negativa / nerviosismo")
1404     top_n: int = Field(default=3, ge=1, le=10)
1405     user_preferences: Optional[Dict[str, Any]] = None
1406     region_filter: Optional[str] = None # e.g. "Veracruz"
1407
1408
1409 class WellnessDestinationItem(BaseModel):
1410     id_destino: str
1411     nombre_lugar: str
1412     estado: str
1413     categoria_wellness: str
1414     match_pct: float
1415     beneficio_optimo_pct: float
1416     alineacion_pct: float
1417     wellness_sentiment_score: float
1418     rank: int
1419     nivel_aislamiento: float
1420     restauracion_pasiva: float
1421     demanda_fisica: float
1422     lat: Optional[float] = None
1423     lon: Optional[float] = None
1424     descripcion_bienestar: str
1425     beneficio_descripcion: str
1426
1427
1428 class WellnessAssessmentResponse(BaseModel):
1429     perfil_interno: str
1430     modo_viaje: str # 'modo_calma' | 'modo_restauracion' | 'modo_equilibrio'
1431     modo_viaje_label: str # 'Modo Calma' | 'Modo Restauración' | 'Modo Equilibrio'
1432     modo_viaje_descripcion: str
1433     confianza: float
1434     metodo: str
1435     destinations: List[WellnessDestinationItem]
1436
1437
1438 _wellness_destinations_cache: Optional[Any] = None
1439
1440
1441 def _get_wellness_destinations():
1442     global _wellness_destinations_cache
1443     if _wellness_destinations_cache is None:
1444         try:
1445             from wellness_matchmaker import load_destinations
1446             _wellness_destinations_cache = load_destinations()
1447         except Exception as e:
1448             logger.warning(f"[wellness] No se pudo cargar destinos: {e}")
1449             import pandas as pd
1450             _wellness_destinations_cache = pd.DataFrame()
1451     return _wellness_destinations_cache
1452
1453
1454 @app.post("/wellness/assess", response_model=WellnessAssessmentResponse)
1455 def wellness_assess(payload: WellnessAssessmentRequest):
1456     """
1457     Clasifica el perfil de vitalidad del usuario (Q1-Q4) y retorna
1458     las top-N recomendaciones de destinos wellness.

```

```

1459     Nunca expone el nombre técnico interno en modo_viaje.
1460     """
1461     try:
1462         from wellness_classifier import (
1463             get_classifier,
1464             MOD0_VIAJE_LABELS,
1465             MOD0_VIAJE_DESCRIPTION,
1466         )
1467         from wellness_matchmaker import recommend_wellness
1468
1469         clf = get_classifier()
1470         perfil, modo, proba_map, confianza, metodo = clf.predict(
1471             payload.q1, payload.q2, payload.q3, payload.q4
1472         )
1473
1474         destinations_df = _get_wellness_destinations()
1475         recs = recommend_wellness(
1476             destinations=destinations_df,
1477             perfil=perfil,
1478             q1=payload.q1,
1479             q2=payload.q2,
1480             q3=payload.q3,
1481             q4=payload.q4,
1482             top_n=payload.top_n,
1483             stress_confidence=confianza,
1484             user_preferences=payload.user_preferences,
1485             region_filter=payload.region_filter,
1486         )
1487
1488         return WellnessAssessmentResponse(
1489             perfil_interno=perfil,
1490             modo_viaje=modo,
1491             modo_viaje_label=MOD0_VIAJE_LABELS.get(modo, modo),
1492             modo_viaje_description=MOD0_VIAJE_DESCRIPTION.get(modo, ""),
1493             confianza=round(confianza, 3),
1494             metodo=metodo,
1495             destinations=[WellnessDestinationItem(**r) for r in recs],
1496         )
1497     except Exception as e:
1498         logger.error(f"[wellness] Error en assessment: {e}")
1499         raise HTTPException(status_code=500, detail=str(e))
1500
1501
1502 @app.get("/wellness/destinations")
1503 def wellness_destinations(
1504     estado: Optional[str] = Query(None),
1505     categoria: Optional[str] = Query(None),
1506     approved_only: bool = Query(True),
1507 ):
1508     """
1509     Lista destinos wellness disponibles en el catálogo.
1510     Si approved_only=True, solo retorna los aprobados por el admin.
1511     """
1512     try:
1513         df = _get_wellness_destinations()
1514         if df.empty:
1515             return {"destinations": []}
1516
1517         if estado:
1518             df = df[df.get("estado", "").str.lower() == estado.lower()]

```

```

1519         if categoria:
1520             cat_col = "categoria_wellness" if "categoria_wellness" in df.columns else "categoria_p
principal"
1521             df = df[df[cat_col].str.lower() == categoria.lower()]
1522             if approved_only and "wellness_status" in df.columns:
1523                 df = df[df["wellness_status"] == "approved"]
1524
1525             return {"destinations": df.to_dict(orient="records"), "total": len(df)}
1526     except Exception as e:
1527         raise HTTPException(status_code=500, detail=str(e))
1528
1529
1530 @app.get("/wellness/pending-count")
1531 def wellness_pending_count():
1532     """
1533     Conteo de servicios/POIs pendientes de validación wellness.
1534     Usada por AdminBadgesContext para el badge del sidebar.
1535     """
1536     try:
1537         import psycopg2
1538         conn = psycopg2.connect(
1539             host=os.getenv("POI_DB_HOST", "localhost"),
1540             port=int(os.getenv("POI_DB_PORT", 5432)),
1541             database=os.getenv("POI_DB_NAME", "smartur"),
1542             user=os.getenv("POI_DB_USER", "postgres"),
1543             password=os.getenv("POI_DB_PASSWORD", os.getenv("DB_PASSWORD", "")),
1544         )
1545         with conn.cursor() as cur:
1546             cur.execute(
1547                 "SELECT COUNT(*) FROM tourist_service WHERE wellness_status = 'pending'"
1548             )
1549             ts_count = cur.fetchone()[0]
1550             cur.execute(
1551                 "SELECT COUNT(*) FROM point_of_interest WHERE wellness_status = 'pending'"
1552             )
1553             poi_count = cur.fetchone()[0]
1554             conn.close()
1555             return {"total_pending": ts_count + poi_count, "services": ts_count, "pois": poi_count}
1556     except Exception as e:
1557         logger.warning(f"[wellness] pending-count DB error: {e}")
1558         return {"total_pending": 0, "services": 0, "pois": 0}
1559
1560
1561 @app.post("/wellness/train")
1562 def wellness_train(background_tasks: BackgroundTasks):
1563     """Re-entrena el clasificador de perfil wellness en background."""
1564     def _train():
1565         try:
1566             from wellness_classifier import WellnessProfileClassifier
1567             clf = WellnessProfileClassifier()
1568             metrics = clf.train()
1569             logger.info(f"[wellness-train] Completado: accuracy={metrics.get('accuracy', '?'):.3f}")
1570         except Exception as e:
1571             logger.error(f"[wellness-train] Error: {e}")
1572
1573     background_tasks.add_task(_train)
1574     return {"ok": True, "message": "Entrenamiento wellness iniciado en background"}
1575
1576

```

```
1577 @app.get("/wellness/metrics")
1578 def wellness_metrics():
1579     """Devuelve métricas del clasificador wellness guardadas en meta.json."""
1580     meta_path = Path("../models/wellness/wellness_profile.meta.json")
1581     if not meta_path.exists():
1582         raise HTTPException(
1583             status_code=404,
1584             detail="Modelo no entrenado aún. Ejecuta POST /wellness/train primero.",
1585         )
1586     with open(meta_path, encoding="utf-8") as f:
1587         meta = json.load(f)
1588     return {
1589         "classifier": {
1590             "accuracy": meta.get("accuracy"),
1591             "macro_f1": meta.get("macro_f1"),
1592             "classification_report": meta.get("classification_report", {}),
1593             "trained_at": meta.get("trained_at"),
1594             "n_samples": meta.get("n_samples"),
1595             "dataset": meta.get("dataset", "synthetic"),
1596         },
1597         "disclaimer": (
1598             "Métricas generadas sobre datos sintéticos (5,000 registros ATARAXIA). "
1599             "No reflejan desempeño con usuarios reales hasta acumular feedback de fit_rating."
1600         ),
1601     }
1602
1603
1604 if __name__ == "__main__":
1605     import uvicorn
1606     uvicorn.run(app, host="0.0.0.0", port=8000)
```